

UNIVERSITAS GUNADARMA
FAKULTAS ILMU KOMPUTER DAN TEKNOLOGI INFORMASI



PENULISAN ILMIAH

**Analisis Komparasi Model Machine Learning untuk Prediksi Pemenang
Formula 1 : Studi Kasus 2025 Mexico City Grand Prix**

DISUSUN OLEH :

NAMA : Kevin Budiawan Sidarta
NPM : 10123585
PROGRAM STUDI : Sistem Informasi
PEMBIMBING : Widya Khafa Nofa, S.Kom., MMSI

JAKARTA

2026

PERNYATAAN ORIGINALITAS DAN PUBLIKASI

Saya yang bertanda tangan di bawah ini,

Nama : Kevin Budiawan Sidarta
NPM : 10123585
Judul PI : Analisis Komparasi Model Machine Learning untuk
Prediksi Pemenang Formula 1 : Studi Kasus 2025
Mexico City Grand Prix
Tanggal Sidang : Juli 2026
Tanggal Lulus : Juli 2026

menyatakan bahwa tulisan ini adalah merupakan hasil karya saya sendiri dan dapat dipublikasikan sepenuhnya oleh Universitas Gunadarma. Segala kutipan dalam bentuk apa pun telah mengikuti kaidah, etika yang berlaku. Mengenai isi dan tulisan adalah merupakan tanggung jawab Penulis, bukan Universitas Gunadarma. Demikian pernyataan ini dibuat dengan sebenarnya dan dengan penuh kesadaran.

Jakarta, 29 Juni 2026

(Kevin Budiawan Sidarta)

LEMBAR PENGESAHAN

Judul Penulisan Ilmiah : Analisis Komparasi Model Machine Learning untuk
Prediksi Pemenang Formula 1 : Studi 2025 *Mexico*
City Grand Prix

Nama : Kevin Budiawan Sidarta

NPM : 10123585

Program Studi : Sistem Informasi

Tanggal Sidang : Juli 2026

Tanggal Lulus : Juli 2026

Mengetahui,

Pembimbing

Kepala Subbagian Sidang PI

(Widya Khafa Nofa, S.Kom., MMSI)

(Dr. Sri Nawangsari, SE., MM., M.I.Kom.)

Ketua Program Studi

(Dr. Naeli Umniati, ST., MMSI.)

ABSTRAK

Kevin Budiawan Sidarta, 10123585

ANALISIS KOMPARASI MODEL MACHINE LEARNING UNTUK PREDIKSI PEMENANG FORMULA 1 : STUDI KASUS 2025 *MEXICO CITY GRAND PRIX*
Tulisan Ilmiah. Sistem Informasi. Fakultas Ilmu Komputer dan Teknologi Informasi. Universitas Gunadarma. 2026

Kata kunci : Analisis Komparatif, Formula 1, *Machine Learning*, *hyperparameter tuning*, FastF1

(xii + 66 + Lampiran)

Formula 1 merupakan ajang balap yang memiliki tingkat persaingan tinggi, di mana hasil balapan dipengaruhi oleh berbagai faktor seperti performa mobil, strategi tim, dan kemampuan pembalap. Kompleksitas tersebut menjadikan prediksi hasil balapan sebagai topik yang menarik untuk dikaji menggunakan machine learning. Penelitian ini bertujuan membandingkan kinerja lima algoritma, yaitu Random Forest, Gradient Boosting, XGBoost, CatBoost, dan LightGBM, dalam memprediksi posisi finis pembalap pada Formula 1 Mexico City Grand Prix 2025. Data diperoleh melalui library FastF1 yang menyediakan informasi telemetri, lap time, dan data sesi balapan. Data latih menggunakan musim 2022–2024 dengan total 60 data dan 42 variabel, sedangkan data musim 2025 digunakan sebagai data uji yang mencakup 20 pembalap. Penelitian dilaksanakan berdasarkan tahapan CRISP-DM, dengan setiap model dioptimalkan menggunakan hyperparameter tuning berbasis Optuna. Hasil pengujian menunjukkan bahwa XGBoost memberikan performa terbaik dengan mampu memprediksi posisi finis pembalap pada peringkat P1 hingga P6 secara tepat. Temuan ini menunjukkan bahwa algoritma boosting lebih efektif dalam mempelajari pola pada data balapan Formula 1 dan berpotensi menjadi dasar pengembangan sistem prediksi hasil balapan yang lebih akurat pada penelitian selanjutnya.

ABSTRACT

Kevin Budiawan Sidarta, 10123585

COMPARATIVE ANALYSIS OF MACHINE LEARNING MODELS FOR FORMULA 1 RACE WINNER PREDICTION: A CASE STUDY OF THE 2025 MEXICO CITY GRAND PRIX

Scientific Writing. Information Systems. Faculty of Computer Science and Information Technology. Universitas Gunadarma. 2026

Keywords : Comparative Analysis, Formula 1, Machine Learning, hyperparameter tuning, FastF1

(xiv + 66 + Appendix)

Formula 1 is one of the world's most competitive motorsport events, where race outcomes are influenced by various factors, including car performance, team strategy, and driver skill. This complexity makes race outcome prediction an interesting application of machine learning. This study aims to compare the performance of five machine learning algorithms, namely Random Forest, Gradient Boosting, XGBoost, CatBoost, and LightGBM, in predicting drivers' finishing positions at the 2025 Formula 1 Mexico City Grand Prix. The dataset was collected using the FastF1 library, which provides telemetry, lap time, and race session data. Training data consisted of the 2022–2024 seasons with 60 records and 42 features, while the 2025 season, involving 20 drivers, was used as the testing dataset. The study followed the CRISP-DM methodology, and each model was optimized through Optuna-based hyperparameter tuning. The results show that XGBoost achieved the best performance by accurately predicting the finishing positions from first to sixth place (P1–P6). These findings indicate that boosting-based algorithms are more effective in capturing complex patterns in Formula 1 race data and have the potential to support the development of more accurate race outcome prediction systems in future research.

KATA PENGANTAR

Puji dan Syukur dipanjatkan ke hadirat Allah SWT, atas berkah rahmat dan karunia-Nya, serta doa dan motivasi dari berbagai pihak sehingga dapat terselesaikannya tugas Penulisan Ilmiah yang berjudul “ANALISIS KOMPARASI MODEL MACHINE LEARNING UNTUK PREDIKSI PEMENANG FORMULA 1 : STUDI KASUS 2025 *MEXICO CITY GRAND PRIX*” dengan sebaik-baiknya. Penyelesaian Penulisan Ilmiah ini tercapai berkat dorongan dan bantuan dari berbagai pihak. Oleh karena itu, penulis menyampaikan terima kasih kepada:

1. Prof. Dr. Hj. E. S. Margianti, SE, MM., selaku rektor Universitas Gunadarma.
2. Prof. Dr. Tubagus Maulana Kusuma, S.Kom., MEngSc., selaku Dekan Fakultas Ilmu Komputer dan Teknologi Informasi Universitas Gunadarma.
3. Dr. Naeli Umniati, ST., MMSI., selaku Ketua Program Studi Sistem Informasi Universitas Gunadarma.
4. Dr. Sri Nawangsari, SE., MM., M.I.Kom., selaku Kepala Subbagian Sidang Penulisan Ilmiah Fakultas Ilmu Komputer dan Teknologi Informasi Universitas Gunadarma.
5. Ibu Widya Khafa Nofa, S.Kom., MMSI, selaku dosen pembimbing yang telah banyak memberikan waktu dan sarannya selama pembuatan Penulisan Ilmiah ini.
6. Perkaraserta restu mereka terselesaikanlah Penulisan Ilmiah ini.
7. Teman-teman 3KA05 serta pihak lainnya yang secara langsung ikut memberikan dukungan.

Penulisan Ilmiah ini diharapkan dapat bermanfaat bagi aktivitas akademis Sistem Informasi Fakultas Ilmu Komputer dan Teknologi Informasi Universitas Gunadarma, dan menyadari atas segala keterbatasan pengetahuan dan kemampuan diharapkan adanya kritik yang membangun dan semoga Penulisan Ilmiah ini dapat bermanfaat untuk orang banyak.

Jakarta, 29 Juni 2026

(Kevin Budiawan Sidarta)

DAFTAR ISI

PERNYATAAN ORIGINALITAS DAN PUBLIKASI	ii
LEMBAR PENGESAHAN	iii
ABSTRAK	iv
ABSTRACT	v
KATA PENGANTAR.....	vi
DAFTAR ISI.....	viii
DAFTAR TABEL	xi
DAFTAR GAMBAR.....	xi
DAFTAR LAMPIRAN	xiv
1. PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Ruang Lingkup.....	2
1.3 Tujuan Penelitian	3
1.4 Metode Penelitian.....	3
1.5 Sistematika Penulisan	4
2. TINJAUAN PUSTAKA	5
2.1 Formula 1 & Data Balapan	5
2.2 Machine Learning	6
2.3 Supervised Learning.....	7
2.4 <i>Cross Industry Standard Process for Data Mining (CRISP-DM)</i>	7
2.5 Supervised Learning.....	9
2.6 Preprocessing Data.....	10
2.6.1 Pembersihan Data (<i>Data Cleaning</i>)	10
2.6.2 Integrasi Data (<i>Data Integration</i>).....	11
2.6.3 <i>Feature Engineering</i>	12
2.6.4 Train-Test-Split	12

2.6.5	Data Scaling	13
2.7	Random Forest	13
2.8	Gradient Boosting	14
2.9	XGBoost.....	15
2.10	CatBoost.....	16
2.11	Hyperparameter Tuning	17
2.11.1	GridSearchCV	18
2.11.2	Bayesian Optimization	18
2.11.3	Optuna	19
2.12	Python	19
2.13	Google Colaboratory.....	20
2.14	FastF1 API	20
2.15	Metrik Evaluasi	21
2.15.1	Mean Absolute Error (MAE)	22
2.15.2	Root Mean Squarred Error	22
2.15.3	Koefisien Determinan (R^2).....	23
2.16	Penelitian Terdahulu	23
3.	PEMBAHASAN.....	27
3.1	Gambaran Umum	27
3.2	Pemahaman Bisnis (<i>Business Understanding</i>)	27
3.3	Pemahaman Data (<i>Data Understanding</i>).....	28
3.4	Persiapan Data (<i>Data Preparation</i>)	38
3.5.1	Pengambilan Data (<i>Fetching Data</i>)	38
3.5.2	Membaca Dataset (<i>Load Dataset</i>).....	40
3.5.3	Identifikasi Nilai Hilang (<i>Finding Missing Values</i>).....	41

3.5.4	Rekayasa Fitur (<i>Feature Engineering</i>) dan Fitur Tambahan	41
3.5.5	Integrasi Fitur Hasil Rekayasa	47
3.5.6	Pembagian Data Training dan Testing (<i>Train-Test-Split</i>).....	48
3.5.7	Penskalaan Data (<i>Data Scaling</i>).....	49
3.5	Modelling	50
3.6.1	Random Forest	50
3.6.2	Gradient Boosting	54
3.6.3	XGBoost.....	59
3.6.4	CatBoost.....	63
3.6.5	LightGBM.....	67
3.6	<i>Evaluation</i> (Evaluasi).....	72
3.7	Tabel Rangkuman Hasil Evaluasi	75
3.7.1	Tabel Perbandingan Metrik Evaluasi Regresi.....	76
3.7.2	Tabel Perbandingan Nilai Prediksi dengan Nilai Aktual	77
3.7.3	Kesimpulan Tabel Evaluasi.....	90
4.	PENUTUP	93
4.1	Kesimpulan	93
4.2	Saran.....	93
	DAFTAR PUSTAKA	95

DAFTAR GAMBAR

Gambar 2.1	Tahapan CRISP-DM	8
Gambar 3.1	Diagram alur penelitian.....	27
Gambar 3.2	Tampilan FastF1 API	29
Gambar 3.3	Tampilan 10 Data Teratas	29
Gambar 3.4	Kode Visualisai Performa <i>Driver</i> dan Poin (2022-2025)	30
Gambar 3.5	Visualisasi performa <i>driver</i>	31
Gambar 3.6	Kode Visualisasi Lap Time, Sektor & Kualifikasi (2022-2025)...	32
Gambar 3.7	Visualisasi <i>Laptime</i> Sektoral.....	32
Gambar 3.8	Kode Visualisasi Analisis Sirkuit & Kondisi Lingkungan	33
Gambar 3.9	Visualisasi kondisi sirkuit	34
Gambar 3.10	Kode Visualisasi Feature Importance	35
Gambar 3.11	Visualiasi <i>feature importance</i>	35
Gambar 3.12	Kode Visualisasi Performa Tim pada Tahun 2024	36
Gambar 3.13	Visualisasi Performa Tim (2024).....	37
Gambar 3.14	Kode Program untuk Mengambil Data	38
Gambar 3.15	Tampilan Dataset yang Telah Digabungkan.....	40
Gambar 3.16	Membaca Dataset	40
Gambar 3.17	Kode untuk Identifikasi Nilai Hilang.....	41
Gambar 3.18	Kode untuk Tambahan Fitur Karakteristik Sirkuit.....	42
Gambar 3.19	Kode untuk Tambahan Fitur Kondisi Cuaca.....	43
Gambar 3.20	Kode Feature Engineering Performa <i>Driver</i> dan Tim.....	43
Gambar 3.21	Kode untuk Tambahan Fitur Efek Ketinggian	45
Gambar 3.22	Kode untuk Rekayasa Fitur Performa dan <i>Grid Position</i>	46
Gambar 3.23	Kode untuk Integrasi Fitur	47
Gambar 3.24	Kode dan Output Ringkasan Fitur	48
Gambar 3.25	Kode Train-Test Split.....	49
Gambar 3.26	Kode Data Scaling.....	49
Gambar 3.27	Tampilan Data Setelah Scaling	50
Gambar 3.28	Kode Pelatihan Random Forest.....	51
Gambar 3.29	Kode Pelatihan Random Forest (Optimasi <i>GridSearchCV</i>).....	51
Gambar 3.30	Proses Instalasi <i>scikit-optimize</i>	52
Gambar 3.31	Kode untuk <i>import BayesSearchCV</i> dari <i>library</i> skopt	53

Gambar 3.32	Kode Pelatihan Random Forest (Bayesian Optimization)	53
Gambar 3.33	Kode <i>Import Model Gradient Boosting Regressor</i>	54
Gambar 3.34	Kode Pelatihan Model Gradient Boosting (Default Parameter) ...	54
Gambar 3.35	Kode Parameter GridSearchCV Gradient Boosting.....	55
Gambar 3.36	Kode <i>Hyperparameter</i> GradientBoost GridSearchCV	56
Gambar 3.37	Kode Pelatihan Gradient Boost (GridSearchCV)	56
Gambar 3.38	Proses Instalasi Optuna	57
Gambar 3.39	Kode <i>Import Library</i> Optuna	57
Gambar 3.40	Kode Ruang Pencarian Parameter Optuna	58
Gambar 3.41	Kode untuk Objek Study Optuna	58
Gambar 3.42	Kode Pelatihan Gradient Boosting (Optuna Parameter)	58
Gambar 3.43	Kode <i>Import Model</i> XGBRegressor	59
Gambar 3.44	Kode Pelatihan Model XGBRegressor	59
Gambar 3.45	Proses Optimasi Hyperparameter XGBoost (Optuna)	60
Gambar 3.46	Kode Pelatihan Model XGBoost dengan Parameter Optuna	61
Gambar 3.47	Pendefinisian <i>Hyperparameter Grid</i> Model XGBoost	61
Gambar 3.48	Kode <i>Hyperparameter</i> XGBoost (GridSearchCV)	62
Gambar 3.49	Kode Pelatihan XGBoost (GridSearchCV).....	62
Gambar 3.50	Proses Instalasi Model CatBoost.....	63
Gambar 3.51	Kode <i>Import Model</i> CatBoostRegressor	63
Gambar 3.52	Kode Pelatihan Model CatBoost (<i>Default Parameters</i>).....	64
Gambar 3.53	Pendefinisian <i>Hyperparameter Grid</i> Model CatBoost	64
Gambar 3.54	Pencarian Parameter Terbaik oleh GridSearchCV (CatBoost)	65
Gambar 3.55	Kode Pelatihan Model CatBoost (GridSearchCV)	65
Gambar 3.56	Kode <i>Objective Function</i> Optuna CatBoost.....	66
Gambar 3.57	Proses Optimasi <i>Hyperparameter</i> Model CatBoost (Optuna)	66
Gambar 3.58	Kode Pelatihan Model CatBoost (Optuna)	67
Gambar 3.59	Proses Import Model LGBMRegressor	68
Gambar 3.60	Kode Pelatihan Model LightGBM (<i>Default Parameter</i>)	68
Gambar 3.61	Pendefinisian <i>Hyperparameter Grid</i> Model LightGBM	68
Gambar 3.62	Kode <i>Hyperparameter</i> LightGBM (GridSearchCV)	69
Gambar 3.63	Kode Pelatihan Model LightGBM (GridSearchCV).....	70
Gambar 3.64	Kode <i>Objective Function</i> Optuna LightGBM.....	70
Gambar 3.65	Kode Pencarian Parameter Terbaik oleh Optuna	71

Gambar 3.66	Proses Pencarian Parameter Terbaik oleh Optuna	71
Gambar 3.67	Kode Pelatihan Model LightGBM (Optuna)	72
Gambar 3.68	Kode Penambahan Hasil Prediksi ke Data Testing.....	73
Gambar 3.69	Kode Perbandingan Hasil Prediksi dan Nilai Aktual 2025.....	73
Gambar 3.70	Output Kode Nilai Prediksi dan Nilai Aktual	74
Gambar 3.71	Kode Impor Metrik Evaluasi Regresi	74
Gambar 3.72	Kode Perhitungan Metrik Evaluasi Regresi.....	74
Gambar 3.73	Kode Tampilan untuk Hasil Perhitungan Metrik.....	75

DAFTAR TABEL

Tabel 2.1	Tabel Penelitian Terdahulu	24
Tabel 3.1	Informasi Dataset yang Diambil	39
Tabel 3.2	Agregasi Data per Pembalap per Musim	44
Tabel 3.3	Tabel Rangkuman Metrik Evaluasi Regresi	76
Tabel 3.4	Tabel Daftar Singkatan Nama Pembalap	78
Tabel 3.5	Tabel Perbandingan Random Forest	79
Tabel 3.6	Tabel Perbandingan Gradient Boosting	81
Tabel 3.7	Tabel Perbandingan XGBoost.....	83
Tabel 3.8	Tabel Perbandingan CatBoost.....	85
Tabel 3.9	Tabel Perbandingan LightGBM.....	87

DAFTAR LAMPIRAN

Lampiran 1. Lembar Pernyataan Ujicoba	L-1
Lampiran 2. Listing Program	L-2
Lampiran 3. Output Program	L-42
Lampiran 4. E-KRS.....	L-45
Lampiran 5. Surat Persetujaun ACC.....	L-46
Lampiran 6. Isian Sertifikat Setara Sarjana Muda	L-47

1. PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi mengalami pertumbuhan yang sangat pesat, salah satunya dalam bidang kecerdasan buatan (*Artificial Intelligence/AI*) dan *machine learning*. Teknologi ini menunjukkan bahwa komputer mampu mempelajari pola dari data yang ada dan menghasilkan sebuah output seperti prediksi atau pengambilan keputusan secara otomatis (An-Nur. 2023). Pemanfaatan *machine learning* telah meluas ke berbagai sektor, termasuk kesehatan, bisnis, pendidikan, hingga bidang olahraga.

Formula 1 sebagai salah satu olahraga balapan paling bergengsi di dunia juga tidak terlepas dari pemanfaatan teknologi dan data (BINUS SIS, 2024). Ajang balap mobil yang dikelola oleh *Fédération Internationale de l'Automobile* (FIA) menghasilkan data dalam jumlah besar setiap balapan-nya seperti performa pembalap, telemetri mobil, hingga kondisi eksternal seperti cuaca dan kondisi sirkuit. Kompleksitas dari banyaknya variabel yang ditunjukkan tersebut mempengaruhi setiap detiknya balapan berlangsung, yang menjadikan prediksi pemenang sebagai suatu tantangan yang menarik untuk diselesaikan.

Machine learning dapat digunakan untuk mengolah data historis balapan guna menganalisis dan memprediksi hasil balapan yang akurat. Berbagai algoritma machine learning memiliki kemampuan yang berbeda dalam menangani dan menghasilkan model prediksi. Oleh karena itu, diperlukan analisis komparatif untuk membandingkan model mana yang memiliki performa terbaik dalam memprediksi hasil balapan.

2025 *Mexico City Grand Prix* dijadikan studi kasus pada penelitian ini. Sirkuit yang memiliki nama Autódromo Hermanos Rodríguez ini merupakan sirkuit dengan ketinggian tertinggi dalam kalender Formula 1 yaitu $\pm 2.200\text{--}2.285$ meter di atas permukaan laut yang menyebabkan penurunan densitas udara $\sim 20\text{--}25\%$ (F1Technical, 2025), ini menjadi tantangan tersendiri bagi model *machine learning* untuk mempelajari kompleksitas data guna menghasilkan prediksi yang akurat.

1.2 Ruang Lingkup

Penelitian ini memiliki beberapa batasan yang perlu diperhatikan. Pertama, keterbatasan ketersediaan data historis bagi pembalap baru (rookie) yang baru berpartisipasi pada musim berjalan, sehingga rekam jejak performa mereka dalam dataset tidak merepresentasikan kapabilitas aktual secara menyeluruh. Kedua, data historis rentan terhadap dampak perubahan regulasi yang diberlakukan Formula 1 dari tahun ke tahun, sehingga penelitian ini membatasi penggunaan data pada tiga musim sebelumnya, yakni musim 2022 hingga 2024, sebagai data latih (training data) dengan total 60 baris dan 11 kolom, serta 20 data untuk musim 2025 sebagai data uji (testing data). Ketiga, keterbatasan sumber daya komputasi yang tersedia menyebabkan penelitian ini hanya mampu melatih lima model machine learning dalam proses eksperimen, sehingga cakupan perbandingan algoritma tidak dapat diperluas secara lebih komprehensif.

Subjek penelitian mencakup seluruh 20 pembalap yang berpartisipasi pada balapan Mexico City Grand Prix 2025. Data diperoleh melalui library FastF1, yang menyediakan data telemetri, lap time, dan informasi sesi balapan secara terstruktur dalam format DataFrame. Variabel atau fitur yang dianalisis mencakup data pembalap (*sector time, race pace, best qualification time, grid position, dll*), performa tim, dan kondisi balapan (karakteristik sirkuit, kondisi cuaca, dll), dengan target fitur *LapTime(s)*/waktu yang dibutuhkan seorang pembalap untuk menyelesaikan satu putaran penuh di sirkuit.

Model *machine learning* yang dikomparasikan meliputi Random Forest, Gradient Boosting, XGBoost, CatBoost dan LightGBM. Kelima model tersebut dipilih karena merupakan model/algoritma ensemble berbasis *decision tree* yang banyak digunakan pada permasalahan klasifikasi dan prediksi. Selain memiliki performa yang tinggi, masing-masing algoritma memiliki mekanisme pembelajaran yang berbeda sehingga menarik untuk dibandingkan dalam memprediksi pemenang Formula 1 Mexico City Grand Prix. Proses *training* model akan dikerjakan menggunakan bahasa pemrograman Python dan Google Colab sebagai *interactive notebook environment* untuk melakukan pengolahan data dan pelatihan model. Model tersebut akan dievaluasi menggunakan metrik evaluasi regresi yang meliputi

Mean Absolute Error (MAE), *Mean Squarred Error* (MSE), *Root Mean Squarred Error* (RMSE) dan R^2 .

Output dari penelitian ini berupa tabel komparasi yang menyajikan perbandingan performa masing-masing model *machine learning* dalam memprediksi peringkat kemenangan tiap pembalap pada 2025 *Mexico City Formula 1 Grand Prix*. Tabel tersebut disusun dan disajikan menggunakan Microsoft Word.

1.3 Tujuan Penelitian

Tujuan penelitian ini adalah untuk membandingkan kinerja masing-masing model *machine learning* berdasarkan hasil evaluasi menggunakan metrik yang telah ditentukan. Hasil evaluasi tersebut disajikan dalam bentuk tabel komparatif menggunakan Microsoft Word, guna mengidentifikasi model yang paling unggul dalam memprediksi peringkat kemenangan setiap pembalap pada *Mexico City Formula 1 Grand Prix 2025*.

1.4 Metode Penelitian

Metode Penelitian yang digunakan dalam penulisan ilmiah ini menggunakan metode CRISP-DM (Cross-Industry Standard Process for Data Mining) (Raihani & Dwitama, 2025) yang melalui beberapa tahap sebagai berikut:

1. *Business Understanding*

Pada tahap ini peneliti memahami karakteristik kompetisi Formula 1, termasuk faktor-faktor yang mempengaruhi hasil balapan seperti telemetri mobil, posisi kualifikasi, kondisi cuaca, dan beberapa aspek lain.

2. *Data Understanding*

Pada tahap ini peneliti mengumpulkan data historis dari beberapa musim sebelumnya dari sumber *FastF1 Python Library*.

3. *Data Preparation*

Tahap ini berfokus pada transformasi data dan menyiapkan data untuk diproses seperti menangani nilai data yang hilang hingga memilih fitur-fitur yang paling relevan.

4. *Modeling*

Pada tahap ini beberapa model *machine learning* yang telah ditentukan sebelumnya akan dilatih menggunakan data yang telah disiapkan.

5. *Evaluation*

Pada tahap ini seluruh model *machine learning* yang telah dilatih dibandingkan secara sistematis menggunakan metrik evaluasi regresi yaitu *Mean Absolute Error* (MAE), *Mean Squarred Error* (MSE), *Root Mean Squarred Error* (RMSE) dan R^2 .

6. Deployment

Pada tahap ini tidak akan dilakukan secara penuh, namun diwujudkan dalam bentuk penyajian hasil komparasi model berbentuk tabel komparatif menggunakan Microsoft Word.

1.5 Sistematika Penulisan

Penelitian ini disusun dalam empat bab, yaitu Pendahuluan, Tinjauan Pustaka, Pembahasan, dan Penutup. Bab Pendahuluan berisi latar belakang, ruang lingkup, tujuan, metode penelitian, dan sistematika penulisan. Bab Tinjauan Pustaka memuat teori-teori pendukung serta penelitian terdahulu yang relevan. Bab Pembahasan menjelaskan proses pengambilan dan pengolahan data, pelatihan model *machine learning* (CatBoost, Random Forest, Gradient Boosting, LightGBM, dan XGBoost), serta evaluasi menggunakan metrik MAE, MSE, RMSE, dan R^2 untuk menentukan model terbaik dalam memprediksi pemenang Mexico City Formula 1 Grand Prix 2025. Bab Penutup berisi kesimpulan dan saran untuk penelitian selanjutnya.

2. TINJAUAN PUSTAKA

2.1 Formula 1 & Data Balapan

Formula 1 (F1) merupakan ajang balap mobil single-seater tertinggi di dunia yang dikelola oleh Fédération Internationale de l'Automobile (FIA) bersama Formula One Group. Kompetisi ini telah berlangsung sejak tahun 1950 dan kini menjadi salah satu olahraga dengan penonton terbanyak di dunia, menampilkan sekitar 20 balapan per musim yang digelar di berbagai negara. Setiap seri balapan, yang disebut Grand Prix, melibatkan 10 tim konstruktor dengan masing-masing dua pembalap, sehingga total terdapat 20 pembalap yang bersaing memperebutkan gelar Juara Dunia Pembalap (*World Drivers' Championship*) dan Juara Dunia Konstruktor (*World Constructors' Championship*) (FIA, 2024).

Sebagai olahraga berteknologi tinggi, Formula 1 menghasilkan volume data yang sangat besar dalam setiap sesi balapannya. Setiap mobil F1 dilengkapi dengan ratusan sensor yang secara kontinu merekam data telemetri seperti kecepatan, akselerasi, pengereman, tekanan ban, suhu mesin, dan konsumsi bahan bakar. Data ini dikirimkan secara real-time ke pit wall dan markas tim untuk dianalisis oleh para insinyur guna mendukung pengambilan keputusan strategis selama balapan berlangsung (BINUS SIS, 2024). Selain data telemetri, setiap balapan juga menghasilkan data lap time, sector time, posisi kualifikasi, hingga informasi kondisi eksternal seperti suhu udara, kelembaban, dan kondisi lintasan.

Salah satu sirkuit dengan karakteristik paling unik dalam kalender Formula 1 adalah Autódromo Hermanos Rodríguez yang berlokasi di Mexico City, Meksiko. Sirkuit ini berada pada ketinggian sekitar 2.200–2.285 meter di atas permukaan laut, menjadikannya sirkuit dengan elevasi tertinggi dalam seluruh kalender F1 (F1Technical, 2025). Kondisi geografis ini menyebabkan densitas udara di lokasi tersebut lebih rendah sekitar 20–25% dibandingkan sirkuit pada umumnya. Akibatnya, downforce aerodinamis yang dihasilkan oleh mobil berkurang secara signifikan, sistem pendingin bekerja lebih keras, dan mesin membutuhkan penyesuaian khusus agar dapat beroperasi secara optimal. Kombinasi faktor-faktor tersebut menciptakan dinamika balapan yang berbeda dari sirkuit lainnya, sehingga

menjadikan Mexico City Grand Prix sebagai studi kasus yang menantang sekaligus menarik untuk dianalisis menggunakan pendekatan machine learning.

Kompleksitas data yang dihasilkan oleh Formula 1, baik dari sisi volume maupun variasi variabelnya, mendorong pemanfaatan teknologi analisis data dan machine learning untuk mengekstrak wawasan yang lebih dalam. Pendekatan ini memungkinkan peneliti maupun tim balap untuk mengidentifikasi pola tersembunyi dalam data historis, memprediksi performa pembalap, serta mendukung pengambilan keputusan strategis yang lebih akurat. Oleh karena itu, ketersediaan data balapan yang terstruktur menjadi fondasi penting dalam pengembangan model prediksi pada penelitian ini.

2.2 Machine Learning

Machine learning adalah salah satu bidang dalam kecerdasan buatan yang memungkinkan komputer untuk mengenali pola dari data serta membuat prediksi atau keputusan secara otomatis tanpa memerlukan pemrograman ulang pada setiap kondisi yang dihadapi (Wijoyo dkk., 2024). Dalam prosesnya, machine learning bekerja dengan mengidentifikasi pola dari data yang tersedia, kemudian menggunakan pola tersebut untuk menghasilkan prediksi atau keputusan terhadap data baru. Kemampuan ini menjadikan machine learning sebagai pendekatan yang sangat relevan untuk menangani permasalahan dengan data berdimensi tinggi dan kompleks, seperti data balapan Formula 1 yang memiliki banyak variabel yang saling berinteraksi (Silalahi et al., 2023).

Secara umum, machine learning dibagi menjadi tiga paradigma utama, yaitu supervised learning, unsupervised learning, dan reinforcement learning. Supervised learning merupakan paradigma yang paling umum digunakan dalam permasalahan prediksi, di mana model dilatih menggunakan data yang telah memiliki label atau nilai target yang diketahui (Hastie et al., 2009). Dalam konteks penelitian ini, pendekatan supervised learning digunakan karena data historis balapan Formula 1

yang tersedia telah memiliki nilai target berupa lap time aktual dari setiap pembalap.

2.3 Supervised Learning

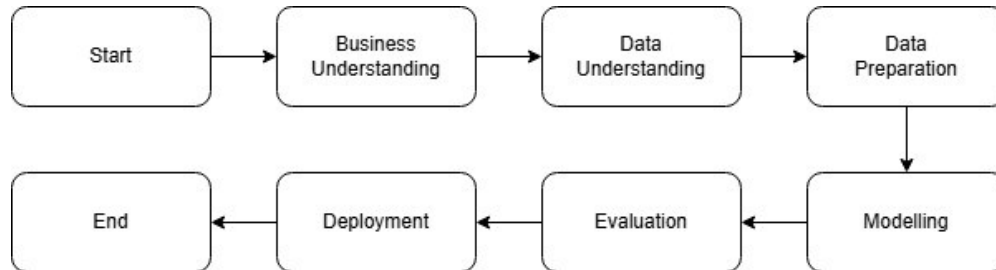
Supervised learning adalah pendekatan machine learning di mana model dilatih menggunakan dataset yang sudah memiliki label atau nilai target yang diketahui, sehingga model dapat mempelajari hubungan antara fitur input dan output yang diharapkan (Géron, 2019). Tujuan utamanya adalah agar model mampu memprediksi output yang tepat ketika diberikan data baru yang belum pernah dilihat sebelumnya. Supervised learning terbagi menjadi dua jenis tugas utama, yaitu klasifikasi yang memprediksi kategori atau kelas, dan regresi yang memprediksi nilai numerik secara kontinu.

Penelitian ini termasuk dalam kategori supervised learning dengan pendekatan regresi, karena variabel target yang diprediksi adalah LapTime (s), yaitu waktu tempuh pembalap per lap dalam satuan detik yang merupakan nilai numerik kontinu. Model dilatih menggunakan data historis balapan Formula 1 di Mexico City Grand Prix dari tahun 2022 hingga 2024, kemudian diuji kemampuannya dalam memprediksi lap time pada musim 2025. Dengan demikian, performa model tidak diukur berdasarkan ketepatan kelas, melainkan seberapa kecil selisih antara nilai prediksi dan nilai aktual menggunakan metrik seperti MAE dan RMSE.

2.4 *Cross Industry Standard Process for Data Mining (CRISP-DM)*

CRISP-DM (*Cross Industry Standard Process for Data Mining*) adalah kerangka kerja standar yang digunakan untuk memandu pelaksanaan proyek data mining secara sistematis dan terstruktur, tanpa terikat pada sektor industri maupun teknologi tertentu. Kerangka kerja ini memberikan alur yang jelas mulai dari tahap pemahaman masalah hingga penyajian hasil, sehingga proses penelitian berbasis data dapat berjalan secara konsisten dan terarah. CRISP-DM terdiri dari enam

tahapan yang saling berhubungan dan bersifat iteratif, di mana setiap tahapan dapat dikaji ulang apabila dibutuhkan penyesuaian selama proses berlangsung.



Gambar 2.1 Tahapan CRISP-DM

Keenam tahapan CRISP-DM diuraikan sebagai berikut :

1. *Business Understanding*

Tahap ini berfokus pada pemahaman konteks permasalahan dan penetapan tujuan penelitian, sehingga seluruh proses yang dijalankan memiliki arah yang jelas dan dapat diukur pencapaiannya.

2. *Data Understanding*

Tahap pengumpulan data awal sekaligus pemahaman terhadap karakteristik dan kualitas data yang akan digunakan, termasuk identifikasi potensi masalah pada data sejak tahap awal.

3. *Data Preparation*

Tahap pengolahan data mentah menjadi data yang siap dianalisis, meliputi penghapusan data yang tidak relevan, penanganan nilai kosong, serta berbagai proses transformasi dan standarisasi data.

4. *Modeling*

Tahap pemilihan dan penerapan teknik pemodelan yang sesuai terhadap data yang telah disiapkan, termasuk penyesuaian parameter model agar menghasilkan performa yang optimal.

5. *Evaluation*

Tahap pengukuran kinerja model menggunakan metrik evaluasi yang telah ditentukan, untuk memastikan model yang dibangun telah memenuhi tujuan penelitian sebelum hasil disajikan.

6. *Deployment*

Tahap penyajian hasil analisis dalam bentuk yang mudah dipahami dan dapat dimanfaatkan, sehingga temuan penelitian dapat dikomunikasikan secara efektif kepada pembaca.

2.5 Supervised Learning

Supervised learning adalah salah satu pendekatan dalam machine learning di mana model dilatih menggunakan dataset yang setiap datanya sudah memiliki label atau nilai output yang diketahui (Hartawan dkk., 2023). Proses pelatihan ini bertujuan agar model dapat mempelajari pola hubungan antara variabel input (fitur) dan variabel output (target), sehingga ketika dihadapkan pada data baru yang belum pernah dilihat sebelumnya, model dapat menghasilkan prediksi yang tepat.

Dalam proses pelatihannya, model supervised learning menerima pasangan data input dan output, kemudian secara iteratif menyesuaikan parameter internalnya untuk meminimalkan selisih antara nilai prediksi dan nilai aktual. Proses ini dikenal sebagai minimisasi fungsi loss atau fungsi kesalahan. Semakin kecil nilai loss yang dihasilkan selama pelatihan, maka semakin baik model dalam merepresentasikan pola yang terdapat dalam data. Namun demikian, model yang terlalu menyesuaikan diri dengan data latih berisiko mengalami overfitting, yaitu kondisi di mana model memiliki performa tinggi pada data latih tetapi buruk ketika diuji pada data baru.

Supervised learning mencakup dua jenis tugas utama yaitu klasifikasi dan regresi. Pada tugas regresi, model memprediksi nilai numerik yang bersifat kontinu, misalnya memprediksi harga rumah berdasarkan luas bangunan dan lokasi. Dalam penelitian ini, tugas yang diselesaikan termasuk ke dalam kategori regresi karena target prediksi yang digunakan adalah lap time pembalap dalam satuan detik.

Pendekatan supervised learning dipilih dalam penelitian ini karena data historis balapan Formula 1 yang dikumpulkan dari beberapa musim sebelumnya sudah memiliki nilai lap time aktual untuk setiap pembalap di setiap balapan. Ketersediaan label ini memungkinkan model untuk belajar dari pola data masa lalu

dan menggunakannya sebagai dasar prediksi pada balapan yang akan datang, dalam hal ini 2025 Mexico City Grand Prix.

2.6 Preprocessing Data

Preprocessing data merupakan tahapan awal yang krusial dalam proses pengembangan model machine learning, di mana data mentah yang diperoleh dari berbagai sumber diolah dan dipersiapkan agar siap digunakan dalam proses pelatihan model. Preprocessing data mencakup serangkaian proses transformasi data yang bertujuan untuk meningkatkan kualitas data sehingga hasil analisis menjadi lebih akurat dan andal. Tahapan ini menjadi sangat penting mengingat data di dunia nyata seringkali mengandung nilai yang hilang (missing values), data yang tidak konsisten, maupun outlier yang dapat menurunkan performa model secara signifikan.

Secara umum, preprocessing data terdiri atas beberapa tahapan utama. Pertama, data cleaning yaitu proses identifikasi dan penanganan data yang tidak lengkap, duplikat, atau mengandung noise. Kedua, data integration yaitu penggabungan data dari beberapa sumber menjadi satu dataset yang kohesif. Ketiga, data transformation yaitu proses konversi data ke dalam format atau skala yang sesuai, seperti normalisasi dan standarisasi. Keempat, feature engineering yaitu proses pembentukan fitur-fitur baru yang lebih representatif berdasarkan domain knowledge untuk meningkatkan kemampuan model dalam menangkap pola yang relevan. Proses-proses tersebut secara keseluruhan bertujuan untuk memastikan bahwa data yang digunakan dalam pemodelan memiliki kualitas yang memadai sehingga menghasilkan prediksi yang optimal.

2.6.1 Pembersihan Data (*Data Cleaning*)

Data cleaning merupakan proses identifikasi dan penanganan data yang tidak valid, tidak lengkap, maupun tidak konsisten dalam suatu dataset. Proses ini menjadi langkah pertama yang perlu dilakukan sebelum data digunakan dalam pelatihan model, mengingat keberadaan missing values, duplikasi data, maupun outlier dapat

menurunkan performa model secara signifikan. Menurut Mohammed et al. (2025), data yang tidak lengkap, mengandung kesalahan, atau tidak representatif dapat menghasilkan model machine learning yang tidak dapat diandalkan serta menghasilkan keputusan yang buruk, sehingga kualitas data pada seluruh tahapan pipeline mulai dari pelatihan hingga pengujian menjadi prasyarat yang tidak dapat diabaikan.

Pada penelitian ini, data diperoleh melalui FastF1 API, yaitu sebuah antarmuka pemrograman aplikasi berbasis Python yang menyediakan akses terhadap data telemetri dan hasil balapan Formula 1 secara resmi dan terstruktur. Karena data yang diambil melalui FastF1 API telah melalui proses pengolahan dari sumber resmi Formula 1, data yang diperoleh dalam kondisi relatif bersih dan terstruktur dengan baik. Meskipun demikian, tetap dilakukan pemeriksaan awal terhadap dataset menggunakan fungsi `dropna()` untuk memastikan tidak terdapat missing values pada kolom-kolom kritis seperti *LapTime (s)*, *Position*, dan *Points* sebelum data digunakan dalam tahap selanjutnya.

2.6.2 Integrasi Data (*Data Integration*)

Data integration adalah proses penggabungan data dari beberapa sumber yang berbeda menjadi satu dataset yang utuh dan siap digunakan. Proses ini penting untuk memastikan seluruh informasi yang dibutuhkan tersedia dalam satu struktur data yang konsisten sebelum masuk ke tahap pemodelan.

Pada penelitian ini, data diambil dari FastF1 API secara terpisah untuk masing-masing musim, yaitu tahun 2022, 2023, dan 2024. Ketiga dataset tersebut kemudian digabungkan secara manual menggunakan Microsoft Excel dengan memastikan keseragaman nama kolom dan format nilai antar tahun, sehingga menghasilkan satu file CSV bernama `data_training.csv`. Dataset gabungan ini digunakan sebagai data latih, sedangkan data tahun 2025 digunakan sebagai data uji.

2.6.3 Feature Engineering

Feature engineering adalah proses pembuatan fitur-fitur baru dari data yang sudah ada dengan memanfaatkan pemahaman terhadap domain permasalahan, dengan tujuan meningkatkan kemampuan model dalam menangkap pola yang relevan (Wibowo, M. A. A., & Setiadi, D. R. I. M. 2024). Fitur yang dirancang dengan baik dapat memberikan informasi tambahan yang tidak tersedia secara langsung di data mentah, sehingga berpotensi meningkatkan akurasi prediksi secara signifikan.

Pada penelitian ini, *feature engineering* dilakukan dalam beberapa kelompok. Pertama, fitur karakteristik sirkuit seperti panjang lintasan, jumlah tikungan, jumlah zona DRS (Drag Reduction System), tingkat degradasi ban, dan ketinggian sirkuit di atas permukaan laut. Kedua, fitur kondisi cuaca seperti suhu udara, suhu aspal, kelembaban, dan kecepatan angin yang disesuaikan dengan kondisi historis *Mexico City Grand Prix*. Ketiga, fitur fisika berbasis ketinggian (altitude) seperti rasio densitas udara dan estimasi penurunan downforce, mengingat Mexico City berada di ketinggian 2.285 meter yang berdampak langsung pada performa mobil. Keempat, fitur performa pembalap per musim seperti win rate, podium rate, dan points per race. Kelima, fitur turunan seperti selisih pace kualifikasi terhadap race pace dan keunggulan posisi start relatif terhadap median grid.

2.6.4 Train-Test-Split

Train-test split adalah proses pembagian dataset menjadi dua bagian, yaitu data latih (training data) yang digunakan untuk melatih model dan data uji (test data) yang digunakan untuk mengukur performa model terhadap data yang belum pernah dilihat sebelumnya (Singh, V., Pencina, M., & Einstein, A. J. 2021). Pembagian ini penting untuk menghindari overfitting, yaitu kondisi di mana model terlalu menyesuaikan diri dengan data latih sehingga gagal melakukan generalisasi pada data baru.

Pada penelitian ini, pembagian data tidak dilakukan secara acak (random split), melainkan berdasarkan urutan waktu (temporal split). Data musim 2022 hingga 2024 digunakan sebagai data latih, sedangkan data musim 2025 digunakan sebagai data uji. Pendekatan ini dipilih karena data balapan bersifat sekuensial dan

memiliki ketergantungan terhadap waktu, sehingga pembagian berbasis waktu lebih mencerminkan skenario prediksi yang sesungguhnya dibandingkan pembagian acak

2.6.5 Data Scaling

Data scaling adalah proses transformasi nilai fitur ke dalam rentang atau skala tertentu agar setiap fitur memiliki kontribusi yang setara dalam proses pelatihan model. Tanpa scaling, fitur dengan rentang nilai yang jauh lebih besar cenderung mendominasi perhitungan model dan berpotensi menghasilkan prediksi yang bias. Dua metode yang paling umum digunakan adalah normalisasi (Min-Max Scaling) yang mengubah nilai ke rentang 0 hingga 1, dan standarisasi (StandardScaler) yang mengubah distribusi data sehingga memiliki rata-rata nol dan standar deviasi satu.

2.7 Random Forest

Random Forest adalah algoritma ensemble learning yang bekerja dengan membangun sejumlah pohon keputusan (*decision tree*) secara bersamaan dan menggabungkan hasilnya untuk menghasilkan prediksi akhir (Breiman, 2001). Pada tugas regresi, prediksi akhir diperoleh dari rata-rata seluruh prediksi pohon yang terbentuk, sebagaimana ditunjukkan pada persamaan berikut:

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B T_b(x)$$

di mana $\hat{y}$ adalah nilai prediksi akhir, B adalah jumlah pohon ($n_estimators$), $T_b(x)$ adalah prediksi dari pohon ke- b , dan x adalah vektor fitur input.

Random Forest menerapkan dua mekanisme utama untuk menghasilkan pohon yang beragam dan tidak saling berkorelasi. Pertama, *bagging* (Bootstrap Aggregating), yaitu setiap pohon dilatih menggunakan subset data yang diambil secara acak dengan pengembalian (*bootstrap sampling*) dari data latih. Kedua, *random feature selection*, yaitu pada setiap pemisahan node, hanya sebagian fitur yang dipilih secara acak sebagai kandidat split, bukan seluruh fitur yang tersedia.

Jumlah fitur yang dipertimbangkan per split umumnya ditentukan sebesar $\sqrt{p}$ untuk klasifikasi dan $\frac{p}{3}$ untuk regresi, di mana p adalah total jumlah fitur (Breiman, 2001). Kombinasi kedua mekanisme ini membuat Random Forest lebih tahan terhadap *overfitting* (model terlalu menyesuaikan data pelatihan sehingga kurang mampu melakukan generalisasi pada data baru) dibandingkan decision tree tunggal.

Selain itu, Random Forest juga menghasilkan informasi *feature importance* yang menunjukkan seberapa besar kontribusi masing-masing fitur terhadap hasil prediksi, dihitung berdasarkan rata-rata penurunan impuritas (*mean decrease in impurity*) di seluruh pohon. Pada penelitian ini, Random Forest Regressor dilatih dengan tiga skema konfigurasi, yaitu parameter default, hasil tuning menggunakan Grid Search, dan hasil tuning menggunakan Bayesian Optimization, untuk membandingkan pengaruh optimasi hyperparameter terhadap akurasi prediksi lap time.

2.8 Gradient Boosting

Gradient Boosting adalah algoritma ensemble learning yang membangun model secara bertahap dan aditif, di mana setiap pohon baru dilatih untuk memperbaiki kesalahan (residual) dari pohon sebelumnya (Friedman, 2001). Berbeda dengan Random Forest yang membangun pohon secara paralel dan independen, Gradient Boosting membangun pohon secara sekuensial sehingga setiap iterasi berfokus pada data yang belum diprediksi dengan baik oleh model sebelumnya.

Secara matematis, model Gradient Boosting dibangun melalui proses optimasi fungsi loss secara bertahap menggunakan pendekatan *gradient descent* pada ruang fungsi. Prediksi pada iterasi ke- m dirumuskan sebagai:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

di mana $F_m(x)$ adalah prediksi model pada iterasi ke- m , $F_{m-1}(x)$ adalah prediksi model sebelumnya, η adalah *learning rate* yang mengontrol besarnya kontribusi setiap pohon, dan $h_m(x)$ adalah pohon baru yang dilatih terhadap nilai residual atau *pseudo-residual* dari iterasi sebelumnya. Pseudo-residual sendiri dihitung sebagai negatif gradien dari fungsi loss L terhadap prediksi sebelumnya:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}$$

Untuk tugas regresi dengan fungsi loss Mean Squared Error (MSE), pseudo-residual menyederhanakan menjadi selisih antara nilai aktual dan prediksi, yaitu $r_{im} = y_i - F_{m-1}(x_i)$.

Keunggulan Gradient Boosting terletak pada kemampuannya menangkap pola yang kompleks dan non-linear dalam data, namun algoritma ini lebih rentan terhadap *overfitting* apabila jumlah iterasi terlalu besar tanpa regularisasi yang memadai (Friedman, 2001). Pada penelitian ini, Gradient Boosting Regressor diterapkan menggunakan implementasi dari scikit-learn dengan konfigurasi hyperparameter yang disesuaikan untuk memperoleh performa prediksi lap time yang optimal.

2.9 XGBoost

XGBoost (*Extreme Gradient Boosting*) adalah pengembangan dari algoritma Gradient Boosting yang dirancang untuk lebih efisien secara komputasi dan memiliki kemampuan generalisasi yang lebih baik (Chen & Guestrin, 2016). XGBoost menambahkan komponen regularisasi pada fungsi objektifnya untuk mencegah *overfitting*, sehingga berbeda dari implementasi Gradient Boosting standar. Fungsi objektif XGBoost terdiri dari dua bagian, yaitu fungsi loss dan regularisasi, yang dirumuskan sebagai:

$$\mathcal{L} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

di mana $l(y_i, \hat{y}_i)$ adalah fungsi loss yang mengukur selisih antara nilai aktual y_i dan nilai prediksi $\hat{y}_i$, sedangkan $\Omega(f_k)$ adalah komponen regularisasi pada pohon ke- k yang didefinisikan sebagai:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

di mana T adalah jumlah daun (*leaf*) pada pohon, w_j adalah skor output pada daun ke- j , γ adalah parameter penalti untuk penambahan daun baru, dan λ adalah parameter regularisasi L2 pada bobot daun. Kombinasi kedua parameter ini secara langsung mengontrol kompleksitas pohon yang terbentuk.

Pada setiap iterasi boosting, prediksi diperbarui sebagai:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

di mana $f_t(x_i)$ adalah pohon baru yang dilatih untuk meminimalkan fungsi objektif pada iterasi ke- t . XGBoost menggunakan pendekatan second-order Taylor expansion untuk mengaproksimasi fungsi loss, sehingga proses optimasi menjadi lebih stabil dan konvergen lebih cepat dibandingkan Gradient Boosting standar yang hanya menggunakan gradien orde pertama (Chen & Guestrin, 2016).

Keunggulan lain XGBoost meliputi dukungan terhadap komputasi paralel pada proses pembangunan pohon, penanganan *missing values* secara otomatis, serta mekanisme *early stopping* untuk menghentikan pelatihan ketika performa pada data validasi tidak lagi membaik. Pada penelitian ini, XGBoost Regressor diterapkan dalam dua skema, yaitu tuning manual dan tuning otomatis menggunakan Optuna, untuk membandingkan efektivitas kedua pendekatan optimasi hyperparameter terhadap akurasi prediksi lap time.

2.10 CatBoost

CatBoost (*Categorical Boosting*) adalah algoritma gradient boosting yang dikembangkan oleh Yandex dan dirancang khusus untuk menangani fitur kategorikal secara langsung tanpa memerlukan proses encoding manual seperti *one-hot encoding* (Prokhorenkova et al., 2018). Keunggulan utama CatBoost terletak pada teknik *ordered boosting* dan *ordered target statistics* yang dikembangkan untuk mengatasi masalah *prediction shift*, yaitu bias yang muncul akibat kebocoran informasi target selama proses pelatihan pada algoritma gradient boosting konvensional.

Fungsi objektif CatBoost pada tugas regresi serupa dengan Gradient Boosting pada umumnya, yaitu meminimalkan akumulasi loss di seluruh iterasi:

$$\mathcal{L} = \sum_{i=1}^n l(y_i, \hat{y}_i)$$

Namun yang membedakan CatBoost adalah cara pembangunan pohonnya. CatBoost menggunakan struktur pohon simetris (*oblivious trees*) di mana setiap level pohon menerapkan kondisi pemisahan yang sama pada seluruh node, sehingga dirumuskan sebagai:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta \cdot h_t(x_i)$$

di mana η adalah *learning rate* dan $h_t(x_i)$ adalah pohon simetris baru pada iterasi ke- t . Struktur pohon simetris ini membuat CatBoost lebih cepat dalam proses inferensi dan lebih tahan terhadap *overfitting* dibandingkan pohon asimetris yang digunakan pada XGBoost maupun Gradient Boosting standar.

Untuk penanganan fitur kategorikal, CatBoost menerapkan *target statistics* berbasis urutan (*ordered target statistics*) yang memanfaatkan informasi dari data sebelumnya dalam urutan acak, sehingga estimasi nilai kategorikal menjadi lebih akurat dan bebas dari kebocoran data. Pada penelitian ini, CatBoost Regressor diterapkan sebagai salah satu model pembanding dalam memprediksi lap time Formula 1 di Mexico City Grand Prix, dengan memanfaatkan kemampuannya dalam menangani kombinasi fitur numerik dan kategorikal yang terdapat dalam dataset.

2.11 Hyperparameter Tuning

Hyperparameter tuning merupakan salah satu tahapan penting dalam pengembangan model machine learning yang bertujuan untuk memperoleh konfigurasi parameter terbaik sebelum proses pelatihan model dilakukan. Menurut Bischl et al. (2023), sebagian besar algoritma machine learning dikonfigurasi oleh sekumpulan hyperparameter yang nilainya harus dipilih secara cermat karena seringkali memberikan dampak yang signifikan terhadap performa model. Setiap algoritma

memiliki hyperparameter yang berbeda dan dapat memberikan pengaruh yang signifikan terhadap performa model. Pengaturan hyperparameter yang sesuai dapat meningkatkan kemampuan model dalam menangkap pola pada data, sedangkan pengaturan yang kurang tepat berpotensi menyebabkan penurunan akurasi prediksi. Pada algoritma Random Forest, Gradient Boosting, XGBoost, CatBoost, dan LightGBM, beberapa hyperparameter yang sering digunakan dalam proses penyesuaian antara lain `n_estimators`, `max_depth`, `learning_rate`, dan `min_samples_split`. Melalui proses hyperparameter tuning, model diharapkan dapat menghasilkan performa yang lebih optimal sehingga mampu memberikan hasil prediksi yang lebih akurat dibandingkan penggunaan parameter bawaan (default parameter).

2.11.1 GridSearchCV

GridSearchCV merupakan metode pencarian hyperparameter yang bekerja dengan cara mengeksplorasi seluruh kombinasi nilai parameter yang telah ditentukan sebelumnya secara exhaustive. Setiap kombinasi parameter dievaluasi menggunakan validasi silang (cross-validation) untuk memperoleh estimasi performa model yang lebih robust dan tidak bergantung pada satu pembagian data tertentu. Meskipun pendekatan ini menjamin ditemukannya kombinasi terbaik dalam ruang pencarian yang didefinisikan, GridSearchCV memiliki kelemahan berupa kompleksitas komputasi yang tinggi karena jumlah kombinasi yang harus diuji bertumbuh secara eksponensial seiring bertambahnya jumlah parameter dan nilai yang dicakup (Géron, 2019).

2.11.2 Bayesian Optimization

Bayesian Optimization merupakan metode pencarian hyperparameter yang lebih efisien dibandingkan GridSearchCV karena tidak mengeksplorasi seluruh ruang pencarian secara menyeluruh, melainkan membangun model probabilistik (surrogate model) untuk memperkirakan performa fungsi objektif berdasarkan hasil evaluasi sebelumnya. Pendekatan ini menggunakan fungsi akuisisi untuk menentukan titik pencarian berikutnya dengan menyeimbangkan antara eksplorasi

(exploration) pada wilayah yang belum tereksplorasi dan eksploitasi (exploitation) pada wilayah yang diperkirakan menghasilkan performa terbaik. Dengan demikian, Bayesian Optimization mampu menemukan hyperparameter optimal dengan jumlah iterasi yang jauh lebih sedikit dibandingkan metode pencarian konvensional (Snoek et al., 2012).

2.11.3 Optuna

Optuna merupakan framework optimasi hyperparameter otomatis berbasis Python yang mengimplementasikan pendekatan define-by-run, yaitu ruang pencarian parameter didefinisikan secara dinamis di dalam fungsi objektif pada setiap trial yang dijalankan. Optuna menggunakan algoritma Tree-structured Parzen Estimator (TPE) sebagai metode pencarian utama yang termasuk dalam kategori Bayesian Optimization, sehingga mampu mengarahkan pencarian secara adaptif menuju kombinasi parameter yang lebih menjanjikan berdasarkan hasil trial sebelumnya. Selain efisiensi pencarian, Optuna juga dilengkapi fitur pruning untuk menghentikan trial yang tidak menjanjikan lebih awal, serta visualisasi proses optimasi yang memudahkan analisis terhadap pengaruh masing-masing hyperparameter terhadap performa model (Akiba et al., 2019).

2.12 Python

Python adalah bahasa pemrograman tingkat tinggi yang bersifat interpreted dan mendukung berbagai paradigma pemrograman, termasuk pemrograman prosedural, berorientasi objek, dan fungsional. Python banyak digunakan dalam bidang ilmu data dan machine learning karena sintaksnya yang sederhana dan mudah dipahami, serta didukung oleh ekosistem library yang sangat kaya. Beberapa library utama yang umum digunakan dalam pengembangan model machine learning antara lain NumPy dan Pandas untuk pengolahan data, Matplotlib dan Seaborn untuk visualisasi, serta Scikit-learn, XGBoost, dan CatBoost untuk pemodelan.

Pada penelitian ini, Python digunakan sebagai bahasa pemrograman utama dalam seluruh tahapan pengerjaan, mulai dari pengambilan data melalui FastF1 API, preprocessing, feature engineering, pelatihan model, hingga evaluasi hasil

prediksi. Seluruh proses dijalankan dalam lingkungan Google Colaboratory (*Colab*) yang memungkinkan eksekusi kode Python berbasis *cloud* tanpa memerlukan konfigurasi lingkungan lokal secara khusus.

2.13 Google Colaboratory

Google Colaboratory atau yang lebih dikenal dengan sebutan Google Colab adalah platform komputasi berbasis *cloud* yang disediakan oleh Google secara gratis, memungkinkan pengguna untuk menulis dan mengeksekusi kode Python langsung melalui browser tanpa perlu melakukan instalasi perangkat lunak apapun di komputer lokal. Platform ini berbasis Jupyter Notebook sehingga mendukung penulisan kode, teks, dan visualisasi dalam satu dokumen yang terintegrasi. Google Colab juga menyediakan akses terhadap sumber daya komputasi seperti GPU dan TPU yang sangat berguna untuk mempercepat proses pelatihan model *machine learning*.

Pada penelitian ini, Google Colab digunakan sebagai lingkungan utama dalam pengembangan model prediksi lap time Formula 1. Seluruh proses mulai dari pengambilan data menggunakan FastF1 API, *preprocessing*, *feature engineering*, pelatihan model, hingga evaluasi dilakukan di dalam Google Colab. Penggunaan platform ini memudahkan proses pengerjaan karena dataset dan notebook dapat diakses kapan saja melalui integrasi dengan Google Drive.

2.14 FastF1 API

FastF1 adalah *library* Python open-source yang menyediakan akses terhadap data Formula 1 secara terstruktur, mencakup data telemetri, hasil sesi balapan, kualifikasi, latihan bebas, serta informasi cuaca dan kondisi ban (Theiner, 2024). Library ini memanfaatkan data resmi yang bersumber dari API timing Formula 1 dan Ergast API, kemudian menyajikannya dalam format yang mudah diolah menggunakan pandas DataFrame, sehingga sangat memudahkan proses analisis dan pengembangan model berbasis data balapan.

FastF1 menyediakan berbagai fungsi utama yang dapat digunakan untuk mengakses data per sesi secara spesifik. Pengambilan data dilakukan dengan

menginisialisasi sesi tertentu menggunakan kombinasi tahun, nama Grand Prix, dan jenis sesi, kemudian memuat seluruh data yang tersedia melalui fungsi `session.load()`. Data yang dapat diakses meliputi lap data per pembalap, telemetri kecepatan dan throttle, informasi strategi ban, posisi balapan, serta kondisi cuaca per lap. Pada penelitian ini, FastF1 digunakan untuk mengambil data historis Mexico City Grand Prix secara terpisah untuk musim 2022, 2023, dan 2024 sebagai data latih, serta musim 2025 sebagai data uji dalam pemodelan prediksi lap time.

2.15 Metrik Evaluasi

Metrik evaluasi digunakan untuk mengukur seberapa baik performa model machine learning dalam memprediksi nilai target dibandingkan nilai aktualnya. Pada tugas regresi, metrik evaluasi yang umum digunakan berfokus pada besarnya selisih antara nilai prediksi dan nilai aktual. Menurut Hodson (2022), MAE dan RMSE merupakan metrik yang telah digunakan secara luas dalam evaluasi model, meskipun terdapat kebingungan yang berkelanjutan mengenai penggunaannya, sehingga praktik yang umum dilakukan adalah menyajikan keduanya secara bersamaan. Selain itu, Chicco, Warrens, dan Jurman (2021) menyatakan bahwa koefisien determinasi (R^2) merupakan metrik yang lebih informatif dibandingkan MAE, RMSE, dalam evaluasi analisis regresi, karena R^2 mengukur proporsi variansi pada variabel target yang mampu dijelaskan oleh model, sehingga memberikan gambaran yang lebih jelas mengenai kualitas prediksi secara keseluruhan. Pada penelitian ini, metrik yang digunakan adalah Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), dan koefisien determinasi (R^2).

2.15.1 Mean Absolute Error (MAE)

Mean Absolute Error (MAE) mengukur rata-rata selisih absolut antara nilai prediksi dan nilai aktual, dirumuskan sebagai:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

di mana y_i adalah nilai aktual, $\hat{y}_i$ adalah nilai prediksi, dan n adalah jumlah data. MAE mudah diinterpretasikan karena satuannya sama dengan satuan variabel target, yaitu detik, sehingga nilai MAE secara langsung menunjukkan rata-rata kesalahan prediksi lap time dalam satuan detik.

2.15.2 Root Mean Squarred Error

Root Mean Squared Error (RMSE) mengukur akar kuadrat dari rata-rata kuadrat selisih antara nilai prediksi dan nilai aktual, dirumuskan sebagai:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Dibandingkan MAE, RMSE memberikan penalti yang lebih besar terhadap kesalahan prediksi yang jauh menyimpang dari nilai aktual karena adanya operasi kuadrat. Hal ini membuat RMSE lebih sensitif terhadap *outlier* sehingga cocok digunakan untuk mendeteksi apakah model menghasilkan kesalahan besar pada kasus-kasus tertentu. Kombinasi MAE dan RMSE dalam penelitian ini memberikan gambaran yang lebih menyeluruh terhadap performa masing-masing model dalam memprediksi lap time Formula 1 di *Mexico City Grand Prix*.

2.15.3 Koefisien Determinan (R^2)

Koefisien determinasi atau R^2 (*R-squared*) merupakan metrik evaluasi yang mengukur seberapa besar proporsi variansi pada variabel target yang mampu dijelaskan oleh model prediksi. Nilai R^2 dihitung menggunakan persamaan berikut:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

di mana y_i adalah nilai aktual, $\hat{y}_i$ adalah nilai prediksi model, $\bar{y}$ adalah rata-rata nilai aktual, dan n adalah jumlah data. Pembilang pada persamaan tersebut merepresentasikan *Residual Sum of Squares* (RSS) yaitu total kuadrat selisih antara nilai aktual dan prediksi, sedangkan penyebut merepresentasikan *Total Sum of Squares* (TSS) yaitu total kuadrat selisih antara nilai aktual dan rata-ratanya.

Nilai R^2 berada pada rentang $-\infty$ hingga 1, di mana nilai 1 menunjukkan bahwa model mampu menjelaskan seluruh variansi data secara sempurna, nilai 0 menunjukkan bahwa model tidak lebih baik dari sekadar memprediksi nilai rata-rata, dan nilai negatif menunjukkan bahwa performa model lebih buruk dari baseline rata-rata tersebut. Secara umum, model dianggap memiliki performa yang baik apabila nilai R^2 berada di atas 0.80, yang berarti model mampu menjelaskan lebih dari 80% variansi pada data (Montgomery et al., 2012).

2.16 Penelitian Terdahulu

Adapun penelitian terdahulu yang digunakan sebagai referensi dalam penelitian ini berasal dari berbagai sumber yang memiliki keterkaitan dengan topik penelitian. Penelitian-penelitian tersebut digunakan sebagai bahan acuan untuk memahami metode, pendekatan, serta hasil penelitian yang telah dilakukan sebelumnya, sehingga dapat menjadi dasar dalam pengembangan penelitian ini.

Tabel 2.1 Tabel Penelitian Terdahulu

No	Peneliti (Tahun)	Judul	Metode	Hasil
1	Bansal et al. (2024)	Advanced Machine Learning Approaches for Formula 1 Race Performance Prediction: A Comprehensive Analysis of Championship Point Forecasting	• Gradient Boosting • LightGBM • Random Forest • XGBoost • Linear/ Ridge / Lasso Reg.	Gradient Boosting menghasilkan performa terbaik dengan $R^2 = 0,999$, RMSE = 0,197, dan MAE = 0,125. Posisi balapan menjadi fitur paling berpengaruh dengan kontribusi 75,84%.
2	García Tejada, L. (2023)	Applying Machine Learning to Forecast Formula 1 Race Outcomes	• SVM • Random Forest • Artificial Neural Network	SVM mencapai performa terbaik untuk prediksi pit stop dengan F1-score = 0,621, diikuti ANN (0,593) dan Random Forest (0,372). Data diambil dari FastF1 API dan Ergast API musim 2019–2022
3	Jafri, A. (2024)	Predicting Formula 1 Race Outcomes: A Machine Learning Approach	• LSTM • Random Forest • Linear Regression • Decision Tree	LSTM menghasilkan performa terbaik dalam memprediksi lap time dan posisi balapan. Model LSTM v2.1 dan v2.2 berhasil

No	Peneliti (Tahun)	Judul	Metode	Hasil
				memprediksi pemenang dan top 5 dengan urutan yang tepat pada Grand Prix Bahrain 2024

Berdasarkan tinjauan terhadap tiga penelitian terdahulu yang telah ditampilkan pada Tabel 2.1, dapat disimpulkan bahwa penerapan *machine learning* dalam prediksi performa Formula 1 telah berkembang dari pendekatan statistik sederhana menuju metode ensemble dan deep learning yang lebih kompleks. Bansal et al. (2024) menunjukkan bahwa algoritma Gradient Boosting mampu mencapai akurasi prediksi poin kejuaraan yang sangat tinggi ($R^2 = 0,999$), dengan posisi balapan dan faktor musiman sebagai prediktor dominan. García Tejada (2023) membuktikan bahwa prediksi pit stop sebagai masalah klasifikasi biner memiliki tingkat kesulitan tersendiri akibat ketidakseimbangan data yang ekstrem, di mana SVM menjadi algoritma terbaik dengan F1-score 0,621. Sementara itu, Jafri (2024) menekankan bahwa model LSTM dengan *custom loss function* yang menggabungkan *lap time loss*, *position loss*, dan *historical loss* menghasilkan prediksi posisi akhir balapan yang lebih akurat dibandingkan minimisasi *lap time loss* semata. Secara keseluruhan, ketiga penelitian menegaskan bahwa pemilihan algoritma yang tepat, rekayasa fitur yang domain-specific, dan strategi evaluasi yang sesuai merupakan faktor krusial dalam membangun model prediktif yang andal untuk olahraga Formula 1.

Penelitian yang sedang dilakukan memiliki keterkaitan langsung dengan ketiga penelitian di atas, sekaligus memiliki kebaruan tersendiri. Secara metodologis, penelitian ini mengadopsi pendekatan komparatif multi-model sebagaimana dilakukan oleh Bansal et al. (2024), dengan membandingkan tujuh model yakni Random Forest (default, Grid Search, dan Bayesian Optimization), Gradient Boosting, XGBoost (default dan Optuna), serta Gradient Boosting. Fokus

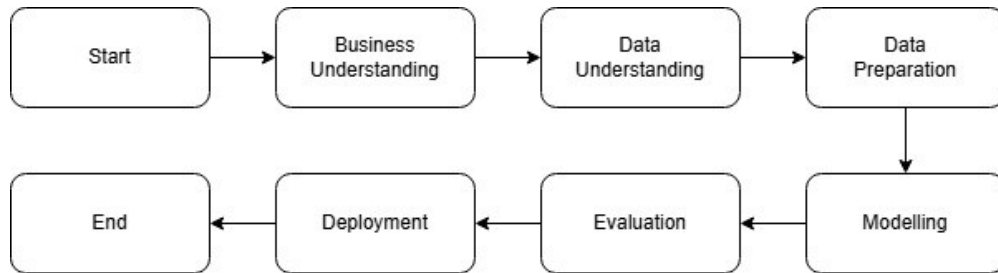
pada prediksi *race pace* (lap time dalam detik) sebagai variabel target menjadikan penelitian ini selaras dengan Jafri (2024) yang juga memprediksi *lap time* sebagai basis inferensi posisi balapan.

Terdapat beberapa hal yang membedakan penelitian ini dari penelitian sebelumnya. Penelitian ini berfokus pada *Mexico City Grand Prix*, yang memiliki kekhasan berupa ketinggian sirkuit Autodromo Hermanos Rodriguez mencapai 2.285 meter di atas permukaan laut. Kondisi ini berdampak langsung pada efisiensi mesin dan tingkat downforce kendaraan, sehingga menjadikannya konteks yang belum banyak dikaji dalam literatur prediksi lap time berbasis machine learning. Selain itu, fitur yang digunakan dalam penelitian ini dirancang secara lebih menyeluruh dengan menggabungkan tiga kelompok variabel, yaitu karakteristik sirkuit, kondisi cuaca historis, dan metrik performa pembalap per musim. Pendekatan ini lebih lengkap dibandingkan García Tejada (2023) yang tidak menyertakan fitur sirkuit, maupun Jafri (2024) yang tidak memasukkan data cuaca secara eksplisit. Dari sisi optimasi model, penelitian ini menggunakan Bayesian Optimization melalui skopt dan Optuna sebagai alternatif yang lebih efisien dibandingkan grid search konvensional yang diterapkan pada penelitian-penelitian sebelumnya.

3. PEMBAHASAN

3.1 Gambaran Umum

Penelitian ini bertujuan untuk menganalisis dan membandingkan kinerja beberapa algoritma *machine learning* pada studi kasus prediksi peringkat kemenangan setiap driver yang berpartisipasi dalam Formula 1 *Mexico City 2025 Grand Prix*, dengan menggunakan metode CRISP-DM (*Cross Industry Standard Process for Data Mining*). Metode CRISP-DM digunakan sebagai alur kerja secara menyeluruh melalui enam aspek utama, meliputi *business understanding*, *data understanding*, *data preparation*, *modeling*, *evaluation*, serta *deployment*. Berikut adalah alur proses penelitian. Alur kerja metode CRISP-DM akan ditampilkan pada Gambar 3.1 berikut.



Gambar 3.1 Diagram alur penelitian

Karena penelitian ini bersifat analisis komparatif, tahapan CRISP-DM hanya dilakukan pada tahap Business Understanding, Data Understanding, Data Preparation, Modelling, Evaluation saja

3.2 Pemahaman Bisnis (*Business Understanding*)

Tahap *business understanding* berfokus pada pemahaman menyeluruh terhadap tujuan penelitian sebelum proses analisis data dilakukan. Penelitian ini bertujuan untuk membandingkan kinerja beberapa algoritma *machine learning* dalam memprediksi peringkat hasil balapan setiap pembalap yang berkompetisi pada 2025 *Mexico City Formula 1 Grand Prix*. Hasil dari perbandingan tersebut diharapkan dapat memberikan rekomendasi mengenai algoritma *machine learning* yang paling

optimal untuk digunakan dalam prediksi performa pembalap pada sirkuit berkarakteristik khusus.

Formula 1 merupakan ajang balap mobil paling bergengsi di dunia yang tidak hanya mengandalkan kemampuan pembalap, tetapi juga melibatkan teknologi tinggi, strategi tim yang kompleks, serta persaingan ketat di setiap seri balapan. Setiap balapan menghasilkan data yang sangat kaya, mencakup performa kendaraan, karakteristik sirkuit, hingga rekam jejak masing-masing pembalap di berbagai lokasi balapan. Data-data tersebut berpotensi besar untuk dimanfaatkan dalam membangun model prediktif berbasis machine learning yang mampu mengidentifikasi pola dan menghasilkan prediksi yang lebih akurat. Atas dasar itu, penelitian ini memusatkan perhatian pada analisis komparatif beberapa model machine learning untuk memprediksi peringkat akhir pembalap, dengan mengambil studi kasus *2025 Mexico City Formula 1 Grand Prix*.

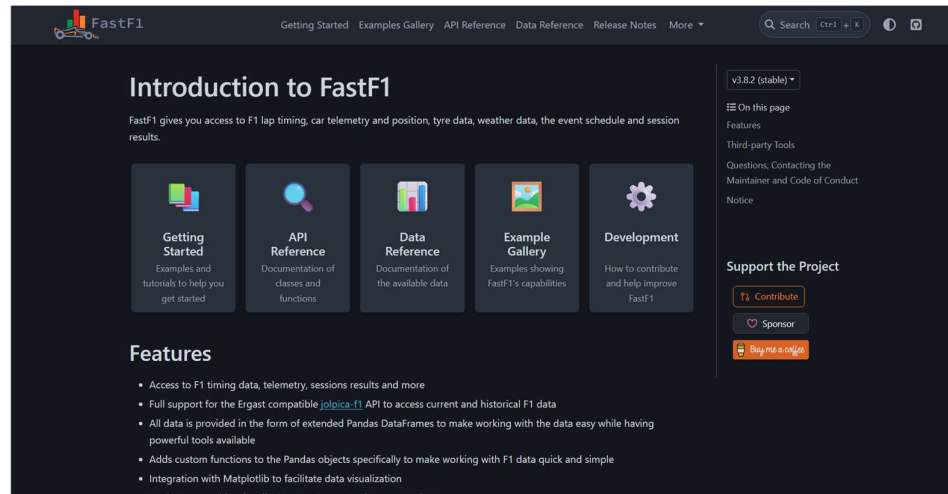
Mexico City Grand Prix dipilih sebagai studi kasus karena sirkuit ini memiliki karakteristik geografis yang tidak ditemukan di kebanyakan sirkuit lain, yaitu letaknya pada ketinggian sekitar 2.200 meter di atas permukaan laut. Kondisi tersebut menyebabkan kepadatan udara yang lebih rendah dari biasanya, yang secara langsung memengaruhi performa aerodinamika dan mesin kendaraan. Keunikan inilah yang menjadikan sirkuit Mexico City sebagai kasus yang menarik sekaligus menantang untuk dianalisis menggunakan pendekatan *machine learning*.

3.3 Pemahaman Data (*Data Understanding*)

Pada tahap ini berfokus pada eksplorasi dan pemahaman karakteristik data yang digunakan.

- a. *Import Library* dan *Package: Library* dan *package* yang diperlukan untuk visualisasi dan analisis data, meliputi Pandas, Matplotlib, Seaborn dan Numpy.
- b. *Data yang Digunakan*: Dataset yang digunakan merupakan data sekunder yang diperoleh dari *FastF1 library*. Data yang digunakan memiliki fitur-fitur seperti

nama pembalap, *laptime(s)*, *sectortime(s)*, *qualification time(s)*, *position*, *grid position*, *points*, *racepace(s)*, yang diambil dalam rentang waktu 2022-2025.



Gambar 3.2 Tampilan FastF1 API

Proses pengumpulan data dilaksanakan menggunakan bahasa pemrograman Python melalui platform Google Colaboratory dengan cara *import library* FastF1 lalu menggunakan fungsi `.get_session` untuk mengambil data balapan dari tahun, sirkuit, dan *sessions* yang diinginkan. Data yang telah dikumpulkan akan digabungkan menggunakan Microsoft Excel seperti ditampilkan pada Gambar 3.3 berikut.

	Year	Abbreviation	LapTime [s]	Sector1Time [s]	Sector2Time [s]	Sector3Time [s]	TotalSectorTime [s]	BestQuali [s]	GridPosition	Points	RacePace [s]
2	2022	ALB	84.99607246	29.64015942	33.27411594	22.0817971	84.99607246	80.859	17	0	84.437
3	2022	ALO	84.82266129	30.18912903	32.75375806	21.87977419	84.82266128	78.939	9	0	84.425
4	2022	BOT	85.03833333	29.91595652	33.05205797	22.07031884	85.03833333	78.401	6	1	84.422
5	2022	GAS	84.99008696	29.72162319	33.29933333	21.96913043	84.99008695	79.672	14	0	84.349
6	2022	HAM	83.50375714	29.62117143	32.26505714	21.61752857	83.50375714	78.084	3	18	82.921
7	2022	LAT	86.38779412	30.23533824	33.70827941	22.44417647	86.38779412	81.167	18	0	85.498
8	2022	LEC	84.21521429	29.79811429	32.65961429	21.75748571	84.21521429	78.555	7	8	83.655
9	2022	MAG	85.51704348	30.10224638	33.28884058	22.12595652	85.51704348	79.833	19	0	84.948
10	2022	MSC	85.50471014	30.0525942	33.33514493	22.11697101	85.50471014	80.419	15	0	84.937

Gambar 3.3 Tampilan 10 Data Teratas

Gambar 3.3 menampilkan 10 data teratas yang telah dikumpulkan. Data tersebut merupakan gabungan data Formula 1 dari tahun 2022 hingga 2025 yang disimpan dalam Microsoft Excel, dengan data tahun 2022–2024 digunakan sebagai data latih (*training data*) dan data tahun 2025 digunakan sebagai data uji (*testing data*).

- c. Jumlah Data: Data yang digunakan terdiri dari 80 baris dan 11 kolom, ditambah dengan kolom yang dihasilkan dari *feature engineering*, menjadikan total 80

baris dan 42 kolom data. Kolom yang ditambahkan terbagi menjadi beberapa kategori meliputi *Circuit features*, *Weather features*, *Altitude features*, *Driver features*, *Derived features*

- d. Eksplorasi Visual: Data divisualisasikan untuk melihat pola dan hubungan antar fitur. Untuk visualisasi data dilakukan menggunakan *library visualization* Python yaitu Seaborn dan Matplotlib yang dilakukan pada platform Google Colaboratory. Untuk visualisasi pertama akan menampilkan informasi performa *driver* dan poin dari tahun 2022-2025 menggunakan serangkaian kode yang akan ditampilkan pada Gambar 3.4 berikut.

```
ax3 = fig.add_subplot(gs[1, 0])

sc = ax3.scatter(
    df_scatter['GridPosition'],
    df_scatter['Position'],
    c=df_scatter['Points'],
    cmap='YlOrRd',
    s=65, alpha=0.88, edgecolors='none',
    vmin=0, vmax=df_scatter['Points'].max()
)

# Garis diagonal referensi (grid = finish)
diag = np.linspace(1, 20, 100)
ax3.plot(diag, diag, color='white', linewidth=0.8,
        linestyle='--', alpha=0.35)

cbar = plt.colorbar(sc, ax=ax3, pad=0.02)
cbar.ax.tick_params(labelsize=7.5, colors='BAAAAA')
cbar.set_label("Poin", fontsize=8.5, color='BAAAAA')

ax3.set_title("Grid Start vs Posisi Akhir Balapan",
             fontsize=11, color='white', pad=8, fontweight='bold')
ax3.set_xlabel("Grid Position (Kualifikasi)", fontsize=9)
ax3.set_ylabel("Race Position (Akhir)", fontsize=9)
ax3.set_xlim(0, 22)
ax3.set_ylim(0, 22)
ax3.grid(alpha=0.3)
ax3.spines['top'].set_visible(False)
ax3.spines['right'].set_visible(False)
```

Gambar 3.4 Kode Visualisai Performa *Driver* dan Poin (2022-2025)

Kode pada Gambar 3.4 menghasilkan serangkaian visualisasi yang ditampilkan pada Gambar 3.5. Visualisasi tersebut terdiri dari empat plot, yaitu: total poin per pembalap yang disajikan menggunakan bar chart, distribusi poin per tahun yang disajikan menggunakan boxplot, hubungan antara posisi start (*grid position*) dan posisi akhir balapan yang disajikan menggunakan scatter plot, serta rata-rata *position gain* per pembalap yang disajikan menggunakan bar chart.



Gambar 3.5 Visualisasi performa *driver*

Visualisasi pada gambar 3.5, menunjukkan bahwa distribusi performa pembalap bersifat tidak merata dan cenderung right-skewed, dengan dominasi kelompok elite seperti Max Verstappen, Charles Leclerc, dan Lando Norris yang mengakumulasi sebagian besar poin selama periode 2022–2025. Distribusi poin tahunan bersifat non-normal dengan median mendekati nol serta keberadaan *outlier* ekstrem, yang mengindikasikan bahwa mayoritas pembalap memperoleh poin rendah sementara hanya sedikit yang konsisten meraih poin tinggi. Selain itu, terdapat korelasi positif antara posisi start dan posisi akhir balapan, yang menunjukkan bahwa performa kualifikasi merupakan prediktor penting hasil balapan. Di sisi lain, analisis position gain mengungkap adanya trade-off antara kualitas kualifikasi dan kemampuan overtaking, di mana pembalap dengan posisi start kurang optimal cenderung memiliki peningkatan posisi lebih tinggi, sedangkan pembalap papan atas menunjukkan peningkatan yang lebih terbatas karena sudah memulai dari posisi terdepan.

Selanjutnya untuk visualisasi kedua akan dilakukan menggunakan cara yang sama seperti visualisasi pertama menggunakan kode yang akan dijalankan pada Google Colaboratory yang akan ditampilkan pada Gambar 3.6.

```

ax1 = fig.add_subplot(gs[0, 0])
x = np.arange(len(years))
width = 0.35

bars1 = ax1.bar(x - width/2, yearly['LapTime (s)'], width,
               label='Lap Time', color='#663946', alpha=0.88, zorder=3)
bars2 = ax1.bar(x + width/2, yearly['RacePace (s)'], width,
               label='Race Pace', color='#2196F3', alpha=0.88, zorder=3)

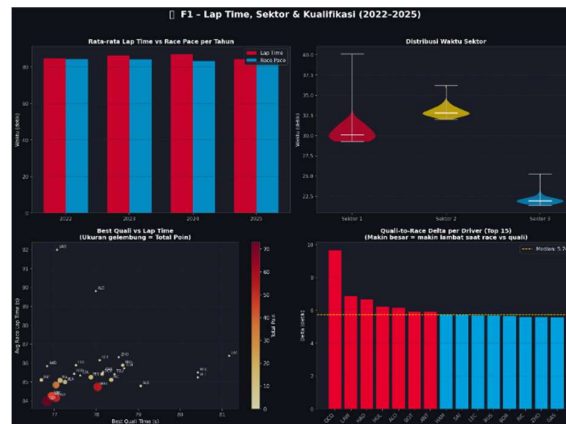
# Label nilai di atas bar
for bar in bars1:
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.15,
            f'{bar.get_height():.1f}', ha='center', va='bottom',
            fontsize=7.5, color='white')
for bar in bars2:
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.15,
            f'{bar.get_height():.1f}', ha='center', va='bottom',
            fontsize=7.5, color='white')

ax1.set_title('Rata-rata Lap Time vs Race Pace per Tahun',
             fontsize=11, color='white', pad=0, fontweight='bold')
ax1.set_xlabel('Tahun', fontsize=9)
ax1.set_ylabel('Waktu (detik)', fontsize=9)
ax1.set_xticks(x)
ax1.set_xticklabels(years, fontsize=9)
ax1.set_ylim(0, yearly[['LapTime (s)', 'RacePace (s)']].max().max() + 5)
ax1.legend(fontsize=8.5, framealpha=0.2, loc='lower right',
          labelcolor='white')
ax1.grid(axis='y', alpha=0.2, zorder=0)
ax1.spines['top'].set_visible(False)
ax1.spines['right'].set_visible(False)

```

Gambar 3.6 Kode Visualisasi Lap Time, Sektor & Kualifikasi (2022-2025)

Kode pada Gambar 3.6 menghasilkan serangkaian visualisasi yang ditampilkan pada Gambar 3.7. Visualisasi tersebut terdiri dari empat plot, yaitu: Rata-rata *Lap Time* dan *Race Pace* per Tahun yang disajikan menggunakan *stacked bar chart*, distribusi waktu sektor yang disajikan menggunakan violin plot, hubungan antara *Best Qualifying Time* dan *Lap Time* yang disajikan menggunakan bubble chart, serta selisih waktu antara kualifikasi dan balapan (*Quali-to-Race Delta*) per pembalap yang disajikan menggunakan bar chart.



Gambar 3.7 Visualisasi *Laptime* Sektoral

Visualisasi pada gambar 3.7, menunjukan bahwa performa waktu *lap* dan *race pace* selama periode 2022–2025 relatif stabil dengan tren peningkatan efisiensi pada tahun terakhir, sementara karakteristik sirkuit tetap konsisten sehingga variasi performa lebih dipengaruhi faktor teknis dan strategi. Distribusi

waktu sektor mengindikasikan tingkat variabilitas yang berbeda, di mana sektor awal cenderung homogen, sektor tengah paling bervariasi, dan sektor akhir paling konsisten. Selain itu, terdapat korelasi positif antara performa kualifikasi dan race pace, yang menegaskan bahwa pembalap dengan waktu kualifikasi lebih cepat cenderung mempertahankan keunggulan saat balapan dan memperoleh poin lebih tinggi. Namun demikian, analisis selisih waktu kualifikasi terhadap balapan mengungkap adanya perbedaan tingkat konsistensi antar pembalap, di mana sebagian mengalami penurunan performa yang signifikan, sementara lainnya mampu mempertahankan kecepatan secara lebih stabil, mencerminkan peran penting strategi dan manajemen balapan dalam menentukan hasil akhir.

Selanjutnya untuk visualisasi ketiga akan dilakukan menggunakan cara yang sama seperti visualisasi pertama dan kedua menggunakan kode yang akan dijalankan pada Google Colaboratory yang akan ditampilkan pada Gambar 3.8.

```
ax = axes[0, 0]

circuit_features = [
    "circuit_length_km",
    "num_corners",
    "num_straights",
    "drs_zones",
    "avg_speed_kmh"
]

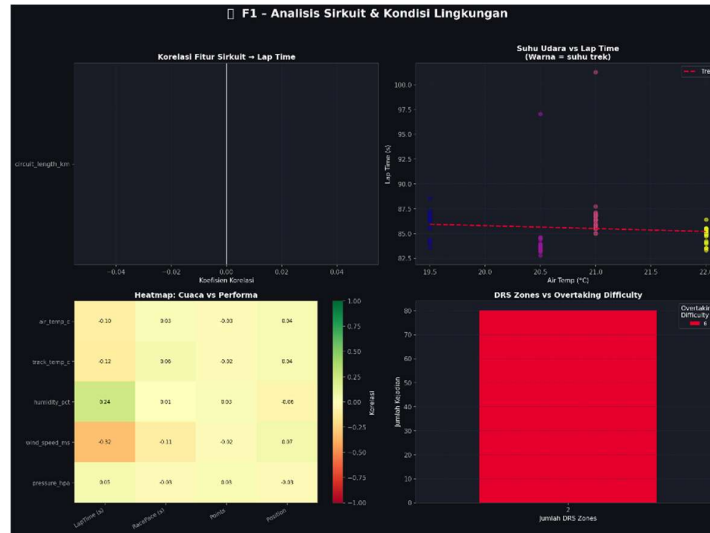
corr = df[circuit_features + ["LapTime (s)"]].corr()["LapTime (s)"][:-1]

sns.barplot(
    x=corr.values,
    y=corr.index,
    palette="viridis",
    ax=ax
)

ax.axvline(0, color="white", linewidth=1)
ax.set_title("Korelasi Fitur Sirkuit + Lap Time")
ax.set_xlabel("Koefisien Korelasi")
ax.set_ylabel("")
```

Gambar 3.8 Kode Visualisasi Analisis Sirkuit & Kondisi Lingkungan

Kode pada Gambar 3.8 menghasilkan serangkaian visualisasi yang akan ditampilkan pada Gambar 3.9. Visualisasi tersebut terdiri dari empat plot, yaitu: korelasi fitur sirkuit dengan *laptime*, hubungan suhu udara dengan *laptime*, hubungan cuaca dengan performa, serta hubungan zona DRS dengan kesulitan *overtaking*.



Gambar 3.9 Visualisasi kondisi sirkuit

Visualisasi pada gambar 3.9, menunjukkan bahwa variabel karakteristik sirkuit dan kondisi lingkungan memiliki pengaruh yang terbatas terhadap lap time dalam dataset yang digunakan. Homogenitas data yang hanya mencakup satu tipe sirkuit menyebabkan sebagian variabel, seperti panjang sirkuit serta jumlah DRS zone, tidak memiliki variasi yang cukup sehingga tidak memberikan kontribusi signifikan secara statistik. Selain itu, hubungan antara suhu udara dan lap time teridentifikasi lemah, dengan variasi waktu yang lebih dipengaruhi oleh faktor lain di luar kondisi suhu. Analisis korelasi kondisi cuaca juga menunjukkan bahwa seluruh variabel lingkungan hanya memiliki hubungan rendah terhadap performa, sehingga tidak bersifat determinatif. Secara keseluruhan, temuan ini menegaskan bahwa keterbatasan variasi data menjadi faktor utama yang menghambat identifikasi hubungan yang kuat, sehingga diperlukan diversifikasi dataset, khususnya pada tipe sirkuit dan kondisi balapan, untuk meningkatkan kualitas analisis dan pemodelan selanjutnya.

Selanjutnya untuk visualisasi keempat akan dilakukan menggunakan cara yang sama seperti visualisasi pertama, kedua, dan ketiga menggunakan kode yang akan dijalankan pada Google Colaboratory yang akan ditampilkan pada Gambar 3.10.

```

# FEATURE IMPORTANCE - Cek kontribusi tiap fitur

from sklearn.ensemble import RandomForestRegressor

rf_importance = RandomForestRegressor(n_estimators=200, random_state=42)
rf_importance.fit(X_train, y_train)

importance_df = pd.DataFrame({
    'feature': feature_columns_enhanced,
    'importance': rf_importance.feature_importances_
}).sort_values('importance', ascending=False)

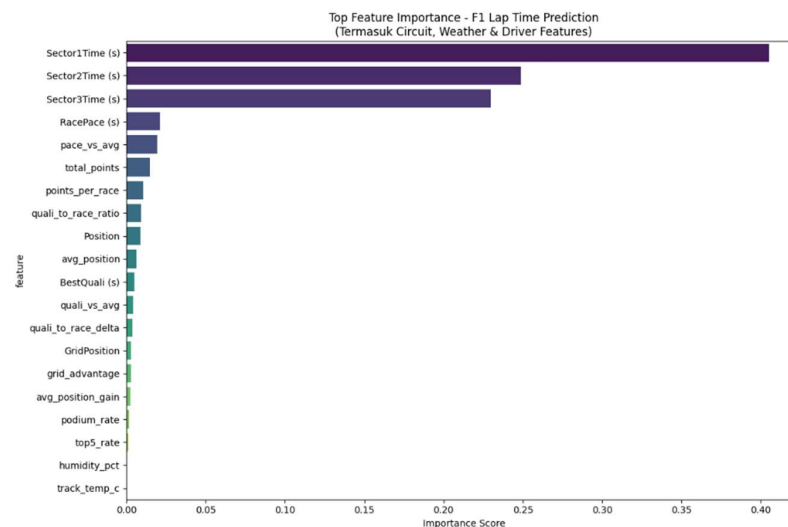
print("Top 20 Feature Importance:")
print(importance_df.head(20).to_string(index=False))

# Visualisasi
plt.figure(figsize=(12, 8))
top_n = min(20, len(importance_df))
sns.barplot(data=importance_df.head(top_n), x='importance', y='feature', palette='viridis')
plt.title("Top Feature Importance - F1 Lap Time Prediction\n(Termasuk Circuit, Weather & Driver Features)")
plt.xlabel('Importance Score')
plt.tight_layout()
plt.savefig('feature_importance_enhanced.png', dpi=150, bbox_inches='tight')
plt.show()
print("\n Plot disimpan ke 'feature_importance_enhanced.png'")

```

Gambar 3.10 Kode Visualisasi Feature Importance

Kode pada Gambar 3.10 akan menghasilkan visualisasi yang akan ditampilkan pada Gambar 3.11. Visualisasi tersebut berisikan informasi mengenai seberapa besar pengaruh suatu fitur terhadap prediksi model machine learning.



Gambar 3.11 Visualiasi *feature importance*

Visualisasi pada gambar 3.11, menunjukan bahwa waktu sektor (*Sector1Time*, *Sector2Time*, dan *Sector3Time*) merupakan prediktor paling dominan dengan kontribusi lebih dari 85% terhadap model, yang secara logis mencerminkan bahwa *lap time* merupakan agregasi langsung dari ketiga komponen tersebut. Sementara itu, variabel menengah seperti *race pace* dan indikator performa relatif (misalnya total poin dan poin per balapan) memberikan kontribusi

terbatas, mengindikasikan adanya pengaruh pola performa historis meskipun tidak dominan. Sebaliknya, variabel terkait posisi, hasil kualifikasi, dan strategi memiliki pengaruh yang sangat rendah, sehingga kurang relevan dalam memprediksi kecepatan *lap* secara langsung. Adapun variabel kondisi lingkungan dan statistik kumulatif pembalap menunjukkan kontribusi paling minimal, yang menguatkan bahwa dalam dataset dengan karakteristik homogen, faktor-faktor tersebut tidak memiliki daya prediktif yang signifikan.

Selanjutnya untuk visualisasi terakhir akan dilakukan menggunakan cara yang sama seperti visualisasi pertama, kedua, ketiga, dan keempat menggunakan kode yang akan dijalankan pada Google Colaboratory yang akan ditampilkan pada Gambar 3.12.

```
fig, ax = plt.subplots(figsize=(12, 8))

# Boxplot
sns.boxplot(
    data=mexico_df,
    x="team",
    y="LapTime (s)",
    palette="husl",
    linewidth=1.2,
    showfliers=False,
    ax=ax
)

# Judul dan Label
ax.set_title(
    "2024 Mexico City Grand Prix",
    fontsize=14,
    pad=12
)

ax.set_xlabel("")
ax.set_ylabel("Lap Time (s)")

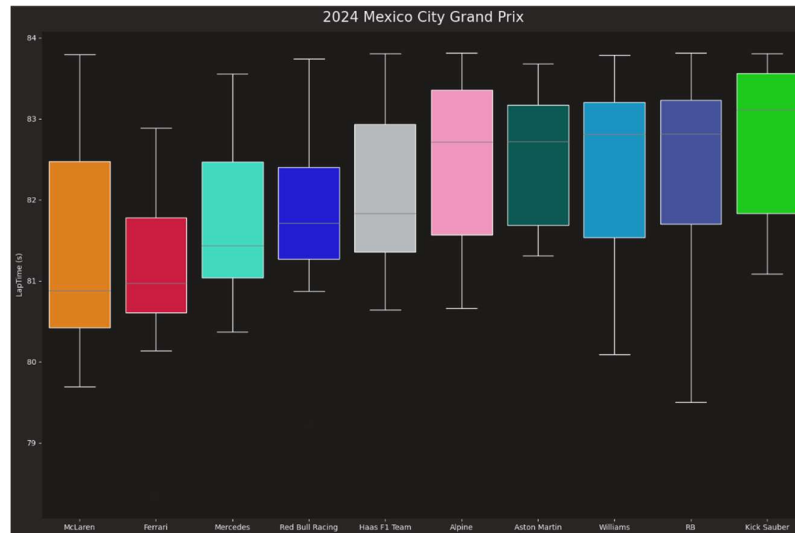
# Rotasi nama tim
plt.xticks(rotation=0)

# Grid tipis
ax.grid(
    True,
    linestyle="--",
    alpha=0.2
)

plt.tight_layout()
plt.show()
```

Gambar 3.12 Kode Visualisasi Performa Tim pada Tahun 2024

Kode pada Gambar 3.12 menghasilkan visualisasi boxplot yang ditampilkan pada Gambar 3.13. Visualisasi tersebut menyajikan distribusi performa setiap tim yang berpartisipasi dalam Formula 1 Mexico City Grand Prix 2024 berdasarkan data lap time.



Gambar 3.13 Visualisasi Performa Tim (2024)

Gambar 3.13 menyajikan distribusi waktu lap (*lap time*) seluruh konstruktor yang berkompetisi pada Grand Prix Mexico City 2024 dalam bentuk diagram kotak (box plot). Diagram ini dipilih karena mampu merepresentasikan sebaran data secara ringkas melalui lima statistik deskriptif sekaligus. Warna setiap kotak pada visualisasi ini merepresentasikan warna identitas masing-masing tim konstruktor, sebagaimana dikenal dalam kompetisi Formula 1, dan tidak memiliki makna statistik tersendiri melainkan berfungsi semata-mata sebagai pembeda visual antar tim. Cara membaca visualisasi ini adalah sebagai berikut: garis horizontal di dalam kotak merepresentasikan nilai median (Q2) atau waktu lap tengah; batas bawah kotak merepresentasikan kuartil pertama (Q1) atau persentil ke-25; batas atas kotak merepresentasikan kuartil ketiga (Q3) atau persentil ke-75; sehingga tinggi kotak mencerminkan rentang antarkuartil (*Interquartile Range/IQR*) yang menunjukkan variabilitas performa inti tim. Garis vertikal (*whisker*) yang memanjang ke atas dan ke bawah menunjukkan jangkauan keseluruhan waktu lap di luar IQR.

Berdasarkan visualisasi tersebut, terlihat bahwa McLaren dan Ferrari mencatatkan median waktu lap terendah, yakni berada pada kisaran 81,2–81,5 detik, yang mengindikasikan keunggulan kecepatan kedua tim tersebut secara

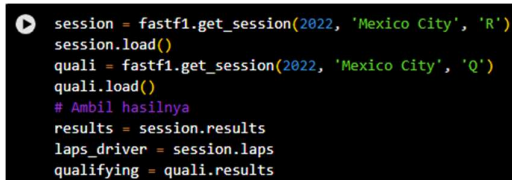
keseluruhan di sirkuit Autodromo Hermanos Rodriguez. Mercedes dan Red Bull Racing berada pada kelompok menengah dengan median sekitar 81,6–81,9 detik, sementara tim-tim kelas tengah dan belakang seperti Alpine, Aston Martin, Williams, RB, serta Kick Sauber mencatatkan median di atas 82,5 detik. Rentang IQR yang lebih lebar pada tim seperti McLaren dan Haas F1 Team mengindikasikan variabilitas strategi pit-stop dan kondisi balapan yang lebih beragam, sedangkan kotak yang lebih sempit pada tim lain menunjukkan konsistensi performa yang lebih tinggi. Secara keseluruhan, visualisasi ini memberikan gambaran awal mengenai hierarki kompetitif antar konstruktor yang menjadi dasar analisis prediksi model pada penelitian ini..

3.4 Persiapan Data (*Data Preparation*)

Pada tahap ini mencakup *preprocessing* data yaitu mempersiapkan dan memproses data sebelum dilatih oleh model. Beberapa persiapan data antara lain :

3.5.1 Pengambilan Data (*Fetching Data*)

Pada proses ini data yang digunakan dalam penelitian diperoleh menggunakan library FastF1, sebuah library *open-source* yang menyediakan antarmuka untuk mengakses data resmi dan terbaru Formula 1, mencakup data telemetri, hasil balapan, waktu putaran, dan hasil kualifikasi. Proses pengambilan data dilakukan dengan memanggil library *FastF1* terlebih dahulu menggunakan perintah *import statement*, lalu memanggil fungsi ‘fastf1.get_session()’ dengan parameter contoh tahun 2022, nama sirkuit ‘Mexico City’, dan kode tipe sesi. Proses pengambilan data dilakukan pada platform Google Colaboratory sebagai berikut:



```

session = fastf1.get_session(2022, 'Mexico City', 'R')
session.load()
quali = fastf1.get_session(2022, 'Mexico City', 'Q')
quali.load()
# Ambil hasilnya
results = session.results
laps_driver = session.laps
qualifying = quali.results

```

Gambar 3.14 Kode Program untuk Mengambil Data

Terdapat dua sesi yang diambil datanya, yaitu sesi balapan (*Race*, kode ‘R’) dan sesi kualifikasi (*Qualifying*, kode ‘Q’). Dari sesi balapan, diambil dua objek

data: ‘results’ yang berisi hasil akhir balapan setiap pembalap, serta ‘laps’ yang berisi data waktu putaran per lap untuk setiap pembalap. Dari sesi kualifikasi, diambil objek ‘results’ yang memuat waktu tercepat setiap pengemudi pada sesi Q1, Q2, dan Q3. Proses ini dilakukan secara berulang sebanyak empat kali, masing-masing untuk tahun 2022, 2023, 2024, dan 2025. Setelah proses *fetching* berhasil dilakukan, diperoleh tiga dataset awal dengan struktur sebagaimana ditunjukkan pada Tabel 3.1 berikut.

Tabel 3.1 Informasi Dataset yang Diambil		
Dataset	Jumlah Baris	Kolom Utama
Race Results	20	Abbreviation, GridPosition, Points, Q1, Q2, Q3, Time, dst.
Laps Data	±1.300	Driver, LapTime, Sector1Time, Sector2Time, Sector3Time, dst.
Qualifying Results	20	Abbreviation, Q1, Q2, Q3, dst.

Berdasarkan informasi pada Tabel 3.1, dataset yang digunakan terdiri dari tiga jenis data utama, yaitu *Race Results*, *Laps Data*, dan *Qualifying Results*. Dataset *Race Results* dan *Qualifying Results* masing-masing memiliki sekitar 20 baris data yang berisi informasi hasil balapan dan kualifikasi, seperti posisi pembalap, grid position, poin, serta waktu sesi Q1, Q2, dan Q3. Sementara itu, dataset *Laps Data* memiliki jumlah data yang jauh lebih besar, yaitu sekitar 1.300 baris, karena mencatat detail performa setiap lap pembalap, termasuk *LapTime* serta waktu pada setiap sektor lintasan. Secara keseluruhan, kombinasi ketiga dataset ini menyediakan informasi yang cukup lengkap untuk melakukan analisis performa pembalap, strategi balapan, maupun pengolahan data dalam proyek data science atau machine learning.

Data selanjutnya akan diinput ke Microsoft Excel untuk dilakukan proses penggabungan(*merging*) yang ditampilkan pada Gambar 3.18 untuk menyatukan beberapa data dari tahun yang berbeda menjadi satu kesatuan, sehingga memudahkan proses selanjutnya.

Year	Abbreviation	LapTime (s)	Sector1Time (s)	Sector2Time (s)	Sector3Time (s)	TotalSectorTime (s)	BestQuali (s)	GridPosition	Points	RacePlace (s)
2022	ALB	84.99607240	29.64015942	33.27411594	22.0817971	84.99607240	80.859	12	17	0
2022	ALO	84.82266129	30.18912903	32.75375806	21.87977419	84.82266128	78.939	19	9	0
2022	BOT	85.0383333	29.91595957	33.05205797	22.07031884	85.0383333	78.401	10	6	1
2022	GAS	84.9900808	29.72162119	33.29933333	21.96911043	84.99008085	79.672	11	14	0
2022	HAM	83.5037574	29.62117171	32.26505714	21.87573887	83.5037574	78.084	3	18	82.921
2022	LAT	86.38779412	30.23538224	33.70827941	22.44417647	86.38779412	81.167	18	18	0
2022	LEC	84.21512429	29.79811429	32.45961429	21.75748571	84.21512429	78.555	6	7	8
2022	MAG	85.15764348	30.30224338	33.28865028	22.12594652	85.15764348	79.833	17	19	0
2022	MVC	85.50472054	30.0520942	33.33514403	22.11097101	85.50472054	80.419	16	15	0
2022	NOR	84.97524038	29.84394203	33.11644928	22.01485507	84.97524038	78.721	9	8	2
2022	OCO	84.9506446	29.797	33.0889886	22.03274644	84.9506446	79.651	8	10	4
2022	PER	83.5268	29.44547143	32.31947143	21.76185714	83.5268	78.128	3	4	15
2022	RIC	84.75454628	29.68050372	33.02516037	22.00231588	84.75454627	79.325	7	11	6
2022	RUS	83.96007143	29.54845714	32.29885714	21.71305714	83.96007142	78.079	4	2	13
2022	SAR	84.0727	29.72594086	32.60752857	21.79922857	84.0727	78.351	5	5	10
2022	STR	85.38717799	30.13811294	33.13944928	21.91903217	85.38717799	80.52	15	20	0
2022	TSU	85.47893837	29.76385106	33.58287755	22.13038778	85.47893837	79.589	20	13	0
2022	VER	83.3068	29.67952857	32.09701429	21.51212714	83.3068	77.775	1	1	25
2022	VET	85.24357971	30.01255072	33.26550725	21.96551134	85.24357971	80.419	14	16	0
2022	ZHO	85.20591304	30.00317081	33.22044928	21.98208068	85.20591305	79.476	13	12	0
2023	ALO	85.53054612	30.23346118	33.34513824	21.95179472	85.53054613	79.828	9	14	2
2023	ALO	88.55184444	31.33444444	34.25782222	22.95957778	88.55184444	78.738	18	13	0
2023	BOT	86.93233333	30.47562119	34.00844928	22.46828507	86.93233334	78.036	15	9	0
2023	GAS	86.74476812	30.80891304	33.50662117	22.42927039	86.74476811	78.521	11	11	0
2023	HAM	84.03623881	29.62702085	32.74122388	21.66798507	84.0362388	77.454	2	6	19
2023	HUI	86.08049377	30.75621389	33.84081139	22.30470109	86.08049376	78.524	13	12	0
2023	LEC	84.14625373	29.50475642	32.78874627	21.80079104	84.14625373	77.146	3	1	15
2023	MAG	86.38823333	30.4373	33.64993333	22.301	86.38823333	79.163	19	16	0
2023	NOR	86.23267538	30.89404348	33.11318478	22.1107971	86.23267538	81.504	5	17	10

Gambar 3.15 Tampilan Dataset yang Telah Digabungkan

Gambar 3.15 menampilkan dataset hasil penggabungan data Formula 1 dari musim 2022 hingga 2025 yang dilakukan di-Microsoft Excel. Pada tahap pemodelan, data dari musim 2022, 2023, dan 2024 digunakan sebagai data latih (*training data*), sedangkan data musim 2025 digunakan sebagai data uji (*testing data*) untuk mengevaluasi kinerja model dalam melakukan prediksi pada data yang belum pernah digunakan selama proses pelatihan.

3.5.2 Membaca Dataset (*Load Dataset*)

Proses *load dataset* dilakukan pada platform Google Colaboratory menggunakan *library* dan fungsi Pandas. *Load dataset* ini berfungsi untuk *input* data yang akan digunakan untuk proses selanjutnya nanti. Proses penulisan kode dan hasil akan ditampilkan pada Gambar 3.16.

```
df = pd.read_csv("data_training.csv")
df.head()
```

	Year	Abbreviation	LapTime (s)	Sector1Time (s)	Sector2Time (s)	Sector3Time (s)	TotalSectorTime (s)	BestQuali (s)	GridPosition	Points	RacePlace (s)
0	2022	ALB	84.996072	29.640159	33.274116	22.081797	84.996072	80.859	17	0	84.437
1	2022	ALO	84.822661	30.189129	32.753758	21.879774	84.822661	78.939	9	0	84.425
2	2022	BOT	85.038333	29.915957	33.052058	22.070319	85.038333	78.401	6	1	84.422
3	2022	GAS	84.990087	29.721623	33.299333	21.969130	84.990087	79.672	14	0	84.349
4	2022	HAM	83.503757	29.621171	32.265057	21.875729	83.503757	78.084	3	18	82.921

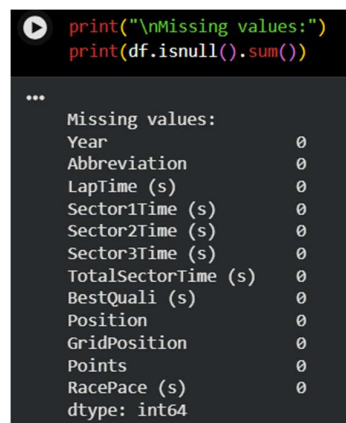
Gambar 3.16 Membaca Dataset

Pada gambar 3.16, membaca dataset dari file CSV yang telah digabungkan (*merging*) menggunakan Microsoft Excel sebelumnya. Kemudian, menggunakan fungsi `.read_csv()` dari *library* pandas untuk memuat isi file tersebut ke dalam sebuah DataFrame bernama `df` untuk diproses lebih lanjut. Fungsi `.head()` digunakan untuk

menampilkan 5 data teratas dari *dataset* yang digunakan untuk memberikan gambaran awal mengenai *dataset*.

3.5.3 Identifikasi Nilai Hilang (*Finding Missing Values*)

Data yang telah melewati proses penginputan pada tahap sebelumnya selanjutnya diidentifikasi kualitasnya untuk memastikan kelengkapan, konsistensi, dan kesesuaiannya sebelum digunakan pada tahap analisis lebih lanjut. Identifikasi nilai hilang (missing value) dilakukan untuk memastikan bahwa data yang telah dikumpulkan dan diinput tidak mengandung nilai yang kosong atau tidak terisi. Tahapan identifikasi nilai hilang tersebut ditunjukkan pada Gambar 3.17.



```

print("\nMissing values:")
print(df.isnull().sum())

...
Missing values:
Year                0
Abbreviation        0
LapTime (s)         0
Sector1Time (s)     0
Sector2Time (s)     0
Sector3Time (s)     0
TotalSectorTime (s) 0
BestQuali (s)       0
Position            0
GridPosition        0
Points              0
RacePace (s)        0
dtype: int64

```

Gambar 3.17 Kode untuk Identifikasi Nilai Hilang

Pada gambar 3.17, dilakukan pemeriksaan nilai hilang menggunakan fungsi `.isnull()` dan `.sum()` yang akan mencari nilai/data yang hilang lalu akan dijumlahkan.

3.5.4 Rekayasa Fitur (*Feature Engineering*) dan Fitur Tambahan

Setelah data dipastikan bersih dan bebas dari permasalahan kualitas data, tahap selanjutnya adalah proses rekayasa fitur. Tahap ini merupakan proses transformasi dan pembuatan variabel baru dari data mentah untuk meningkatkan kemampuan model/algoritma *machine learning* dalam menangkap pola dan menghasilkan prediksi yang lebih akurat. Tahap ini dilakukan pada platform Google Colaboratory menggunakan bahasa Python dan *library* seperti pandas dan numpy. Prosesnya adalah sebagai berikut.

a. Karakteristik Sirkuit (*Circuit Characteristics*)

Tahap rekayasa fitur pertama dilakukan dengan menambahkan fitur yang merepresentasikan karakteristik sirkuit pada dataset. Proses penambahan fitur tersebut dilakukan menggunakan bahasa pemrograman Python pada platform Google Colaboratory dan ditunjukkan pada Gambar 3.18.



```

circuit_data = {
    # Format: 'CircuitName': {fitur-fitur}
    # Mexico City Grand Prix - Autodromo Hermanos Rodriguez
    'Mexico City': {
        'circuit_length_km': 4.304,
        'num_corners': 17,
        'num straights': 3,
        'drs_zones': 2,
        'altitude_m': 2285,
        'circuit_type': 'street_permanent',
        'traction_demand': 8,
        'braking_demand': 7,
        'downforce_level': 8,
        'overtaking_difficulty': 6,
        'tire_degradation': 6,
        'safety_car_probability': 0.45,
        'lap_record_s': 76.050,
        'avg_speed_kmh': 203.0,
    },
}

```

Gambar 3.18 Kode untuk Tambahan Fitur Karakteristik Sirkuit

Pada gambar 3.18, dilakukan penambahan fitur pertama yaitu karakteristik sirkuit yang meliputi panjang sirkuit, jumlah tikungan, zona DRS (*Drag Reduction System*), dan data lainnya. Data-data teknis ini diperoleh dari beberapa sumber resmi Formula 1, sedangkan data seperti tingkat traksi dan kesulitan *overtaking* merupakan hasil *feature engineering* berdasarkan karakteristik sirkuit.

b. Kondisi Cuaca (Weather Data)

Tahap rekayasa fitur selanjutnya dilakukan dengan menambahkan fitur cuaca untuk setiap tahun pada dataset. Penambahan fitur ini bertujuan untuk memperkaya informasi yang digunakan dalam proses pemodelan sehingga dapat meningkatkan kemampuan model dalam menangkap pengaruh kondisi cuaca terhadap hasil prediksi. Proses penambahan fitur tersebut dilakukan menggunakan bahasa pemrograman Python pada platform Google Colaboratory dan ditunjukkan pada Gambar 3.19.

```

weather_data_sample = {
    # Mexico City - kondisi cuaca historis (November)
    2022: {'CircuitName': 'Mexico City', 'air_temp_c': 22.0, 'track_temp_c': 38.0,
          'humidity_pct': 45, 'wind_speed_ms': 3.2, 'rainfall_mm': 0.0,
          'is_wet_race': 0, 'pressure_hpa': 775.0}, # Tekanan rendah karena altitude tinggi
    2023: {'CircuitName': 'Mexico City', 'air_temp_c': 19.5, 'track_temp_c': 32.0,
          'humidity_pct': 55, 'wind_speed_ms': 2.8, 'rainfall_mm': 0.0,
          'is_wet_race': 0, 'pressure_hpa': 778.0},
    2024: {'CircuitName': 'Mexico City', 'air_temp_c': 21.0, 'track_temp_c': 35.0,
          'humidity_pct': 50, 'wind_speed_ms': 3.0, 'rainfall_mm': 0.0,
          'is_wet_race': 0, 'pressure_hpa': 776.0},
    2025: {'CircuitName': 'Mexico City', 'air_temp_c': 20.5, 'track_temp_c': 34.0,
          'humidity_pct': 48, 'wind_speed_ms': 3.5, 'rainfall_mm': 0.0,
          'is_wet_race': 0, 'pressure_hpa': 777.0},
}

```

Gambar 3.19 Kode untuk Tambahan Fitur Kondisi Cuaca

Pada gambar 3.19, dilakukan penambahan fitur kedua yaitu kondisi cuaca yang meliputi suhu udara, suhu lintasan, kelembapan, kecepatan angin, kondisi lintasan basah akibat hujan, dan tekanan rendah. Data-data cuaca ini bersumber dari OpenF1 API yang dikumpulkan dari musim Formula 1 tahun 2022 hingga 2025.

c. Performa *Driver*

Tahap rekayasa fitur selanjutnya dilakukan dengan mengolah fitur bawaan yang telah tersedia pada dataset untuk menghasilkan atribut baru yang merepresentasikan performa pembalap pada setiap musim. Atribut yang dihasilkan meliputi win rate, podium rate, top-5 rate, rata-rata posisi finis, rata-rata poin, serta peningkatan posisi selama balapan. Proses pengolahan fitur tersebut dilakukan menggunakan bahasa pemrograman Python pada platform Google Colaboratory dan ditunjukkan pada Gambar 3.20.

```

# 1. Win rate dan podium rate per driver per season
driver_season_stats = df_clean.groupby(['Year', 'Abbreviation']).agg(
    races_count=('Position', 'count'),
    wins=('Position', lambda x: (x == 1).sum()),
    podiums=('Position', lambda x: (x <= 3).sum()),
    top5=('Position', lambda x: (x <= 5).sum()),
    avg_position=('Position', 'mean'),
    total_points=('Points', 'sum'),
    avg_grid_pos=('GridPosition', 'mean'),
    position_gain_avg=('GridPosition', lambda x: x.mean()), # akan dikurangi avg_position
    dnf_count=('Position', lambda x: (x > 20).sum()), # proxy untuk DNF
).reset_index()

driver_season_stats['win_rate'] = driver_season_stats['wins'] / driver_season_stats['races_count']
driver_season_stats['podium_rate'] = driver_season_stats['podiums'] / driver_season_stats['races_count']
driver_season_stats['top5_rate'] = driver_season_stats['top5'] / driver_season_stats['races_count']
driver_season_stats['avg_position_gain'] = driver_season_stats['avg_grid_pos'] - driver_season_stats['avg_position']
driver_season_stats['points_per_race'] = driver_season_stats['total_points'] / driver_season_stats['races_count']

print("Driver Season Stats (sample):")
print(driver_season_stats.head(10).to_string(index=False))

```

Gambar 3.20 Kode Feature Engineering Performa *Driver* dan Tim

Pada Gambar 3.20 data mentah dikelompokkan berdasarkan tahun (*Year*) dan singkatan nama pembalap (*Abbreviation*) menggunakan operasi *groupby*, kemudian

diagregasi untuk menghasilkan variabel-variabel berikut yang akan ditampilkan pada Tabel 3.2.

Tabel 3.2 Agregasi Data per Pembalap per Musim

Variabel	Keterangan
<i>races_count</i>	Jumlah total balapan yang diikuti pembalap dalam satu musim
<i>wins</i>	Jumlah kemenangan (posisi finis = 1)
<i>podiums</i>	Jumlah podium (posisi finis ≤ 3)
<i>top5</i>	Jumlah finis di lima besar (posisi finis ≤ 5)
<i>avg_position</i>	Rata-rata posisi finis sepanjang musim
<i>total_points</i>	Total poin yang diperoleh dalam satu musim
<i>avg_grid_pos</i>	Rata-rata posisi start (<i>grid position</i>)
<i>position_gain_avg</i>	Rata-rata posisi start sebagai proksi perhitungan <i>position gain</i>
<i>dnf_count</i>	Jumlah <i>Did Not Finish</i> (DNF), yaitu balapan di mana pembalap tidak menyelesaikan lomba (posisi > 20) sebagai proksi
<i>Win Rate</i>	Proporsi kemenangan terhadap total balapan yang diikuti, diformulasikan sebagai $win_rate = wins / race_count$
<i>Podium Rate</i>	Proporsi perolehan podium terhadap total balapan, diformulasikan sebagai $podiums / races_count$
<i>Top-5_Rate</i>	Proporsi finis di posisi lima besar terhadap total balapan, diformulasikan sebagai $top5 / races_count$
<i>Average Position Gain</i>	Selisih antara rata-rata posisi start dan rata-rata posisi finis, yang mengindikasikan kemampuan pembalap dalam merebut posisi selama balapan berlangsung, diformulasikan sebagai $avg_position_gain = avg_grid_pos - avg_position$
<i>Points per Race</i>	Rata-rata poin yang diperoleh per balapan sebagai indikator konsistensi performa, diformulasikan sebagai $points_per_race = total_points / races_count$

d. Pengaruh Performa Mesin dan *Downforce* pada Ketinggian (*Mexico City*)

Tahap selanjutnya akan dilakukan rekayasa fitur untuk menambahkan atribut yang merepresentasikan pengaruh ketinggian sirkuit terhadap performa kendaraan. Proses ini dilakukan pada platform Google Colaboratory menggunakan bahasa pemrograman Python dan *library* Pandas.

```
def atmospheric_density_ratio(altitude_m):
    """Hitung rasio densitas udara vs sea level"""
    return np.exp(-altitude_m / 8500)

df_enhanced['air_density_ratio'] = atmospheric_density_ratio(
    df_enhanced.get('altitude_m', 2285)
)

# Estimasi efek altitude pada downforce dan pace
df_enhanced['downforce_penalty'] = 1 - df_enhanced['air_density_ratio'] # ~0.235 untuk Mexico
df_enhanced['altitude_pace_factor'] = df_enhanced['air_density_ratio'] # makin rendah = makin lambat (relatively)

# DRS effectiveness (lebih efektif di Mexico karena udara tipis - top speed gap lebih besar)
if 'drs_zones' in df_enhanced.columns:
    df_enhanced['drs_effectiveness'] = df_enhanced['drs_zones'] * (2 - df_enhanced['air_density_ratio'])
else:
    df_enhanced['drs_effectiveness'] = 2 * (2 - df_enhanced.get('air_density_ratio', 0.765))
```

Gambar 3.21 Kode untuk Tambahan Fitur Efek Ketinggian

Sirkuit Autodromo Hermanos Rodriguez di Mexico City berada pada ketinggian 2.285 mdpl, sehingga memiliki kondisi atmosfer yang memengaruhi performa kendaraan Formula 1. Kerapatan udara yang rendah menyebabkan downforce menurun sekitar 20% dan mengurangi performa mesin akibat berkurangnya suplai oksigen. Meskipun teknologi turbo hybrid mampu mengompensasi sebagian kehilangan tenaga, dampak atmosfer tetap signifikan. Selain itu, penggunaan sayap yang lebih besar untuk meningkatkan downforce turut meningkatkan drag dan memengaruhi kecepatan puncak kendaraan.

Untuk memodelkan pengaruh ketinggian secara kuantitatif, digunakan formula barometrik sederhana yang mengaproksimasi rasio kerapatan udara pada suatu ketinggian terhadap kerapatan udara di permukaan laut (ρ_0):

$$\frac{\rho}{\rho_0} = e^{-h/8500}$$

di mana h adalah ketinggian dalam meter dan 8500 merupakan konstanta *scale height* atmosfer bumi (dalam meter).

e. Konversi Performa Kualifikasi ke Balapan dan Selisih Pace

Rekayasa fitur selanjutnya adalah fitur relasional antara performa kualifikasi dan race. Proses ini dilakukan pada platform Google Colaboratory menggunakan bahasa pemrograman Python yang ditampilkan pada Gambar 3.22.

```
# Pace delta: selisih race pace vs quali pace (indikator konsistensi)
if 'BestQuali (s)' in df_enhanced.columns and 'RacePace (s)' in df_enhanced.columns:
    df_enhanced['quali_to_race_delta'] = df_enhanced['RacePace (s)'] - df_enhanced['BestQuali (s)']
    df_enhanced['quali_to_race_ratio'] = df_enhanced['RacePace (s)'] / df_enhanced['BestQuali (s)']

# Normalized pace vs field average (per race/year)
df_enhanced['pace_vs_avg'] = df_enhanced.groupby('Year')['RacePace (s)'].transform(
    lambda x: x - x.mean()
)
df_enhanced['quali_vs_avg'] = df_enhanced.groupby('Year')['BestQuali (s)'].transform(
    lambda x: x - x.mean()
)
print("✅ Quali-to-race pace delta features created")

# Grid position advantage (posisi start relative ke median)
if 'GridPosition' in df_enhanced.columns:
    df_enhanced['grid_advantage'] = df_enhanced.groupby('Year')['GridPosition'].transform(
        lambda x: x.median() - x # positif = lebih baik dari median
    )
```

Gambar 3.22 Kode untuk Rekayasa Fitur Performa dan *Grid Position*

Pada Gambar 3.22 Dari dua kolom yang sudah ada, yaitu *BestQuali (s)* dan *RacePace (s)*, diturunkan dua fitur baru: *quali_to_race_delta* yang merepresentasikan selisih antara pace race dan waktu kualifikasi sebagai indikator konsistensi performa pembalap, serta *quali_to_race_ratio* yang menggambarkan rasio antara keduanya. Selain itu, dibangun fitur *pace_vs_avg* dan *quali_vs_avg* yang mengukur deviasi pace masing-masing pembalap terhadap rata-rata lapangan pada tahun yang sama menggunakan agregasi *groupby* per tahun. Pendekatan normalisasi berbasis kelompok ini memungkinkan perbandingan yang lebih adil antar musim dengan kondisi kompetisi yang berbeda-beda.

Fitur kedua yang dibangun adalah *grid_advantage*, yakni keunggulan posisi start relatif terhadap nilai median posisi grid pada tahun yang sama. Nilai positif pada fitur ini menandakan bahwa pembalap memulai balapan dari posisi yang lebih baik dibanding titik tengah lapangan, sedangkan nilai negatif menunjukkan sebaliknya. Transformasi ini bertujuan untuk mengubah posisi grid dari nilai absolut menjadi ukuran relatif yang lebih informatif, khususnya dalam konteks komparasi lintas musim di mana jumlah peserta dan distribusi kualifikasi dapat bervariasi.

3.5.5 Integrasi Fitur Hasil Rekayasa

Setelah melalui tahap rekayasa fitur (feature engineering) yang menghasilkan berbagai atribut baru, seluruh fitur yang telah dibentuk kemudian diintegrasikan dengan dataset awal untuk membentuk dataset akhir yang akan digunakan pada proses pemodelan. Proses integrasi data tersebut dilakukan menggunakan bahasa pemrograman Python pada platform Google Colaboratory yang akan ditampilkan pada Gambar 3.23.

```
# FEATURE COLUMNS FINAL - GABUNGAN SEMUA FITUR

# Core features (original)
base_features = [
    'Sector1Time (s)', 'Sector2Time (s)', 'Sector3Time (s)',
    'BestQuali (s)', 'GridPosition', 'Position', 'RacePace (s)'
]

# Circuit characteristic features
circuit_features = [col for col in [
    'circuit_length_km', 'num_corners', 'num_straights', 'drs_zones',
    'altitude_m', 'traction_demand', 'braking_demand', 'downforce_level',
    'overtaking_difficulty', 'tire_degradation', 'safety_car_probability',
    'avg_speed_kmh'
] if col in df_enhanced.columns]

# Weather features
weather_features = [col for col in [
    'air_temp_c', 'track_temp_c', 'humidity_pct', 'wind_speed_ms',
    'rainfall_mm', 'is_wet_race', 'pressure_hpa'
] if col in df_enhanced.columns]

# Altitude/physics features
altitude_features = [col for col in [
    'air_density_ratio', 'downforce_penalty', 'altitude_pace_factor', 'drs_effectiveness'
] if col in df_enhanced.columns]

# Driver performance features
driver_features = [col for col in [
    'win_rate', 'podium_rate', 'top5_rate', 'avg_position_gain',
    'points_per_race', 'avg_position', 'total_points'
] if col in df_enhanced.columns]

# Derived features
derived_features = [col for col in [
    'quali_to_race_delta', 'quali_to_race_ratio', 'pace_vs_avg',
    'quali_vs_avg', 'grid_advantage', 'compound_encoded',
    'tire_age_normalized', 'tire_deg_effect'
] if col in df_enhanced.columns]

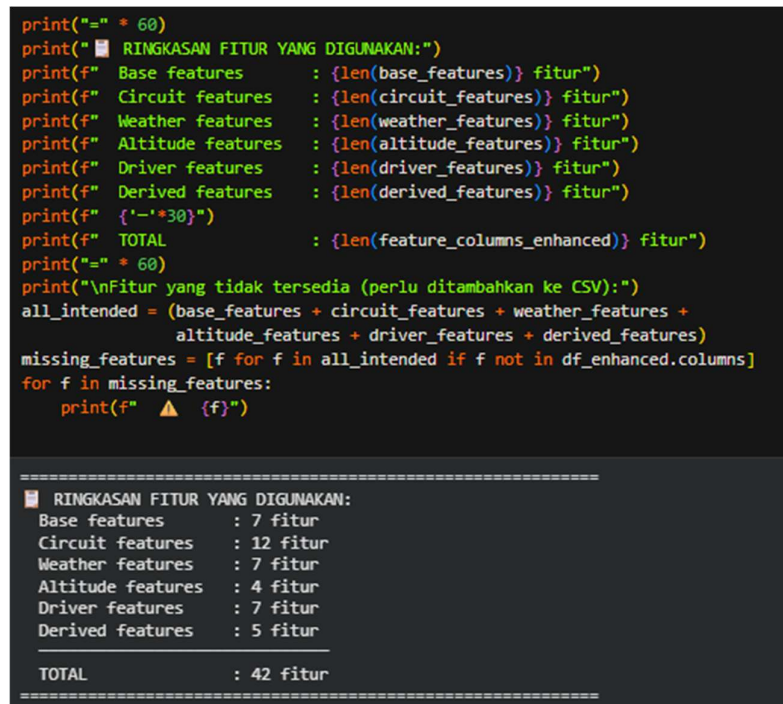
# Gabungkan semua fitur
feature_columns_enhanced = (base_features + circuit_features + weather_features +
                             altitude_features + driver_features + derived_features)

# Filter hanya kolom yang benar-benar ada
feature_columns_enhanced = [f for f in feature_columns_enhanced if f in df_enhanced.columns]
```

Gambar 3.23 Kode untuk Integrasi Fitur

Pada Gambar 3.26 telah dilakukan proses penggabungan antara fitur baru dengan dataset awal. Fitur-fitur tersebut merupakan hasil penggabungan berbagai kelompok fitur yang telah disiapkan pada tahap sebelumnya, meliputi fitur dasar (*base features*), fitur karakteristik sirkuit (*circuit features*), fitur cuaca (*weather features*), fitur pengaruh ketinggian dan aerodinamika (*altitude features*), fitur performa pembalap (*driver performance features*), serta fitur turunan (*derived features*). Setelah seluruh fitur digabungkan, dilakukan proses validasi untuk memastikan bahwa setiap fitur yang dipilih tersedia pada dataset. Hasil dari tahapan ini berupa himpunan fitur akhir (*feature set*) yang digunakan sebagai variabel masukan dalam proses pelatihan dan pengujian model prediksi. Untuk

menampilkan ringkasan fitur yang telah digabungkan, digunakan fungsi `print()` pada bahasa pemrograman Python. Ringkasan ini bertujuan untuk memudahkan interpretasi dan verifikasi jumlah serta jenis fitur yang akan digunakan dalam proses pemodelan. Proses pembuatan ringkasan akan ditampilkan pada Gambar 3.24.



```

print("=" * 60)
print(" RINGKASAN FITUR YANG DIGUNAKAN:")
print(f" Base features      : {len(base_features)} fitur")
print(f" Circuit features    : {len(circuit_features)} fitur")
print(f" Weather features     : {len(weather_features)} fitur")
print(f" Altitude features    : {len(altitude_features)} fitur")
print(f" Driver features      : {len(driver_features)} fitur")
print(f" Derived features     : {len(derived_features)} fitur")
print(f" {'-'*30}")
print(f" TOTAL                : {len(feature_columns_enhanced)} fitur")
print("=" * 60)
print("\nFitur yang tidak tersedia (perlu ditambahkan ke CSV):")
all_intended = (base_features + circuit_features + weather_features +
               altitude_features + driver_features + derived_features)
missing_features = [f for f in all_intended if f not in df_enhanced.columns]
for f in missing_features:
    print(f" ⚠ {f}")

=====
 RINGKASAN FITUR YANG DIGUNAKAN:
Base features      : 7 fitur
Circuit features   : 12 fitur
Weather features   : 7 fitur
Altitude features  : 4 fitur
Driver features    : 7 fitur
Derived features   : 5 fitur
-----
TOTAL              : 42 fitur
=====

```

Gambar 3.24 Kode dan Output Ringkasan Fitur

Berdasarkan hasil yang diperoleh pada Gambar 3.24, terdapat 7 fitur dasar (*base features*), 12 fitur karakteristik sirkuit (*circuit features*), 7 fitur cuaca (*weather features*), 4 fitur pengaruh ketinggian (*altitude features*), 7 fitur performa pembalap (*driver features*), dan 5 fitur turunan (*derived features*). Dengan demikian, jumlah keseluruhan fitur yang digunakan dalam penelitian ini adalah 42 fitur.

3.5.6 Pembagian Data Training dan Testing (*Train-Test-Split*)

Setelah seluruh fitur berhasil digabungkan ke dalam dataset akhir, dilakukan proses pembagian data menjadi dua kelompok, yaitu data pelatihan (*training data*) dan data pengujian (*testing data*). Pada penelitian ini, data balapan tahun 2022, 2023, dan 2024 digunakan sebagai data pelatihan (*training data*), sedangkan data balapan tahun 2025 digunakan sebagai data pengujian (*testing data*). Proses

pembagian data dilakukan menggunakan bahasa pemrograman Python pada platform Google Colaboratory dan ditunjukkan pada Gambar 3.25.

```
# TRAIN-TEST SPLIT DENGAN FITUR BARU

target_column = 'LapTime (s)'

# Drop missing values untuk fitur yang digunakan
df_model = df_enhanced.dropna(subset=[target_column] + feature_columns_enhanced).copy()

train_data = df_model[df_model['Year'].isin([2022, 2023, 2024])].copy()
test_data = df_model[df_model['Year'] == 2025].copy()

print(f"Training data: {len(train_data)} records")
print(f"Test data : {len(test_data)} records")
print(f"Features : {len(feature_columns_enhanced)}")

X_train = train_data[feature_columns_enhanced]
y_train = train_data[target_column]
X_test = test_data[feature_columns_enhanced]
y_test = test_data[target_column]

Training data: 60 records
Test data : 20 records
Features : 42
```

Gambar 3.25 Kode Train-Test Split

Berdasarkan hasil pembagian data yang ditunjukkan pada Gambar 3.25, diperoleh 60 baris data pelatihan (*training data*) dan 20 baris data pengujian (*testing data*). Selain itu, baik data pelatihan maupun data pengujian memiliki 42 fitur yang digunakan sebagai variabel masukan dalam proses pemodelan.

3.5.7 Penskalaan Data (*Data Scaling*)

Setelah data terbagi menjadi train dan test, data akan *discaling* untuk mengubah nilai numerik pada fitur ke dalam rentang yang sama. Proses ini menggunakan library `sklearn.preprocessing` yaitu `StandardScaler` yang akan diimplementasikan pada platform Google Colaboratory. Prosesnya seperti pada Gambar 3.26.

```
from sklearn.preprocessing import StandardScaler

scaler_enhanced = StandardScaler()
X_train_scaled = scaler_enhanced.fit_transform(X_train)
X_test_scaled = scaler_enhanced.transform(X_test)

print(f"X_train shape: {X_train_scaled.shape}")
print(f"X_test shape : {X_test_scaled.shape}")
```

Gambar 3.26 Kode Data Scaling

Berdasarkan hasil yang ditunjukkan pada Gambar 3.29, proses penskalaan data (*data scaling*) telah diterapkan pada data pelatihan (*training data*) dan data pengujian (*testing data*). Setelah data di-*scaling*, nilai data akan berubah seperti pada Gambar 3.27 yang ditampilkan menggunakan fungsi dari *library* Pandas pada platform Google Colaboratory.

```
pd.DataFrame(X_train_scaled, columns=feature_columns_enhanced).head()
```

	Sector1Time (s)	Sector2Time (s)	Sector3Time (s)	BestQuali (s)	GridPosition	Position	RacePace (s)	circuit_length_km	num_corners	num_straights	drs_zones	altitude_m	traction_demand	braking_demand	downforce
0	-0.755628	0.306211	0.013423	2.090307	1.270026	0.383140	0.301265	-8.881784e-16	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	-0.367918	-0.530806	-0.418560	0.578616	-0.131382	1.688871	0.296024	-8.881784e-16	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	-0.560846	-0.050978	-0.011121	0.155027	-0.656910	0.012359	0.294714	-8.881784e-16	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	-0.698094	0.346774	-0.227491	1.155736	0.744498	0.197750	0.262833	-8.881784e-16	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	-0.769038	-1.316902	-0.979315	-0.094559	-1.182438	-1.470762	-0.360805	-8.881784e-16	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Gambar 3.27 Tampilan Data Setelah Scaling

Melalui proses ini, seluruh fitur berada pada skala yang seragam sehingga perbedaan rentang nilai antar fitur dapat diminimalkan.

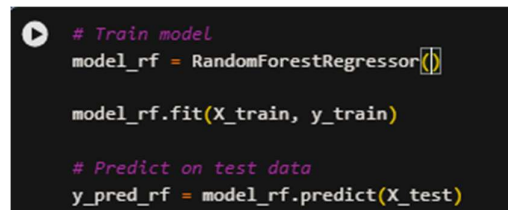
3.5 Modelling

Pada penelitian ini, tahap pemodelan dilakukan sebanyak lima kali dengan menggunakan algoritma Random Forest, Gradient Boosting, XGBoost, CatBoost, dan LightGBM. Model yang dibangun akan dievaluasi menggunakan dua konfigurasi. Konfigurasi pertama menggunakan parameter bawaan (*default parameters*) dari masing-masing algoritma, sedangkan konfigurasi kedua menggunakan parameter hasil optimasi (*hyperparameter tuning*). Proses optimasi hyperparameter dilakukan dengan beberapa metode, yaitu GridSearchCV, Bayesian Optimization, dan Optuna, untuk memperoleh kombinasi parameter yang dapat meningkatkan performa model. Penggunaan lima algoritma tersebut bertujuan untuk membandingkan kemampuan masing-masing model dalam melakukan prediksi terhadap data Formula 1 yang digunakan. Setiap algoritma dilatih menggunakan data latih yang sama dan dievaluasi menggunakan metrik regresi yang telah ditentukan sehingga dapat diketahui model yang menghasilkan performa prediksi terbaik.

3.6.1 Random Forest

Random Forest merupakan algoritma ensemble berbasis metode bagging (*bootstrap aggregating*) yang membangun sejumlah pohon keputusan (*decision*

tree) secara paralel dari subset data yang dipilih secara acak, kemudian mengagregasi prediksi seluruh pohon melalui rata-rata untuk menghasilkan output akhir (referensi). Algoritma ini dipilih karena kemampuannya menangani data dengan banyak fitur numerik, tahan terhadap overfitting, serta mampu menangkap hubungan non-linear antar variabel yang umum ditemukan dalam data balapan Formula 1. Proses pengkodean akan dilakukan pada platform Google Colaboratory yang ditunjukkan pada Gambar 3.28.



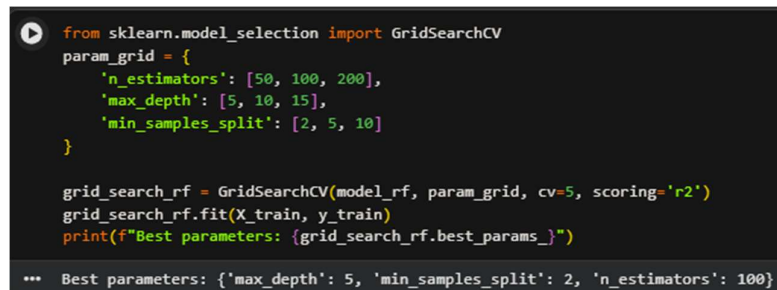
```
# Train model
model_rf = RandomForestRegressor()

model_rf.fit(X_train, y_train)

# Predict on test data
y_pred_rf = model_rf.predict(X_test)
```

Gambar 3.28 Kode Pelatihan Random Forest

Pada Gambar 3.28 dilakukan pelatihan model Random Forest menggunakan default parameter sebagai *baseline* untuk dibandingkan dengan model Random Forest yang akan dioptimasi menggunakan *GridSearchCV* dan *Bayesian Optimization*. Selanjutnya untuk tahap optimasi model (*hyperparameter tuning*) pertama akan dilakukan pada platform yang sama yang ditunjukkan pada Gambar 3.29.



```
from sklearn.model_selection import GridSearchCV
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 15],
    'min_samples_split': [2, 5, 10]
}

grid_search_rf = GridSearchCV(model_rf, param_grid, cv=5, scoring='r2')
grid_search_rf.fit(X_train, y_train)
print(f"Best parameters: {grid_search_rf.best_params_}")

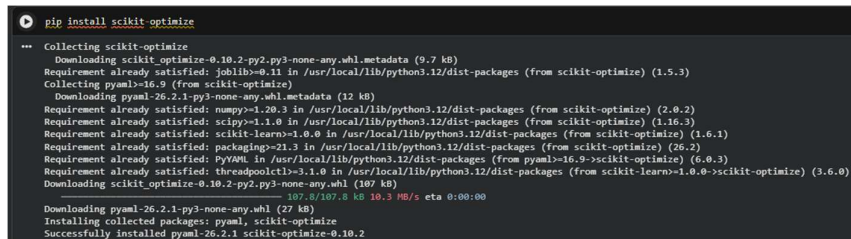
*** Best parameters: {'max_depth': 5, 'min_samples_split': 2, 'n_estimators': 100}
```

Gambar 3.29 Kode Pelatihan Random Forest (Optimasi *GridSearchCV*)

Berdasarkan Gambar 3.29, proses optimasi hyperparameter dilakukan menggunakan *GridSearchCV* dengan mendefinisikan lingkup pencarian (*param_grid*) yang terdiri dari tiga hyperparameter utama, yaitu *n_estimators* (jumlah pohon keputusan yang dibangun dalam model) dengan nilai kandidat [50, 100, 200], *max_depth* (kedalaman maksimum setiap pohon keputusan) dengan nilai kandidat [5, 10, 15], dan *min_samples_split* (jumlah minimum sampel yang

diperlukan untuk membagi suatu node menjadi node baru) dengan nilai kandidat [2, 5, 10], sehingga total terdapat 27 kombinasi hyperparameter yang dievaluasi secara exhaustive. *GridSearchCV* dijalankan dengan parameter `cv=5` yang berarti setiap kombinasi dievaluasi menggunakan 5-fold *cross-validation* pada data latih, serta `scoring='r2'` sebagai metrik pemilihan kombinasi terbaik. Setelah seluruh kombinasi dievaluasi, *GridSearchCV* berhasil menemukan konfigurasi hyperparameter optimal dengan nilai `max_depth=5`, `min_samples_split=2`, dan `n_estimators=100`. Nilai `max_depth=5` menunjukkan bahwa kedalaman pohon yang lebih dangkal lebih optimal untuk dataset ini guna mencegah overfitting, sementara `n_estimators=100` mengindikasikan bahwa 100 pohon keputusan sudah cukup untuk menghasilkan prediksi yang stabil tanpa menambah kompleksitas komputasi yang tidak perlu.

Selanjutnya untuk optimasi model (*hyperparameter tuning*) kedua yaitu *Bayesian Optimization* akan dilakukan beberapa proses terlebih dahulu seperti menginstall *library scikit-optimize* dan *import BayesSearchCV* dari *library skopt*. Prosesnya dilakukan pada platform Google Colaboratory menggunakan *package manager* Python yaitu *pip* yang ditunjukkan pada Gambar 3.33 dan Gambar 3.30.



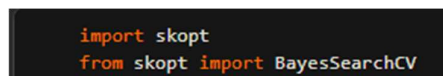
```

$ pip install scikit-optimize
Collecting scikit-optimize
  Downloading scikit-optimize-0.10.2-py2.py3-none-any.whl.metadata (9.7 kB)
Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.12/dist-packages (from scikit-optimize) (1.5.3)
Collecting pyaml>=16.9 (from scikit-optimize)
  Downloading pyaml-26.2.1-py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: numpy>=1.20.3 in /usr/local/lib/python3.12/dist-packages (from scikit-optimize) (2.0.2)
Requirement already satisfied: scipy>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-optimize) (1.16.3)
Requirement already satisfied: scikit-learn>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from scikit-optimize) (1.6.1)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.12/dist-packages (from scikit-optimize) (26.2)
Requirement already satisfied: PyYAML in /usr/local/lib/python3.12/dist-packages (from pyaml>=16.9->scikit-optimize) (6.0.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=1.0.0->scikit-optimize) (3.6.0)
Downloading scikit_optimize-0.10.2-py2.py3-none-any.whl (107 kB)
   107.8 kB/s | 100% | 0:00:00
Installing collected packages: pyaml, scikit-optimize
Successfully installed pyaml-26.2.1 scikit-optimize-0.10.2

```

Gambar 3.30 Proses Instalasi *scikit-optimize*

Setelah proses instalasi selesai, *BayesSearchCV* akan langsung diimport dari *library scikit-optimize* (*skopt*), proses ini akan dilakukan pada Google Colaboratory dengan menggunakan kode program seperti di Gambr 3.31.



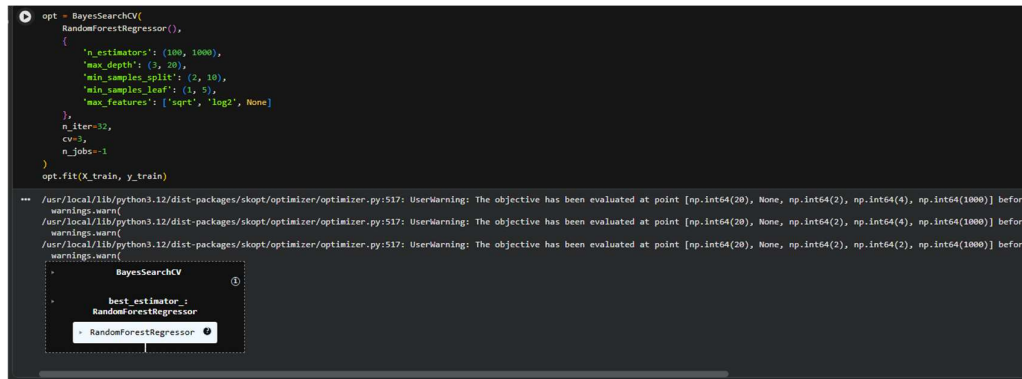
```

import skopt
from skopt import BayesSearchCV

```

Gambar 3.31 Kode untuk *import BayesSearchCV* dari *library* skopt

Setelah proses instalasi dan *import library* selesai, proses selanjutnya adalah pelatihan model Random Forest menggunakan *BayesSearchCV* yang akan dilakukan pada platform Google Colaboratory yang ditunjukan pada Gambar 3.32.



```

opt = BayesSearchCV(
    RandomForestRegressor(),
    {
        'n_estimators': (100, 1000),
        'max_depth': (3, 20),
        'min_samples_split': (2, 10),
        'min_samples_leaf': (1, 5),
        'max_features': ['sqrt', 'log2', None]
    },
    n_iter=32,
    cv=3,
    n_jobs=-1
)
opt.fit(X_train, y_train)

/usr/local/lib/python3.12/dist-packages/skopt/optimizer/optimizer.py:517: UserWarning: The objective has been evaluated at point [np.int64(20), None, np.int64(2), np.int64(4), np.int64(1000)] before
warnings.warn(
/usr/local/lib/python3.12/dist-packages/skopt/optimizer/optimizer.py:517: UserWarning: The objective has been evaluated at point [np.int64(20), None, np.int64(2), np.int64(4), np.int64(1000)] before
warnings.warn(
/usr/local/lib/python3.12/dist-packages/skopt/optimizer/optimizer.py:517: UserWarning: The objective has been evaluated at point [np.int64(20), None, np.int64(2), np.int64(4), np.int64(1000)] before
warnings.warn(

BayesSearchCV
best_estimator_:
RandomForestRegressor
RandomForestRegressor
  
```

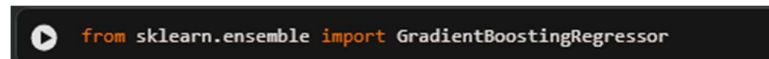
Gambar 3.32 Kode Pelatihan Random Forest (Bayesian Optimization)

Berdasarkan Gambar 3.32, optimasi model kedua dilakukan menggunakan *BayesSearchCV* Bayesian Optimization membangun model probabilistik (*surrogate model*) yang secara iteratif mempersempit ruang pencarian menuju kombinasi hyperparameter yang paling menjanjikan berdasarkan hasil evaluasi sebelumnya. Ruang pencarian didefinisikan pada lima *hyperparameter*, yaitu *n_estimators* (jumlah pohon keputusan yang dibangun dalam model) dalam rentang (100, 1000), *max_depth* (kedalaman maksimum setiap pohon keputusan) dalam rentang (3, 20), *min_samples_split* (jumlah minimum sampel yang diperlukan untuk membagi suatu node) dalam rentang (2, 10), *min_samples_leaf* (jumlah minimum sampel yang harus terdapat pada setiap node daun) dalam rentang (1, 5), serta *max_features* (jumlah fitur yang dipertimbangkan saat mencari pemisahan terbaik pada setiap node) dengan pilihan ['sqrt', 'log2', None]. Proses optimasi dijalankan sebanyak *n_iter* = 32 iterasi dengan 3-fold *cross-validation* (metode validasi yang membagi dataset menjadi tiga bagian (*fold*) sehingga setiap bagian secara bergantian digunakan sebagai data pelatihan dan data pengujian) (*cv* = 3) dan *n_jobs* = -1 (menggunakan seluruh inti prosesor yang tersedia untuk menjalankan proses secara paralel). Peringatan (*UserWarning*) yang muncul pada output mengindikasikan bahwa beberapa titik evaluasi menghasilkan kombinasi hyperparameter yang

duplikat, hal ini merupakan perilaku normal dalam proses Bayesian Optimization ketika algoritma cenderung mengeksplorasi wilayah ruang pencarian yang dianggap optimal. Hasil akhir proses optimasi menghasilkan model RandomForestRegressor terbaik yang selanjutnya digunakan untuk prediksi dan evaluasi performa.

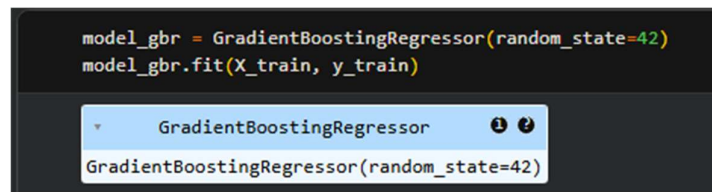
3.6.2 Gradient Boosting

Model kedua yang akan dilatih adalah Gradient Boosting. Pada penelitian ini, model diimplementasikan dalam tiga konfigurasi, yaitu konfigurasi default yaitu model Gradient Boosting akan dilatih menggunakan *default parameter* (parameter bawaan) lalu model akan dioptimasi (*hyperparameter tuning*) menggunakan GridSearchCV dengan validasi silang 5-fold; serta konfigurasi tuning kedua menggunakan Optuna. Ketiga konfigurasi ini dibandingkan untuk menganalisis pengaruh proses tuning terhadap akurasi prediksi lap time di Mexico City Grand Prix. Proses pengkodean akan dilakukan pada platform Google Colaboratory yang akan ditunjukkan pada Gambar 3.33.



Gambar 3.33 Kode Import Model Gradient Boosting Regressor

Setelah model diimport, model akan dilatih menggunakan *default parameter* terlebih dahulu. Proses pelatihan model akan dilakukan pada platform Google Colaboratory yang akan ditunjukkan pada Gambar 3.37.



Gambar 3.34 Kode Pelatihan Model Gradient Boosting (Default Parameter)

Gambar 3.34 menunjukkan proses pelatihan model Gradient Boosting menggunakan parameter default dari *library scikit-learn*, dengan `random_state=42` untuk memastikan hasil yang dapat direproduksi. Model kemudian dilatih

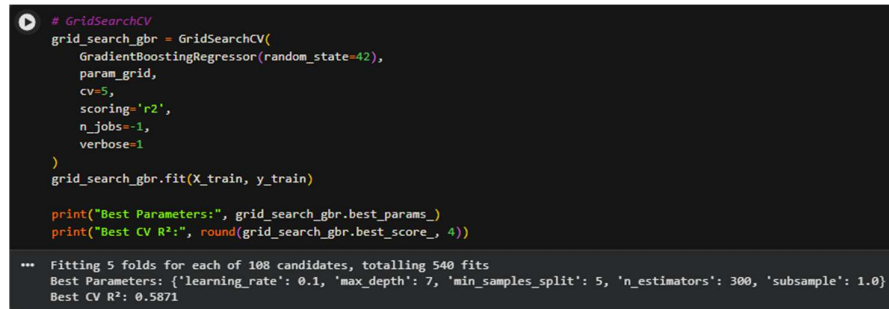
menggunakan data training yang telah disiapkan sebelumnya melalui pemanggilan fungsi `fit(X_train, y_train)`.

Selanjutnya, akan dilakukan optimasi model (*hyperparameter tuning*) pertama yaitu menggunakan GridSearchCV. Untuk proses pertama akan ditentukan dulu parameter-parameter model yang akan diuji menggunakan platform Google Colaboratory dengan bahas pemrograman Python yang ditunjukkan pada Gambar 3.35.

```
# Define parameter grid
param_grid = {
    'n_estimators':    [100, 200, 300],
    'learning_rate':   [0.01, 0.05, 0.1],
    'max_depth':       [3, 5, 7],
    'min_samples_split': [2, 5],
    'subsample':       [0.8, 1.0],
}
```

Gambar 3.35 Kode Parameter GridSearchCV Gradient Boosting

Gambar 3.35 menunjukkan pendefinisian ruang pencarian parameter (*parameter grid*) yang akan digunakan dalam proses GridSearchCV. Parameter yang diuji meliputi `n_estimators` (jumlah pohon keputusan yang dibangun dalam model) dengan nilai `[100, 200, 300]`, `learning_rate` (besar langkah pembelajaran yang mengontrol kontribusi setiap pohon terhadap model akhir) dengan nilai `[0.01, 0.05, 0.1]`, `max_depth` (kedalaman maksimum setiap pohon keputusan) dengan nilai `[3, 5, 7]`, `min_samples_split` (jumlah minimum sampel yang diperlukan untuk membagi suatu node) dengan nilai `[2, 5]`, serta `subsample` (proporsi data pelatihan yang digunakan untuk membangun setiap pohon) dengan nilai `[0.8, 1.0]`. Kombinasi seluruh parameter tersebut akan dieksplorasi secara sistematis untuk menemukan konfigurasi terbaik model Gradient Boosting. Setelah parameter ditentukan akan dilakukan pencarian parameter terbaik menggunakan GridSearchCV. Proses ini akan dilakukan pada platform Google Colaboratory menggunakan bahasa pemrograman Python yang ditunjukkan pada Gambar 3.36.



```
# GridSearchCV
grid_search_gbr = GridSearchCV(
    GradientBoostingRegressor(random_state=42),
    param_grid,
    cv=5,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)
grid_search_gbr.fit(X_train, y_train)

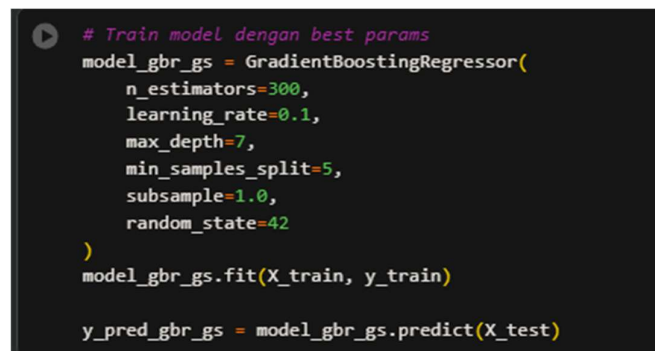
print("Best Parameters:", grid_search_gbr.best_params_)
print("Best CV R²:", round(grid_search_gbr.best_score_, 4))

*** Fitting 5 folds for each of 108 candidates, totalling 540 fits
Best Parameters: {'learning_rate': 0.1, 'max_depth': 7, 'min_samples_split': 5, 'n_estimators': 300, 'subsample': 1.0}
Best CV R²: 0.5871
```

Gambar 3.36 Kode *Hyperparameter* GradientBoost GridSearchCV

Gambar 3.36 menunjukkan proses pencarian parameter terbaik menggunakan GridSearchCV dengan validasi silang 5-fold ($cv=5$) dan metrik evaluasi R^2 ($scoring='r2'$). Proses pencarian dilakukan terhadap 108 kombinasi parameter, menghasilkan total 540 proses fitting. Hasil pencarian menunjukkan bahwa parameter terbaik yang ditemukan adalah $learning_rate=0.1$, $max_depth=7$, $min_samples_split=5$, $n_estimators=300$, dan $subsample=1.0$, dengan nilai Best CV R^2 sebesar 0.5871.

Setelah parameter terbaik model telah diketahui, model selanjutnya akan dilatih menggunakan parameter yang telah ditentukan oleh GridSearchCV sebelumnya. Proses pelatihan model akan dilakukan pada platform Google Colab yang akan ditunjukkan pada Gambar 3.37.



```
# Train model dengan best params
model_gbr_gs = GradientBoostingRegressor(
    n_estimators=300,
    learning_rate=0.1,
    max_depth=7,
    min_samples_split=5,
    subsample=1.0,
    random_state=42
)
model_gbr_gs.fit(X_train, y_train)

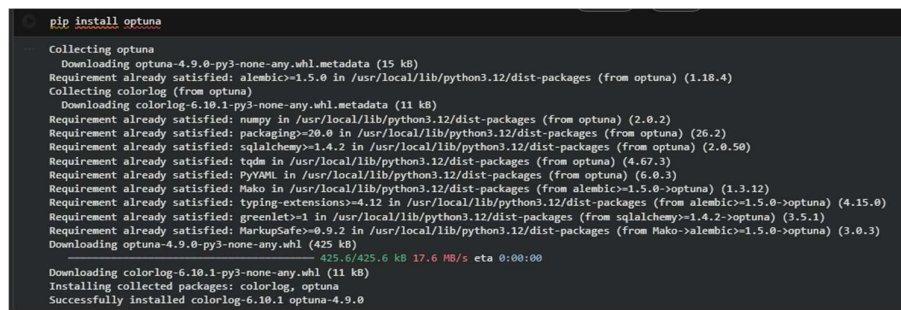
y_pred_gbr_gs = model_gbr_gs.predict(X_test)
```

Gambar 3.37 Kode Pelatihan Gradient Boost (GridSearchCV)

Gambar 3.37 menunjukkan proses pelatihan ulang model Gradient Boosting menggunakan parameter terbaik yang telah diperoleh dari GridSearchCV, yaitu $n_estimators=300$, $learning_rate=0.1$, $max_depth=7$, $min_samples_split=5$, dan $subsample=1.0$. Model kemudian dilatih pada data training melalui 'fit(X_train,

y_train)' dan digunakan untuk menghasilkan prediksi pada data testing melalui predict(X_test) yang disimpan dalam variabel y_pred_gbr_gs.

Selanjutnya akan dilakukan optimasi model kedua menggunakan Optuna untuk model Gradient Boosting. Proses ini akan dilakukan pada platform Google Colaboratory yang akan ditunjukkan pada Gambar 3.41. Untuk tahap pertama akan dilakukan proses instalasi Optuna pada menggunakan *package manager* Python yaitu pip yang akan dilakukan pada platform Google Colaboratory, ditunjukkan pada Gambar 3.38.



```

pip install optuna
Collecting optuna
  Downloading optuna-4.9.0-py3-none-any.whl.metadata (15 kB)
Requirement already satisfied: alembic>=1.5.0 in /usr/local/lib/python3.12/dist-packages (from optuna) (1.18.4)
Collecting colorlog (from optuna)
  Downloading colorlog-6.10.1-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from optuna) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from optuna) (26.2)
Requirement already satisfied: sqlalchemy>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from optuna) (2.0.50)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from optuna) (4.67.3)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from optuna) (6.0.3)
Requirement already satisfied: mako in /usr/local/lib/python3.12/dist-packages (from alembic>=1.5.0->optuna) (1.3.12)
Requirement already satisfied: typing_extensions>=4.12 in /usr/local/lib/python3.12/dist-packages (from alembic>=1.5.0->optuna) (4.15.0)
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.12/dist-packages (from sqlalchemy>=1.4.2->optuna) (3.5.1)
Requirement already satisfied: MarkupSafe>=0.9.2 in /usr/local/lib/python3.12/dist-packages (from mako->alembic>=1.5.0->optuna) (3.0.3)
Downloading optuna-4.9.0-py3-none-any.whl (425 kB)
425.6/425.6 kB 17.6 MB/s eta 0:00:00
Installing collected packages: colorlog, optuna
Successfully installed colorlog-6.10.1 optuna-4.9.0

```

Gambar 3.38 Proses Instalasi Optuna

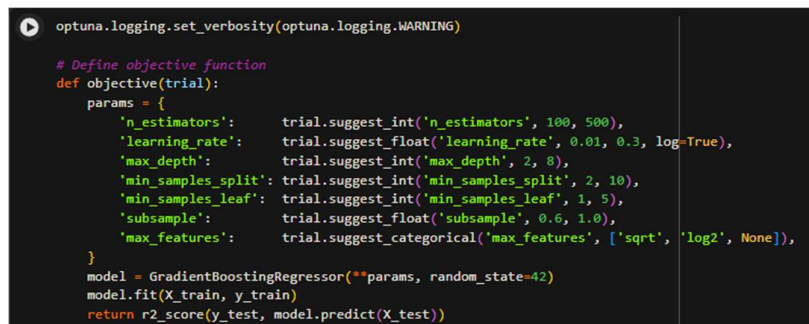
Setelah proses instalasi selesai, akan dilakukan *import library* optuna menggunakan keyword 'import' yang ditunjukkan pada Gambar 3.39.



```
import optuna
```

Gambar 3.39 Kode *Import Library* Optuna

Setelah proses *import* selesai, akan dilakukan pembuatan fungsi objektif (*objective function*) yang berisi ruang pencarian (*search space*) hyperparameter yang akan dioptimasi. Yang akan ditunjukkan pada Gambar 3.40.



```

optuna.logging.set_verbosity(optuna.logging.WARNING)

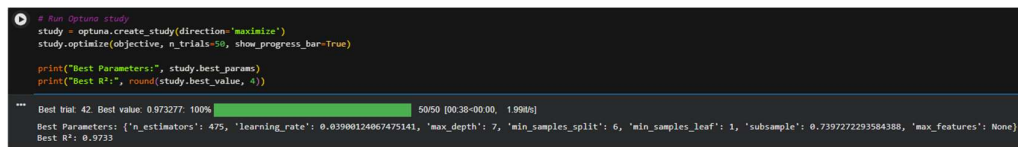
# Define objective function
def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3, log=True),
        'max_depth': trial.suggest_int('max_depth', 2, 8),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 10),
        'min_samples_leaf': trial.suggest_int('min_samples_leaf', 1, 5),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'max_features': trial.suggest_categorical('max_features', ['sqrt', 'log2', None]),
    }
    model = GradientBoostingRegressor(**params, random_state=42)
    model.fit(X_train, y_train)
    return r2_score(y_test, model.predict(X_test))

```


Gambar 3.40 Kode Ruang Pencarian Parameter Optuna

Pada fungsi ini, Optuna akan mencoba berbagai kombinasi nilai hyperparameter untuk model Gradient Boosting Regressor, kemudian mengevaluasi performanya menggunakan metrik R^2 pada data pengujian.

Setelah fungsi objektif didefinisikan, proses optimasi dijalankan menggunakan Optuna dengan membuat objek *study* yang diarahkan untuk memaksimalkan nilai R^2 yang akan ditunjukkan pada Gambar 3.41.



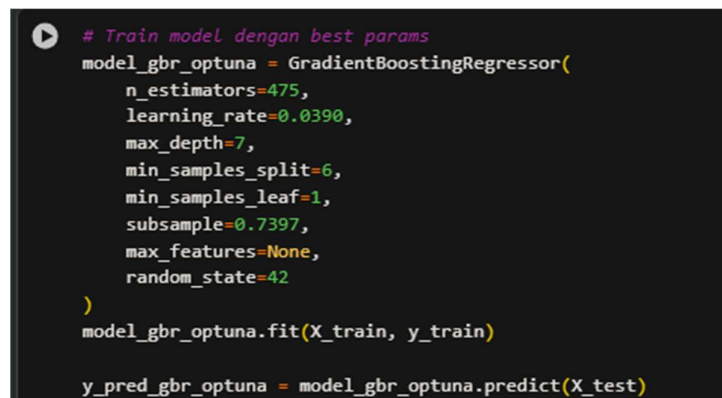
```
# Run Optuna study
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50, show_progress_bar=True)

print("Best Parameters:", study.best_params)
print("Best R²:", round(study.best_value, 4))

*** Best trial: 42. Best value: 0.973277. 100% 50/50 [00:38<00:00, 1.990s]
Best Parameters: {'n_estimators': 475, 'learning_rate': 0.03900124067475141, 'max_depth': 7, 'min_samples_split': 6, 'min_samples_leaf': 1, 'subsample': 0.7397272293584388, 'max_features': None}
Best R²: 0.9733
```

Gambar 3.41 Kode untuk Objek Study Optuna

Pada Gambar 3.41, optuna akan menjalankan pencarian sebanyak 50 *trial* dan menemukan kombinasi hyperparameter terbaik pada *trial* ke-42 dengan nilai R^2 sebesar 0.9733. Parameter optimal yang dihasilkan adalah `n_estimators=475`, `learning_rate=0.0390`, `max_depth=7`, `min_samples_split=6`, `min_samples_leaf=1`, `subsample=0.7397`, dan `max_features=None`, yang selanjutnya digunakan untuk melatih model Gradient Boosting pada tahap pelatihan akhir yang akan ditunjukkan pada Gambar 3.42.



```
# Train model dengan best params
model_gbr_optuna = GradientBoostingRegressor(
    n_estimators=475,
    learning_rate=0.0390,
    max_depth=7,
    min_samples_split=6,
    min_samples_leaf=1,
    subsample=0.7397,
    max_features=None,
    random_state=42
)
model_gbr_optuna.fit(X_train, y_train)

y_pred_gbr_optuna = model_gbr_optuna.predict(X_test)
```

Gambar 3.42 Kode Pelatihan Gradient Boosting (Optuna Parameter)

Gambar 3.42 menunjukkan proses pelatihan model Gradient Boosting menggunakan parameter optimal hasil pencarian Optuna. Model dilatih pada data training melalui '`fit(X_train, y_train)`' dan menghasilkan prediksi pada data testing yang disimpan dalam variabel `y_pred_gbr_optuna`.

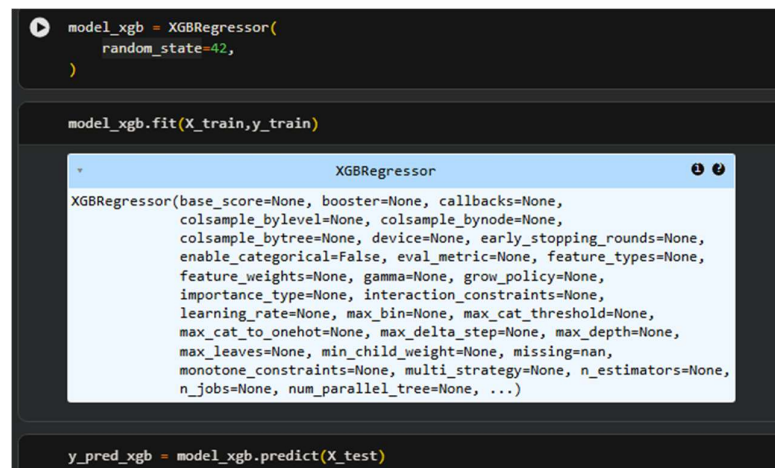
3.6.3 XGBoost

Model ketiga yang dilatih adalah XGBoost. Pada penelitian ini, model XGBoost akan dilatih dalam tiga konfigurasi, yaitu pelatihan model dengan *default parameter* dan pelatihan model menggunakan 2 proses *hyperparameter tuning* seperti model sebelumnya. Optimasi model XGBoost akan dilakukan menggunakan GridSearchCV dan optuna sebanyak 50 iterasi. Kedua konfigurasi ini dibandingkan untuk menganalisis pengaruh proses tuning terhadap akurasi prediksi lap time di *Mexico City Grand Prix*. Proses pengkodean akan dilakukan pada platform Google Colaboratory dengan beberapa tahap. Tahap pertama akan dilakukan proses *import* model dari *library* xgboost menggunakan keyword ‘from’ dan ‘import’ yang akan ditunjukkan pada Gambar 3.43.



Gambar 3.43 Kode Import Model XGBRegressor

Setelah model diimport, model akan langsung dilatih menggunakan *default parameter* terlebih dahulu yang akan ditunjukkan pada Gambar 3.47.

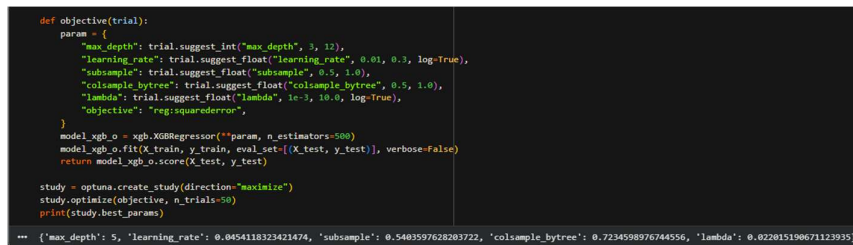


Gambar 3.44 Kode Pelatihan Model XGBRegressor

Gambar 3.44 menunjukkan proses pelatihan model XGBoost menggunakan parameter default dari library XGBoost, dengan `random_state=42` untuk memastikan hasil yang dapat direproduksi. Model dilatih menggunakan data training melalui ‘`fit(X_train, y_train)`’ dan menghasilkan prediksi pada data testing yang

disimpan dalam variabel 'y_pred_xgb'. *Tooltip* yang muncul memperlihatkan seluruh parameter yang tersedia pada kelas XGBRegressor, di mana parameter yang tidak didefinisikan secara eksplisit akan menggunakan nilai default masing-masing.

Selanjutnya akan dilakukan optimasi model menggunakan optuna. Untuk proses instalasi dan *import* tidak dilakukan pada tahap ini karena sudah dilakukan pada model sebelumnya. Oleh karena itu tahapan pertama akan menentukan ruang pencarian untuk beberapa parameter yang akan dilakukan pada platform Google Colaboratory menggunakan bahasa pemrograman Python, ditunjukkan pada Gambar 3.45.



```
def objective(trial):
    param = {
        "max_depth": trial.suggest_int("max_depth", 3, 12),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3, log=True),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0),
        "lambda": trial.suggest_float("lambda", 1e-3, 10.0, log=True),
        "objective": "reg:squarederror",
    }
    model_xgb_o = xgb.XGBRegressor(**param, n_estimators=500)
    model_xgb_o.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=False)
    return model_xgb_o.score(X_test, y_test)

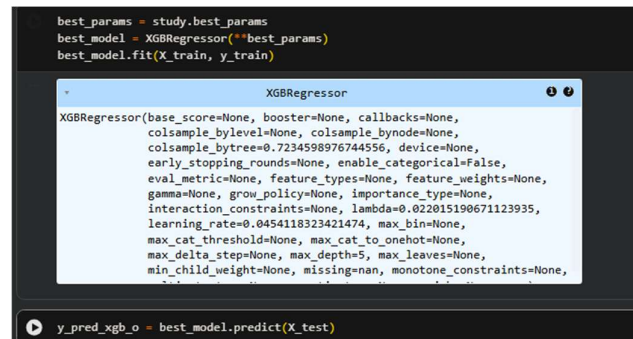
study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=50)
print(study.best_params)
```

```
*** {'max_depth': 5, 'learning_rate': 0.0454118323421474, 'subsample': 0.5403597628203722, 'colsample_bytree': 0.7234598976744556, 'lambda': 0.022015190671123935}
```

Gambar 3.45 Proses Optimasi Hyperparameter XGBoost (Optuna)

Gambar 3.45 menunjukkan pendefinisian fungsi objektif dan proses optimasi hyperparameter XGBoost menggunakan Optuna. Fungsi objektif mendefinisikan ruang pencarian parameter yang meliputi max_depth pada rentang [3, 12], learning_rate pada rentang [0.01, 0.3], subsample pada rentang [0.5, 1.0], colsample_bytree pada rentang [0.5, 1.0], serta lambda pada rentang [1e-3, 10.0], dengan fungsi loss reg:squarederror. Setiap trial melatih model XGBRegressor dengan n_estimators=500 dan mengembalikan nilai R^2 pada data testing sebagai metrik evaluasi. Optuna kemudian menjalankan pencarian sebanyak 50 trial dengan tujuan memaksimalkan nilai R^2 , dan menghasilkan parameter terbaik yaitu max_depth=5, learning_rate=0.0454, subsample=0.5404, colsample_bytree=0.7235, dan lambda=0.0220.

Tahap selanjutnya, model akan dilatih menggunakan parameter terbaik yang sudah ditentukan sebelumnya dan disimpan pada variabel yang bernama 'best_params' yang akan ditunjukkan pada Gambar 3.46.



```
best_params = study.best_params
best_model = XGBRegressor(**best_params)
best_model.fit(X_train, y_train)

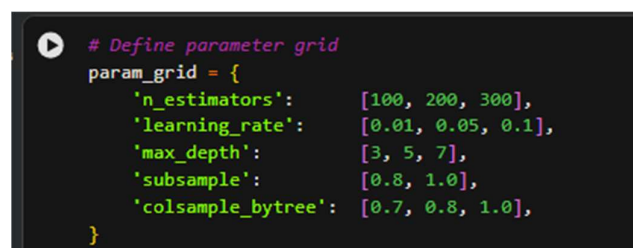
XGBRegressor(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=0.7234598976744556, device=None,
              early_stopping_rounds=None, enable_categorical=False,
              eval_metric=None, feature_types=None, feature_weights=None,
              gamma=None, grow_policy=None, importance_type=None,
              interaction_constraints=None, lambda=0.022015190671123935,
              learning_rate=0.0454118323421474, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=5, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              num_parallel_tree=None, random_state=None,
              reg_alpha=None, reg_lambda=None,
              silent=None, subsample=None, tree_method=None,
              validate_features=None, verbosity=None,
              warm_start=None)

y_pred_xgb_o = best_model.predict(X_test)
```

Gambar 3.46 Kode Pelatihan Model XGBoost dengan Parameter Optuna

Gambar 3.46 menunjukkan proses pelatihan model XGBoost menggunakan parameter optimal hasil pencarian Optuna. Parameter terbaik yang tersimpan dalam `study.best_params` digunakan langsung untuk menginisialisasi `XGBRegressor` melalui `unpacking **best_params`, sehingga model dikonfigurasi dengan `colsample_bytree=0.7235`, `lambda=0.0220`, `learning_rate=0.0454`, dan `max_depth=5`. Model kemudian dilatih pada data training melalui `'fit(X_train, y_train)'` dan menghasilkan prediksi pada data testing yang disimpan dalam variabel `'y_pred_xgb_o'`.

Untuk optimasi model kedua akan dilakukan menggunakan `GridSearchCV`. Tahap pertama akan dilakukan pendefinisian ruang pencarian *hyperparameter* (*parameter grid*). Proses ini dilakukan pada platform Google Colaboratory menggunakan bahasa pemrograman Python yang ditunjukkan pada Gambar 3.50.



```
# Define parameter grid
param_grid = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.7, 0.8, 1.0],
}
```

Gambar 3.47 Pendefinisian *Hyperparameter Grid* Model XGBoost

Gambar 3.47 menunjukkan pendefinisian ruang pencarian parameter (*parameter grid*) yang akan digunakan dalam proses `GridSearchCV` untuk model XGBoost. Parameter yang diuji meliputi `n_estimators` dengan nilai `[100, 200, 300]`, `learning_rate` dengan nilai `[0.01, 0.05, 0.1]`, `max_depth` dengan nilai `[3, 5, 7]`, `subsample` dengan nilai `[0.8, 1.0]`, serta `colsample_bytree` dengan nilai `[0.7, 0.8, 1.0]`.

1.0]. Selanjutnya GridSearchCV akan melakukan pencarian parameter terbaik dari parameter yang sudah ditentukan, ditunjukkan pada Gambar 3.48.

```
# GridSearchCV
grid_search_xgb = GridSearchCV(
    XGBRegressor(random_state=42, verbosity=0),
    param_grid,
    cv=5,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)
grid_search_xgb.fit(X_train, y_train)

print("Best Parameters:", grid_search_xgb.best_params_)
print("Best CV R²:", round(grid_search_xgb.best_score_, 4))

Fitting 5 folds for each of 162 candidates, totalling 810 fits
Best Parameters: {'colsample_bytree': 1.0, 'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 300, 'subsample': 0.8}
Best CV R²: 0.5696
```

Gambar 3.48 Kode *Hyperparameter* XGBoost (GridSearchCV)

Gambar 3.48 menunjukkan proses pencarian parameter terbaik XGBoost menggunakan GridSearchCV dengan validasi silang 5-fold ($cv=5$) dan metrik evaluasi R^2 ($scoring='r2'$). Proses pencarian dilakukan terhadap 162 kombinasi parameter, menghasilkan total 810 proses *fitting*. Hasil pencarian menunjukkan bahwa parameter terbaik yang ditemukan adalah `colsample_bytree=1.0`, `learning_rate=0.1`, `max_depth=3`, `n_estimators=300`, dan `subsample=0.8`, dengan nilai Best CV R^2 sebesar 0.5696.

Model selanjutnya akan dilatih menggunakan parameter terbaik yang sudah ditentukan yang ditunjukkan pada Gambar 3.49.

```
# Train model dengan best params
model_xgb_gs = XGBRegressor(
    **grid_search_xgb.best_params_,
    random_state=42,
    verbosity=0
)
model_xgb_gs.fit(X_train, y_train)

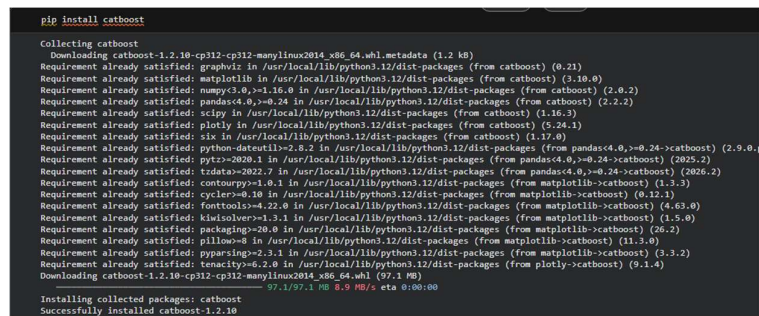
y_pred_xgb_gs = model_xgb_gs.predict(X_test)
```

Gambar 3.49 Kode Pelatihan XGBoost (GridSearchCV)

Gambar 3.49 menunjukkan proses pelatihan ulang model XGBoost menggunakan parameter terbaik yang telah diperoleh dari GridSearchCV melalui *unpacking* `**grid_search_xgb.best_params_`, sehingga seluruh parameter optimal diterapkan secara otomatis tanpa perlu didefinisikan satu per satu. Model dilatih pada data *training* melalui `fit(X_train, y_train)` dan menghasilkan prediksi pada data testing yang disimpan dalam variabel `y_pred_xgb_gs`.

3.6.4 CatBoost

Model keempat yang dilatih adalah CatBoost. Pada penelitian ini model CatBoost juga dilatih dengan tiga konfigurasi yang sama yaitu pelatihan model menggunakan *default parameter* dan 2 *hyperparameter tuning*. Untuk *hyperparameter tuning* akan sama seperti model selanjutnya, menggunakan GridSearchCV dan optuna untuk menjaga konsistensi ruang lingkup model. Proses pelatihan model akan dilakukan menggunakan platform Google Colaboratory dan bahasa pemrograman. Proses ini akan melalui beberapa tahapan, tahapan pertama akan dilakukan proses instalasi model menggunakan *package manager* Python yaitu pip yang akan dilakukan pada platform Google Colaboratory pada Gambar 3.50.

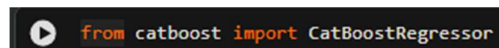


```

pip install catboost
Collecting catboost
  Downloading catboost-1.2.10-cp312-cp312-manylinux2014_x86_64.whl.metadata (1.2 kB)
Requirement already satisfied: graphviz in /usr/local/lib/python3.12/dist-packages (from catboost) (0.21)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from catboost) (3.10.0)
Requirement already satisfied: numpy<3.0,>=1.16.0 in /usr/local/lib/python3.12/dist-packages (from catboost) (2.0.2)
Requirement already satisfied: pandas<4.0,>=0.24 in /usr/local/lib/python3.12/dist-packages (from catboost) (2.2.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from catboost) (1.16.3)
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (from catboost) (5.24.1)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from catboost) (1.17.0)
Requirement already satisfied: python-dateutil<=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas<4.0,>=0.24->catboost) (2.9.0)
Requirement already satisfied: pytz<=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<4.0,>=0.24->catboost) (2025.2)
Requirement already satisfied: tzdata<=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<4.0,>=0.24->catboost) (2026.2)
Requirement already satisfied: contourpy<=1.4.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (1.3.3)
Requirement already satisfied: cycler<=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (0.12.1)
Requirement already satisfied: fonttools<=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (4.63.0)
Requirement already satisfied: kdtree<=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (1.5.0)
Requirement already satisfied: packaging<=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (26.2)
Requirement already satisfied: pillow<=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (11.3.0)
Requirement already satisfied: pyparsing<=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (3.3.2)
Requirement already satisfied: tenacity<=6.2.0 in /usr/local/lib/python3.12/dist-packages (from plotly->catboost) (5.1.4)
Downloading catboost-1.2.10-cp312-cp312-manylinux2014_x86_64.whl (97.1 MB)
  97.1/97.1 MB 0.9 MB/s eta 0:00:00
Installing collected packages: catboost
Successfully installed catboost-1.2.10
  
```

Gambar 3.50 Proses Instalasi Model CatBoost

Setelah proses instalasi yang ditunjukkan pada Gambar 3.53 selesai, model CatBoostRegressor akan dipanggil menggunakan *keyword* Python yaitu ‘from’ dan ‘import’, ditunjukkan pada Gambar 3.51.

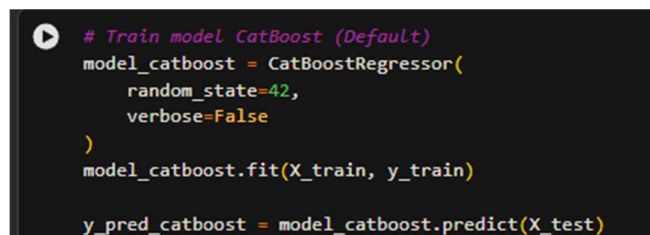


```

from catboost import CatBoostRegressor
  
```

Gambar 3.51 Kode *Import* Model CatBoostRegressor

Setelah proses *import* model selesai, akan dilakukan pelatihan model menggunakan konfigurasi pertama yaitu *default parameters*. Proses pelatihan tetap dilakukan pada platform Google Colaboratory yang ditunjukkan pada Gambar 3.55.



```

# Train model CatBoost (Default)
model_catboost = CatBoostRegressor(
    random_state=42,
    verbose=False
)
model_catboost.fit(X_train, y_train)

y_pred_catboost = model_catboost.predict(X_test)
  
```

Gambar 3.52 Kode Pelatihan Model CatBoost (*Default Parameters*)

Gambar 3.52 menunjukkan proses pelatihan model CatBoost menggunakan parameter bawaan dari *library* CatBoost. Ada beberapa parameter yang dideklarasikan seperti `random_state=42` untuk memastikan hasil yang dapat direproduksi dan `verbose=False` untuk menonaktifkan *log output* selama proses pelatihan. Model dilatih menggunakan data *training* melalui `'fit(X_train, y_train)'` dan menghasilkan prediksi pada data *testing* yang disimpan dalam variabel `y_pred_catboost`.

Tahap selanjutnya akan dilakukan pelatihan model dengan konfigurasi kedua yaitu menggunakan optimasi parameter GridSearchCV. Tahap pertama dalam optimasi yaitu mendefinisikan ruang pencarian *hyperparameter* (*parameter grid*), ditunjukkan pada Gambar 3.53 yang nantinya akan dicoba selama proses *hyperparameter tuning*.

```
# Define parameter grid
param_grid = {
    'iterations': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1],
    'depth': [4, 6, 8],
    'l2_leaf_reg': [1, 3, 5],
}
```

Gambar 3.53 Pendefinisian Hyperparameter Grid Model CatBoost

Selanjutnya, GridSearchCV akan melakukan pencarian kombinasi *hyperparameter* terbaik berdasarkan parameter grid yang telah ditentukan. Proses tersebut dijalankan menggunakan platform Google Colaboratory, sebagaimana ditunjukkan pada Gambar 3.54.

```
# GridSearchCV
grid_search_catboost = GridSearchCV(
    CatBoostRegressor(random_state=42, verbose=False),
    param_grid,
    cv=5,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)
grid_search_catboost.fit(X_train, y_train)

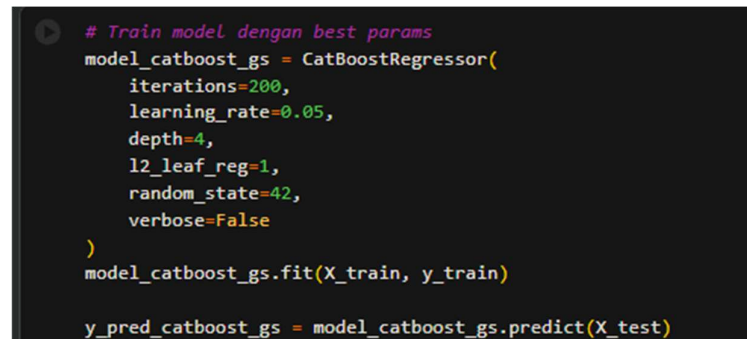
print("Best Parameters:", grid_search_catboost.best_params_)
print("Best CV R²:", round(grid_search_catboost.best_score_, 4))

*** Fitting 5 folds for each of 81 candidates, totalling 405 fits
Best Parameters: {'depth': 4, 'iterations': 200, 'l2_leaf_reg': 1, 'learning_rate': 0.05}
Best CV R²: 0.6052
```


Gambar 3.54 Pencarian Parameter Terbaik oleh GridSearchCV (CatBoost)

Gambar 3.54 menunjukkan proses pencarian parameter terbaik menggunakan GridSearchCV dengan validasi silang *5-fold* ($cv=5$) dan metrik evaluasi R^2 ($scoring='r2'$). Proses pencarian dilakukan terhadap 81 kombinasi parameter, menghasilkan total 405 proses fitting. Hasil pencarian menunjukkan bahwa parameter terbaik yang ditemukan adalah $depth=4$, $iterations=200$, $l2_leaf_reg=1$, dan $learning_rate=0.05$, dengan nilai Best CV R^2 sebesar 0.6052.

Setelah parameter terbaik ditemukan, model akan dilatih menggunakan parameter tersebut menggunakan data *training*. Proses-nya akan dilakukan pada platform Google Colaboratory yang ditunjukkan pada Gambar 3.55.



```
# Train model dengan best params
model_catboost_gs = CatBoostRegressor(
    iterations=200,
    learning_rate=0.05,
    depth=4,
    l2_leaf_reg=1,
    random_state=42,
    verbose=False
)
model_catboost_gs.fit(X_train, y_train)

y_pred_catboost_gs = model_catboost_gs.predict(X_test)
```

Gambar 3.55 Kode Pelatihan Model CatBoost (GridSearchCV)

Gambar 3.55 menunjukkan proses pelatihan ulang model CatBoost menggunakan parameter terbaik yang telah diperoleh dari GridSearchCV, yaitu $iterations=200$, $learning_rate=0.05$, $depth=4$, dan $l2_leaf_reg=1$. Model dilatih pada data training melalui ‘ $fit(X_train, y_train)$ ’ dan menghasilkan prediksi pada data testing yang disimpan dalam variabel ‘ $y_pred_catboost_gs$ ’.

Tahap selanjutnya, akan dilakukan pelatihan model menggunakan konfigurasi ketiga yaitu optimasi parameter menggunakan Optuna. Tidak dilakukan proses instalasi dan *import* karena sudah dilakukan pada tahapan model sebelumnya. Oleh karena itu dilakukan pendefinisian fungsi objektif (*objective function*) pada Optuna yang bertujuan untuk mencari kombinasi *hyperparameter* terbaik untuk model CatBoost. Proses ini dilakukan pada platform Google Colaboratory yang ditunjukkan pada Gambar 3.56.


```

# Define objective function
def objective(trial):
    params = {
        'iterations': trial.suggest_int('iterations', 100, 500),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3, log=True),
        'depth': trial.suggest_int('depth', 3, 10),
        'l2_leaf_reg': trial.suggest_float('l2_leaf_reg', 1e-3, 10.0, log=True),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bylevel': trial.suggest_float('colsample_bylevel', 0.5, 1.0),
    }
    model = CatBoostRegressor(**params, random_state=42, verbose=False)
    model.fit(X_train, y_train)
    return r2_score(y_test, model.predict(X_test))

```

Gambar 3.56 Kode *Objective Function* Optuna CatBoost

Gambar 3.56 menunjukkan pendefinisian fungsi objektif untuk proses optimasi hyperparameter CatBoost menggunakan Optuna. Fungsi objektif mendefinisikan ruang pencarian parameter yang meliputi `iterations` pada rentang [100, 500], `learning_rate` pada rentang [0.01, 0.3], `depth` pada rentang [3, 10], `l2_leaf_reg` pada rentang [1e-3, 10.0], `subsample` pada rentang [0.6, 1.0], serta `colsample_bylevel` pada rentang [0.5, 1.0]. Setiap trial melatih model CatBoostRegressor dengan kombinasi parameter yang disarankan Optuna dan mengembalikan nilai R² pada data testing sebagai metrik evaluasi yang akan dimaksimalkan. Proses selanjutnya adalah optimasi *hyperparameter* CatBoost menggunakan Optuna dengan menjalankan 50 *trial* pencarian. Ditunjukkan pada Gambar 3.57.

```

# Run Optuna study
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50, show_progress_bar=True)

print('Best Parameters:', study.best_params)
print('Best R^2:', study.best_value)

[1] 2020-06-20 11:17:38.261 | A new study created in memory with name: no-name-7072ab6d-8888-413f-8783-6d9fab80754
Best loss: 30. Best value: 0.6800000000000001
[2] 2020-06-20 11:17:48.457 | Trial 0 finished with value: 0.4994026364127067 and parameters: {'iterations': 449, 'learning_rate': 0.0788044800955067, 'depth': 7, 'l2_leaf_reg': 0.001376111262718137, 'subsample': 0.8192549575, 'colsample_bylevel': 0.97516995469551}
[3] 2020-06-20 11:17:49.471 | Trial 1 finished with value: 0.16572677720066 and parameters: {'iterations': 432, 'learning_rate': 0.09046513218183, 'depth': 9, 'l2_leaf_reg': 0.0038486121296, 'subsample': 0.479216995469551, 'colsample_bylevel': 0.798427740881}
[4] 2020-06-20 11:17:49.864 | Trial 2 finished with value: 0.35848987678306 and parameters: {'iterations': 274, 'learning_rate': 0.4129647380183737, 'depth': 9, 'l2_leaf_reg': 0.00857775425651, 'subsample': 0.798427740881, 'colsample_bylevel': 0.798427740881}
[5] 2020-06-20 11:17:49.873 | Trial 3 finished with value: 0.34547608020736 and parameters: {'iterations': 449, 'learning_rate': 0.06454397312166, 'depth': 9, 'l2_leaf_reg': 0.07326302167575, 'subsample': 0.7134476868252, 'colsample_bylevel': 0.7134476868252}
[6] 2020-06-20 11:17:49.854 | Trial 4 finished with value: 0.308453351098667 and parameters: {'iterations': 359, 'learning_rate': 0.2211283630810808, 'depth': 9, 'l2_leaf_reg': 0.00157383820383656, 'subsample': 0.62708447276, 'colsample_bylevel': 0.62708447276}
[7] 2020-06-20 11:17:49.843 | Trial 5 finished with value: 0.0774444020236 and parameters: {'iterations': 189, 'learning_rate': 0.17476428197168, 'depth': 5, 'l2_leaf_reg': 0.0707100812582626, 'subsample': 0.828724342165, 'colsample_bylevel': 0.828724342165}
[8] 2020-06-20 11:17:49.843 | Trial 6 finished with value: 0.781747474610532 and parameters: {'iterations': 472, 'learning_rate': 0.406574727987983, 'depth': 3, 'l2_leaf_reg': 0.0401521682626567, 'subsample': 0.95826629760, 'colsample_bylevel': 0.95826629760}
[9] 2020-06-20 11:17:49.823 | Trial 7 finished with value: 0.0774444020236 and parameters: {'iterations': 189, 'learning_rate': 0.17476428197168, 'depth': 5, 'l2_leaf_reg': 0.0707100812582626, 'subsample': 0.828724342165, 'colsample_bylevel': 0.828724342165}
[10] 2020-06-20 11:17:49.799 | Trial 8 finished with value: 0.382488154221383 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[11] 2020-06-20 11:17:49.799 | Trial 9 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[12] 2020-06-20 11:17:49.799 | Trial 10 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[13] 2020-06-20 11:17:49.799 | Trial 11 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[14] 2020-06-20 11:17:49.799 | Trial 12 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[15] 2020-06-20 11:17:49.799 | Trial 13 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[16] 2020-06-20 11:17:49.799 | Trial 14 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[17] 2020-06-20 11:17:49.799 | Trial 15 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[18] 2020-06-20 11:17:49.799 | Trial 16 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[19] 2020-06-20 11:17:49.799 | Trial 17 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[20] 2020-06-20 11:17:49.799 | Trial 18 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[21] 2020-06-20 11:17:49.799 | Trial 19 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[22] 2020-06-20 11:17:49.799 | Trial 20 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[23] 2020-06-20 11:17:49.799 | Trial 21 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[24] 2020-06-20 11:17:49.799 | Trial 22 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[25] 2020-06-20 11:17:49.799 | Trial 23 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[26] 2020-06-20 11:17:49.799 | Trial 24 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[27] 2020-06-20 11:17:49.799 | Trial 25 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[28] 2020-06-20 11:17:49.799 | Trial 26 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[29] 2020-06-20 11:17:49.799 | Trial 27 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[30] 2020-06-20 11:17:49.799 | Trial 28 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[31] 2020-06-20 11:17:49.799 | Trial 29 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[32] 2020-06-20 11:17:49.799 | Trial 30 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[33] 2020-06-20 11:17:49.799 | Trial 31 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[34] 2020-06-20 11:17:49.799 | Trial 32 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[35] 2020-06-20 11:17:49.799 | Trial 33 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[36] 2020-06-20 11:17:49.799 | Trial 34 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[37] 2020-06-20 11:17:49.799 | Trial 35 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[38] 2020-06-20 11:17:49.799 | Trial 36 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[39] 2020-06-20 11:17:49.799 | Trial 37 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[40] 2020-06-20 11:17:49.799 | Trial 38 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[41] 2020-06-20 11:17:49.799 | Trial 39 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[42] 2020-06-20 11:17:49.799 | Trial 40 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[43] 2020-06-20 11:17:49.799 | Trial 41 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[44] 2020-06-20 11:17:49.799 | Trial 42 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[45] 2020-06-20 11:17:49.799 | Trial 43 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[46] 2020-06-20 11:17:49.799 | Trial 44 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[47] 2020-06-20 11:17:49.799 | Trial 45 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[48] 2020-06-20 11:17:49.799 | Trial 46 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[49] 2020-06-20 11:17:49.799 | Trial 47 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[50] 2020-06-20 11:17:49.799 | Trial 48 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[51] 2020-06-20 11:17:49.799 | Trial 49 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
[52] 2020-06-20 11:17:49.799 | Trial 50 finished with value: 0.420164170990413 and parameters: {'iterations': 287, 'learning_rate': 0.04233931321854613, 'depth': 10, 'l2_leaf_reg': 0.00122086836163862, 'subsample': 0.8199484131, 'colsample_bylevel': 0.8199484131}
Best Parameters: {'iterations': 286, 'learning_rate': 0.12746018007796, 'depth': 4, 'l2_leaf_reg': 0.041302634324, 'subsample': 0.927513253164, 'colsample_bylevel': 0.9087762064677}
Best R^2: 0.6800

```

Gambar 3.57 Proses Optimasi *Hyperparameter* Model CatBoost (Optuna)

Pada Gambar 3.57 Optuna membuat objek study dengan tujuan memaksimalkan nilai R² melalui `direction='maximize'`, kemudian menjalankan setiap trial secara adaptif berdasarkan hasil evaluasi sebelumnya. Proses optimasi selesai dalam

waktu 23 menit dan menemukan kombinasi terbaik pada trial ke-30 dengan nilai R^2 sebesar 0.8595. Parameter optimal yang dihasilkan adalah `iterations=300`, `learning_rate=0.1272`, `depth=4`, `l2_leaf_reg=8.5813`, `subsample=0.7257`, dan `colsample_bylevel=0.9967`, yang selanjutnya digunakan untuk melatih model CatBoost pada tahap pelatihan akhir yang akan ditunjukkan pada Gambar 3.58.

```
# Train model dengan best params
model_catboost_optuna = CatBoostRegressor(
    iterations=280,
    learning_rate=0.1118,
    depth=4,
    l2_leaf_reg=5.5314,
    subsample=0.8280,
    colsample_bylevel=0.9613,
    random_state=42,
    verbose=False
)
model_catboost_optuna.fit(X_train, y_train)

y_pred_catboost_optuna = model_catboost_optuna.predict(X_test)
```

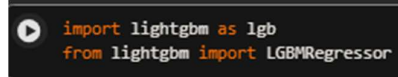
Gambar 3.58 Kode Pelatihan Model CatBoost (Optuna)

Gambar 3.58 menunjukkan proses pelatihan model CatBoost menggunakan parameter optimal hasil pencarian Optuna, yaitu `iterations=280`, `learning_rate=0.1118`, `depth=4`, `l2_leaf_reg=5.5314`, `subsample=0.8280`, dan `colsample_bylevel=0.9613`. Parameter `verbose=False` digunakan untuk menonaktifkan log output selama pelatihan. Model dilatih pada data training melalui ‘`fit(X_train, y_train)`’ dan menghasilkan prediksi pada data testing yang disimpan dalam variabel ‘`y_pred_catboost_optuna`’.

3.6.5 LightGBM

LightGBM merupakan model terakhir yang akan dilatih dan diuji dalam penelitian ini. Untuk menjaga konsistensi serta memastikan perbandingan yang setara (*apple-to-apple*), proses pelatihan dilakukan menggunakan tiga konfigurasi, yaitu model dengan parameter bawaan (*default parameters*) serta dua konfigurasi yang menggunakan hasil optimasi hyperparameter (*hyperparameter tuning*). Untuk konfigurasi pertama akan dilakukan pada platform Google Colaboratory dengan tahapan pertama yaitu mengimport model `LGBMRegressor` dari *library* `lightgbm`

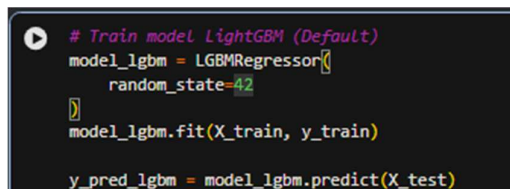
menggunakan *keyword* Python ‘import’ dan ‘from’. Prosesnya dilakukan pada platform Google Colaboratory yang ditunjukkan pada Gambar 3.59.



```
import lightgbm as lgb
from lightgbm import LGBMRegressor
```

Gambar 3.59 Proses Import Model LGBMRegressor

Setelah model berhasil di-*import*, model akan melalui proses pelatihan menggunakan parameter bawaan menggunakan data *training* yang akan ditunjukkan pada Gambar 3.60.



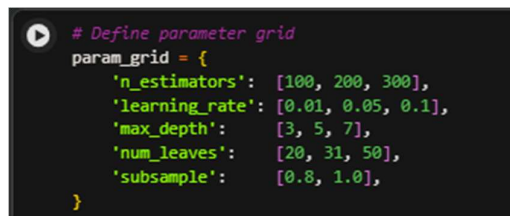
```
# Train model LightGBM (Default)
model_lgbm = LGBMRegressor(
    random_state=42
)
model_lgbm.fit(X_train, y_train)

y_pred_lgbm = model_lgbm.predict(X_test)
```

Gambar 3.60 Kode Pelatihan Model LightGBM (*Default Parameter*)

Gambar 3.60 menunjukkan proses pelatihan model LightGBM menggunakan parameter default dari library LightGBM, dengan `random_state=42` untuk memastikan hasil yang dapat direproduksi. Model dilatih menggunakan data training melalui ‘`fit(X_train, y_train)`’ dan menghasilkan prediksi pada data testing yang disimpan dalam variabel ‘`y_pred_lgbm`’.

Selanjutnya, pelatihan model akan dilakukan menggunakan konfigurasi kedua yaitu optimasi parameter menggunakan GridSearchCV. Tahapan pertama akan dilakukan pendefinisian ruang pencarian *hyperparameter* (*parameter grid*) yang akan dilakukan pada platform Google Colaboratory menggunakan bahasa pemrograman Python, ditunjukkan pada Gambar 3.61.



```
# Define parameter grid
param_grid = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7],
    'num_leaves': [20, 31, 50],
    'subsample': [0.8, 1.0],
}
```

Gambar 3.61 Pendefinisian *Hyperparameter Grid* Model LightGBM

Gambar 3.61 menunjukkan proses pendefinisian ruang pencarian *hyperparameter* (*parameter grid*) yang digunakan dalam GridSearchCV untuk

model LightGBM. Beberapa parameter yang diuji meliputi `n_estimators` dengan nilai 100, 200, dan 300; `learning_rate` dengan nilai 0,01, 0,05, dan 0,1; `max_depth` dengan nilai 3, 5, dan 7; `num_leaves` dengan nilai 20, 31, dan 50; serta `subsample` dengan nilai 0,8 dan 1,0. Seluruh kombinasi parameter tersebut akan dievaluasi oleh `GridSearchCV` untuk menentukan konfigurasi yang memberikan performa terbaik. Proses pencarian parameter optimal tersebut ditunjukkan pada Gambar 3.62.

```
# GridSearchCV
grid_search_lgbm = GridSearchCV(
    LGBMRegressor(random_state=42, verbose=-1),
    param_grid,
    cv=5,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)
grid_search_lgbm.fit(X_train, y_train)

print("Best Parameters:", grid_search_lgbm.best_params_)
print("Best CV R²:", round(grid_search_lgbm.best_score_, 4))

*** Fitting 5 folds for each of 162 candidates, totalling 810 fits
Best Parameters: {'learning_rate': 0.05, 'max_depth': 3, 'n_estimators': 100, 'num_leaves': 20, 'subsample': 0.8}
Best CV R²: 0.3784
```

Gambar 3.62 Kode *Hyperparameter* LightGBM (`GridSearchCV`)

Gambar 3.62 menunjukkan proses pencarian parameter terbaik LightGBM menggunakan `GridSearchCV` dengan validasi silang *5-fold* (`cv=5`) dan metrik evaluasi R^2 (`scoring='r2'`). Proses pencarian dilakukan terhadap 162 kombinasi parameter, menghasilkan total 810 proses *fitting*. Hasil pencarian menunjukkan bahwa parameter terbaik yang ditemukan adalah `learning_rate=0.05`, `max_depth=3`, `n_estimators=100`, `num_leaves=20`, dan `subsample=0.8`, dengan nilai Best CV R^2 sebesar 0.3784.

Model selanjutnya akan dilatih menggunakan parameter terbaik yang telah ditemukan menggunakan data *training* yang ditunjukkan pada Gambar 3.63.

```
# Train model dengan best params
model_lgbm_gs = LGBMRegressor(
    n_estimators=100,
    learning_rate=0.05,
    max_depth=3,
    num_leaves=20,
    subsample=0.8,
    random_state=42,
    verbose=-1
)
model_lgbm_gs.fit(X_train, y_train)

y_pred_lgbm_gs = model_lgbm_gs.predict(X_test)
```

Gambar 3.63 Kode Pelatihan Model LightGBM (GridSearchCV)

Pada Gambar 3.63, model LightGBM dilatih kembali menggunakan parameter optimal hasil GridSearchCV yang terdiri atas `n_estimators=100`, `learning_rate=0.05`, `max_depth=3`, `num_leaves=20`, dan `subsample=0.8`. Parameter `verbose=-1` digunakan untuk menyembunyikan output pelatihan yang tidak diperlukan. Setelah proses pelatihan menggunakan ‘`model_lgbm_gs.fit(X_train, y_train)`’ selesai, model menghasilkan prediksi terhadap data testing yang disimpan pada variabel ‘`y_pred_lgbm_gs`’.

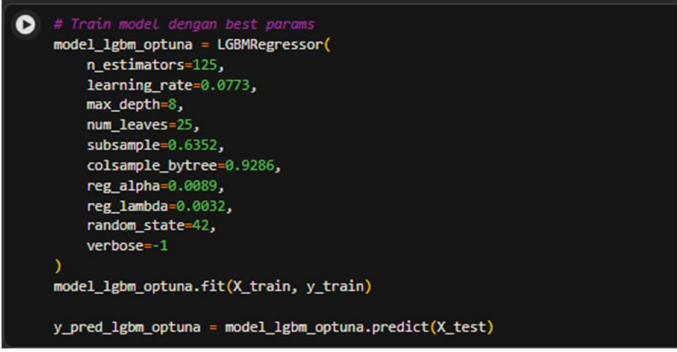
Selanjutnya, model memasuki tahap konfigurasi ketiga yaitu optimasi parameter menggunakan Optuna. Tahap pertama yang dilakukan adalah mendefinisikan fungsi objektif yang bertujuan untuk Optuna mencari kombinasi *hyperparameter* terbaik untuk model LightGBM berdasarkan nilai R^2 . Proses-nya akan dilakukan menggunakan platform Google Colaboratory dan menggunakan bahasa pemrograman Python, akan ditunjukkan pada Gambar 3.64.

```
# Define objective function
def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3, log=True),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'num_leaves': trial.suggest_int('num_leaves', 20, 100),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0),
        'reg_alpha': trial.suggest_float('reg_alpha', 1e-3, 10.0, log=True),
        'reg_lambda': trial.suggest_float('reg_lambda', 1e-3, 10.0, log=True),
    }
    model = LGBMRegressor(**params, random_state=42, verbose=-1)
    model.fit(X_train, y_train)
    return r2_score(y_test, model.predict(X_test))
```

Gambar 3.64 Kode *Objective Function* Optuna LightGBM

Gambar 3.64 menunjukkan pendefinisian fungsi objektif untuk proses optimasi hyperparameter LightGBM menggunakan Optuna. Fungsi objektif mendefinisikan ruang pencarian parameter yang meliputi `n_estimators` pada rentang [100, 500], `learning_rate` pada rentang [0.01, 0.3], `max_depth` pada rentang [3, 10], `num_leaves` pada rentang [20, 100], `subsample` pada rentang [0.6, 1.0], `colsample_bytree` pada rentang [0.5, 1.0], serta `reg_alpha` dan `reg_lambda` masing-masing pada rentang [1e-3, 10.0] dengan skala logaritmik. Setiap *trial* melatih model `LGBMRegressor` dengan kombinasi parameter yang disarankan Optuna

Gambarr 3.66 adalah proses optimasi yang dijalankan menggunakan Optuna dengan membuat objek *study* yang diarahkan untuk memaksimalkan nilai R^2 . Optuna kemudian menjalankan pencarian sebanyak 50 *trial* dan menemukan kombinasi hyperparameter terbaik pada *trial* ke-23 dengan nilai R^2 sebesar 0.4297. Parameter optimal yang dihasilkan adalah `n_estimators=323`, `learning_rate=0.0256`, `max_depth=4`, `num_leaves=40`, `subsample=0.8975`, `colsample_bytree=0.9422`, `reg_alpha=0.0283`, dan `reg_lambda=0.0216`. Parameter ini akan digunakan untuk melatih model LightGBM selanjutnya yang akan ditunjukkan pada Gambar 3.67.



```
# Train model dengan best params
model_lgbm_optuna = LGBMRegressor(
    n_estimators=125,
    learning_rate=0.0773,
    max_depth=8,
    num_leaves=25,
    subsample=0.6352,
    colsample_bytree=0.9286,
    reg_alpha=0.0089,
    reg_lambda=0.0032,
    random_state=42,
    verbose=-1
)
model_lgbm_optuna.fit(X_train, y_train)

y_pred_lgbm_optuna = model_lgbm_optuna.predict(X_test)
```

Gambar 3.67 Kode Pelatihan Model LightGBM (Optuna)

Gambar 3.67 menunjukkan proses pelatihan model LightGBM menggunakan parameter optimal hasil pencarian Optuna, yaitu `n_estimators=125`, `learning_rate=0.0773`, `max_depth=8`, `num_leaves=25`, `subsample=0.6352`, `colsample_bytree=0.9286`, `reg_alpha=0.0089`, dan `reg_lambda=0.0032`. Parameter `verbose=-1` digunakan untuk menonaktifkan log output selama proses pelatihan, dan `random_state=42` untuk memastikan hasil yang dapat direproduksi. Model dilatih pada data training melalui ‘`fit(X_train, y_train)`’ dan menghasilkan prediksi pada data testing yang disimpan dalam variabel ‘`y_pred_lgbm_optuna`’.

Setelah kelima model dilatih, model akan dievaluasi menggunakan metrik regresi yang telah ditentukan sebelumnya. Proses evaluasi tersebut akan dibahas pada subbab selanjutnya 3.7 *Evaluation* (Evaluasi)

3.6 *Evaluation* (Evaluasi)

Pada tahap ini model *machine learning* yang telah dilatih sebelumnya akan dievaluasi menggunakan metrik regresi yang telah ditentukan yaitu *mean absolute error* (MAE), *root mean squared error* (RMSE), dan koefisien determinan (R^2). Tahap evaluasi dilakukan untuk mengukur sejauh mana model-model yang telah dilatih mampu memprediksi posisi kemenangan tiap *driver* pada Formula 1 *Mexico City Grand Prix* dengan tepat. Proses evaluasi model dilakukan menggunakan platform Google Colaboratory menggunakan bahasa pemrograman Python, sementara hasil evaluasi disajikan dalam bentuk tabel yang disusun menggunakan Microsoft Word. Karena seluruh model melalui tahapan evaluasi yang sama, model

Random Forest dipilih sebagai representasi untuk menjelaskan alur dan proses evaluasi yang diterapkan dalam penelitian ini.

Setelah prediksi dihasilkan, hasil prediksi kemudian digabungkan ke dalam dataframe baru yang memuat kolom *Year*, *Abbreviation*, *RacePace (s)*, *Position*, dan *Points* dari data *testing*, ditunjukkan pada Gambar 3.68.

```
# Add predictions to test data
results = test_data[['Year', 'Abbreviation', 'RacePace (s)', 'Position', 'Points', 'LapTime (s)']].copy()
results['PredictedLapTime (s)'] = y_pred_rf
results['PredictedPosition'] = results['PredictedLapTime (s)'].rank()
```

Gambar 3.68 Kode Penambahan Hasil Prediksi ke Data Testing

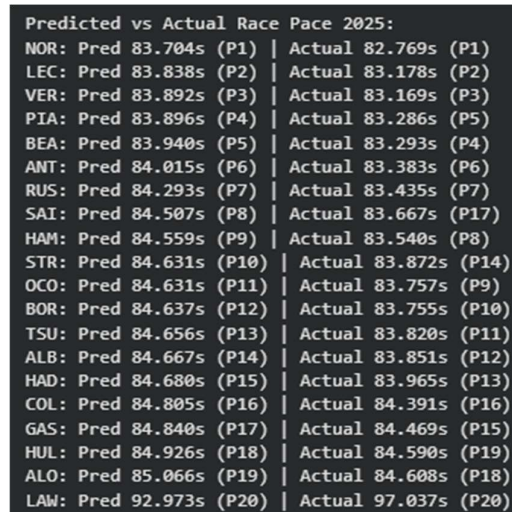
Kolom *PredictedLapTime (s)* ditambahkan untuk menyimpan hasil prediksi lap time oleh model, sedangkan kolom *PredictedPosition* diturunkan dari peringkat nilai *PredictedLapTime (s)* menggunakan fungsi `rank()`, sehingga pembalap dengan prediksi lap time tercepat mendapatkan posisi terdepan.

Setelah hasil prediksi ditambahkan ke dalam dataframe, selanjutnya dilakukan proses visualisasi perbandingan antara nilai prediksi dan nilai aktual race pace pembalap pada musim 2025, prosesnya ditunjukkan pada Gambar 3.69.

```
# Display predictions vs actual
print("\nPredicted vs Actual Lap Time 2025:")
results_sorted = results.sort_values('PredictedLapTime (s)')
for idx, row in results_sorted.iterrows():
    actual_pos = f"P{int(row['Position'])}" if not pd.isna(row['Position']) else "DNF"
    print(f"{row['Abbreviation']}: Pred {row['PredictedLapTime (s)']:.3f}s (P{int(row['PredictedPosition'])}) | "f"Actual {row['LapTime (s)']:.3f}s ({actual_pos})")
```

Gambar 3.69 Kode Perbandingan Hasil Prediksi dan Nilai Aktual 2025

Pada Gambar 3.69, menunjukkan proses menampilkan perbandingan antara hasil prediksi dan nilai aktual *Lap Time (s)* pembalap pada musim 2025. Data diurutkan terlebih dahulu berdasarkan nilai *PredictedLapTime (s)* dari yang tercepat hingga terlambat menggunakan `sort_values()`. Setiap baris kemudian ditampilkan dalam format yang memuat kode pembalap (*Abbreviation*), prediksi *Lap Time (s)* beserta prediksi posisi *finish*, serta nilai *Lap Time (s)* aktual beserta posisi *finish* sebenarnya. Untuk pembalap yang tidak menyelesaikan balapan, posisi aktual ditampilkan sebagai DNF menggunakan pengecekan kondisi `pd.isna()`. Kode pada Gambar 3.69 akan menghasilkan *output* seperti yang ditunjukkan pada Gambar 3.70.

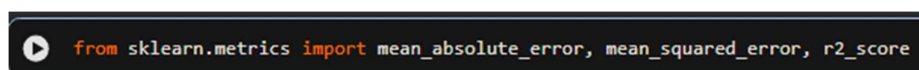


Predicted vs Actual Race Pace 2025:			
NOR:	Pred 83.704s (P1)		Actual 82.769s (P1)
LEC:	Pred 83.838s (P2)		Actual 83.178s (P2)
VER:	Pred 83.892s (P3)		Actual 83.169s (P3)
PIA:	Pred 83.896s (P4)		Actual 83.286s (P5)
BEA:	Pred 83.940s (P5)		Actual 83.293s (P4)
ANT:	Pred 84.015s (P6)		Actual 83.383s (P6)
RUS:	Pred 84.293s (P7)		Actual 83.435s (P7)
SAI:	Pred 84.507s (P8)		Actual 83.667s (P17)
HAM:	Pred 84.559s (P9)		Actual 83.540s (P8)
STR:	Pred 84.631s (P10)		Actual 83.872s (P14)
OCO:	Pred 84.631s (P11)		Actual 83.757s (P9)
BOR:	Pred 84.637s (P12)		Actual 83.755s (P10)
TSU:	Pred 84.656s (P13)		Actual 83.820s (P11)
ALB:	Pred 84.667s (P14)		Actual 83.851s (P12)
HAD:	Pred 84.680s (P15)		Actual 83.965s (P13)
COL:	Pred 84.805s (P16)		Actual 84.391s (P16)
GAS:	Pred 84.840s (P17)		Actual 84.469s (P15)
HUL:	Pred 84.926s (P18)		Actual 84.590s (P19)
ALO:	Pred 85.066s (P19)		Actual 84.608s (P18)
LAW:	Pred 92.973s (P20)		Actual 97.037s (P20)

Gambar 3.70 Output Kode Nilai Prediksi dan Nilai Aktual

Output pada Gambar 3.70 akan disajikan menggunakan tabel yang dibuat pada platform Microsoft Word sebanyak lima tabel sesuai dengan lima model *machine learning* yang diujikan.

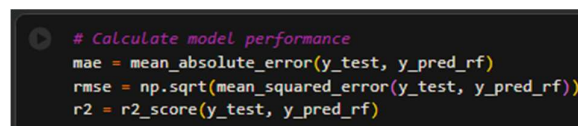
Tahap selanjutnya adalah melakukan evaluasi model menggunakan metrik regresi. Pada tahap ini, metrik regresi yang digunakan diimpor dari *library* sklearn.metrics dengan memanfaatkan perintah from dan import pada Python yang dilakukan pada platform Google Colaboratory, sebagaimana ditunjukkan pada Gambar 3.71.



```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Gambar 3.71 Kode Impor Metrik Evaluasi Regresi

Setelah proses impor yang dilakukan pada Gambar 3.71 dilakukan, akan dilakukan proses pengkodean untuk melakukan perhitungan metrik evaluasi yang akan ditunjukkan pada Gambar 3.72.



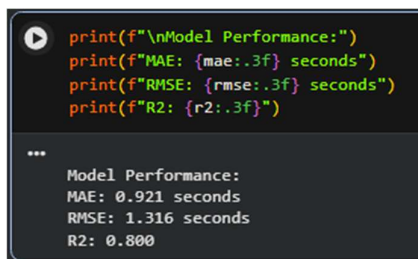
```
# Calculate model performance
mae = mean_absolute_error(y_test, y_pred_rf)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_rf))
r2 = r2_score(y_test, y_pred_rf)
```

Gambar 3.72 Kode Perhitungan Metrik Evaluasi Regresi

Gambar 3.72 merupakan proses perhitungan metrik evaluasi menggunakan tiga metrik utama, yaitu *Mean Absolute Error* (MAE), *Root Mean Square Error*

(RMSE), dan koefisien determinasi (R^2). MAE dihitung menggunakan fungsi `mean_absolute_error()`, RMSE diperoleh dari akar kuadrat hasil fungsi `mean_squared_error()`, dan R^2 dihitung menggunakan fungsi `r2_score()`. Ketiga metrik tersebut dihitung dengan membandingkan nilai prediksi `y_pred_rf` terhadap nilai aktual `y_test` pada data testing tahun 2025.

Hasil perhitungan metrik akan ditampilkan menggunakan perintah ‘print’ pada bahasa pemrograman Python yang akan ditunjukan pada Gambar 3.73.



```

print(f"\nModel Performance:")
print(f"MAE: {mae:.3f} seconds")
print(f"RMSE: {rmse:.3f} seconds")
print(f"R2: {r2:.3f}")

***
Model Performance:
MAE: 0.921 seconds
RMSE: 1.316 seconds
R2: 0.800
  
```

Gambar 3.73 Kode Tampilan untuk Hasil Perhitungan Metrik

Kode pada Gambar 3.73 menunjukkan proses pencetakan hasil evaluasi performa model. Nilai MAE, RMSE, dan R^2 yang telah dihitung sebelumnya ditampilkan menggunakan format tiga angka desimal. Hasil tersebut digunakan sebagai dasar perbandingan performa antar model yang diuji dalam penelitian ini dan akan disajikan menggunakan tabel yang dibuat pada platform Microsoft Word sebanyak lima tabel sesuai dengan lima model yang diujikan.

3.7 Tabel Rangkuman Hasil Evaluasi

Pada tahapan ini akan dilakukan rangkuman evaluasi model-model *machine learning* guna melengkapi tujuan penelitian ini yaitu membandingkan kinerja masing-masing model *machine learning* berdasarkan hasil evaluasi menggunakan metrik evaluasi yang telah ditentukan. Tabel tersebut akan disajikan menggunakan fitur *insert Table* pada Microsoft Word. Terdapat 2 kategori tabel yaitu tabel pertama berfokus pada perbandingan metrik evaluasi regresi tiap model *machine learning* yang telah dilatih dan tabel kedua berfokus pada perbandingan nilai prediksi yaitu *LapTime (s)* balapan dan posisi *finisih* setiap driver yang mengikuti ajang

balapan Formula 1 *Mexico City Grand Prix* dengan nilai aktual yang didapat dari hasil balapan itu sendiri. Setiap tabel akan diberi kesimpulan/kalimat penjelas.

3.7.1 Tabel Perbandingan Metrik Evaluasi Regresi

Sesuai metrik evaluasi regresi yang sudah ditentukan sebelumnya, yaitu *Mean Absolute Error* (MAE), *Root Mean Squared Error* (RMSE), dan Koefisien Determinan (R^2). Metrik-metrik evaluasi ini didapat dari proses evaluasi model yang telah dilakukan pada subbab 3.6 *Evaluation* (Evaluasi) sebelumnya. Untuk tabel pertama akan ditunjukkan pada Tabel 3.3.

Tabel 3.3 Tabel Rangkuman Metrik Evaluasi Regresi

Model	MAE	RMSE	R^2 (%)
RF Default	0.8723	1.1541	84,61%
RF GridSearch	0.9212	1.3163	79,98%
RF Bayesian OPT	1.1213	1.671	67,73%
GBR Default	0.693	0.7201	94,01%
GBR GridSearch	0.7727	0.8381	91,88%
GBR Optuna	0.4524	0.4805	97,33%
XGBoost Default	0.7333	2.0491	51,48%
XGBoost GridSearch	0.99	1.9125	57,74%
XGBoost Optuna	0.81	0.9692	89,15%
CatBoost Default	1.9539	2.0134	53,16%
CatBoost GridSearch	1.6767	1.7065	66,35%
CatBoost Optuna	1.5498	1.6201	69,67%
LightGBM Default	1.6821	2.2371	42,17%
LightGBM GridSearch	1.7591	2.3265	37,46%
LightGBM Optuna	1.668	2.2219	42,96%

Berdasarkan hasil evaluasi yang ditampilkan pada Tabel 3.3, terhadap 15 model machine learning yang diuji dalam penelitian ini, model Gradient Boosting dengan optimasi Optuna mencatatkan performa terbaik secara keseluruhan dengan nilai R^2 sebesar 0.9733, MAE sebesar 0.4524 detik, dan RMSE sebesar 0.4805 detik. Hasil ini mengindikasikan bahwa model tersebut mampu menjelaskan 97.33% variansi lap time pembalap di Mexico City Grand Prix 2025, dengan rata-rata selisih prediksi terhadap nilai aktual kurang dari setengah detik.

Secara umum, terdapat pola yang konsisten di seluruh kelompok model, yaitu optimasi menggunakan Optuna secara konsisten menghasilkan performa yang lebih baik dibandingkan GridSearchCV maupun konfigurasi default. Hal ini terlihat jelas pada kelompok Gradient Boosting, di mana konfigurasi Optuna ($R^2=0.9733$) unggul signifikan dibandingkan GridSearchCV ($R^2=0.9188$) dan default ($R^2=0.9401$). Pola serupa juga terjadi pada XGBoost, di mana Optuna menghasilkan $R^2=0.8915$ jauh melampaui GridSearchCV ($R^2=0.5774$) dan default ($R^2=0.5148$).

Di sisi lain, kelompok LightGBM secara keseluruhan mencatatkan performa paling rendah di antara semua model yang diuji, dengan nilai R^2 tertinggi hanya mencapai 0.4296 pada konfigurasi Optuna. Hal ini mengindikasikan bahwa arsitektur LightGBM kurang cocok untuk karakteristik dataset F1 Mexico City Grand Prix yang digunakan dalam penelitian ini, khususnya dengan jumlah data training yang terbatas. Sementara itu, kelompok Random Forest menunjukkan hasil yang menarik, di mana konfigurasi default justru menghasilkan performa lebih baik ($R^2=0.8461$) dibandingkan GridSearchCV ($R^2=0.7998$) dan Bayesian Optimization ($R^2=0.6773$), yang mengindikasikan adanya potensi overfitting pada proses tuning akibat keterbatasan jumlah data.

3.7.2 Tabel Perbandingan Nilai Prediksi dengan Nilai Aktual

Setelah dilakukan perbandingan metrik evaluasi pada subbab sebelumnya, akan dilakukan perbandingan nilai prediksi dengan nilai aktual waktu putaran sirkuit dalam satuan detik (*LapTime (s)*). Nilai prediksi waktu putaran sirkuit didapatkan dari hasil kode pada Gambar 3.69 pada subbab 3.6 *Evaluation* sebelumnya.

Tabel perbandingan menampilkan singkatan nama pembalap (*Abbreviation*), beserta posisi dan waktu hasil prediksi serta posisi dan waktu aktual. Sebelum tabel perbandingan disajikan, akan ditampilkan tabel yang berisi singkatan nama pembalap beserta nama lengkapnya untuk memudahkan pembaca dalam memahami isi tabel. Tabel tersebut akan ditampilkan pada Tabel 3.4.

Tabel 3.4 Tabel Daftar Singkatan Nama Pembalap

<i>Abbreviation</i>	<i>Full Name</i>
NOR	Lando Norris
BEA	Oliver Bearman
PIA	Oscar Piastrì
LEC	Charles Leclerc
VER	Max Verstappen
ANT	Kimi Antonelli
RUS	George Russel
SAI	Carlos Sainz
STR	Lance Stroll
HAM	Lewis Hamilton
OCO	Sebastian Ocon
BOR	Gabriel Bortoleto
ALB	Alex Albon
TSU	Yuki Tsunado
HAD	Isack Hadjar
COL	Franco Colapinto
GAS	Pierre Gasly
HUL	Nico Hulkenberg
ALO	Fernando Alonso
LAW	Liam Lawson

Tabel 3.4 menyajikan daftar singkatan nama pembalap beserta nama lengkapnya yang berpartisipasi dalam Formula 1 *Mexico City Grand Prix* 2025.

Sebanyak 20 pembalap yang berkompetisi pada musim Formula 1 2025 ditampilkan untuk memudahkan identifikasi pada tabel hasil.

Tabel selanjutnya akan menyajikan perbandingan antara nilai prediksi yang dihasilkan model dengan nilai aktual, untuk memudahkan pemahaman penyajian tabel akan dikategorikan berdasarkan masing-masing model.

a. Random Forest

Tabel perbandingan pertama menyajikan hasil prediksi yang dihasilkan oleh model Random Forest. Tabel akan ditunjukkan pada Tabel 3.5.

Tabel 3.5 Tabel Perbandingan Random Forest

Driver	Aktual	RF Default	RF GridSearchCV	RF Bayesian
NOR	82.769 (P1)	83.704 (P1)	83.709 (P1)	83.995 (P4)
LEC	83.178 (P2)	83.838 (P2)	83.839 (P2)	84.081 (P5)
VER	83.169 (P3)	83.892 (P3)	83.873 (P5)	84.117 (P6)
PIA	83.286 (P5)	83.896 (P4)	83.863 (P3)	83.889 (P2)
BEA	83.293 (P4)	83.940 (P5)	83.865 (P4)	83.881 (P1)
ANT	83.383 (P6)	84.015 (P6)	84.065 (P6)	83.962 (P3)
RUS	83.435 (P7)	84.293 (P7)	84.336 (P7)	84.517 (P7)
SAI	83.667 (P17)	84.507 (P8)	84.569 (P8)	84.755 (P14)
HAM	83.540 (P8)	84.559 (P9)	84.616 (P9)	84.812 (P16)
STR	83.872 (P14)	84.631 (P10)	84.623 (P10)	84.523 (P8)
OCO	83.757 (P9)	84.631 (P11)	84.644 (P12)	84.548 (P9)
BOR	83.755 (P10)	84.637 (P12)	84.641 (P11)	84.578 (P10)
TSU	83.820 (P11)	84.656 (P13)	84.665 (P14)	84.552 (P10)
ALB	83.851 (P12)	84.667 (P14)	84.663 (P13)	84.617 (P12)
HAD	83.965 (P13)	84.680 (P15)	84.732 (P15)	84.889 (P17)
COL	84.391 (P16)	84.805 (P16)	84.761 (P16)	84.739 (P13)
GAS	84.469 (P15)	84.840 (P17)	84.827 (P17)	84.795 (P15)
HUL	84.590 (P19)	84.926 (P18)	84.923 (P18)	85.266 (P19)
ALO	84.608 (P18)	85.066 (P19)	85.085 (P19)	85.379 (P18)

Driver	Aktual	RF Default	RF GridSearchCV	RF Bayesian
LAW	97.037 (P20)	92.973 (P20)	92.115 (P20)	90.690 (P20)

Berdasarkan Tabel 3.5, ketiga model (Default, GridSearchCV, dan Bayesian) memiliki kecenderungan untuk memprediksi waktu putaran yang lebih lambat (lebih besar) dibandingkan waktu aktual untuk hampir seluruh pembalap, kecuali pada kasus Lawson (LAW). Untuk 19 pembalap lainnya, seluruh prediksi berada di atas nilai aktual, yang mengindikasikan adanya positive bias (overestimasi) pada model terhadap performa di lintasan tersebut.

Dari segi akurasi posisi *finish* (urutan pembalap), model Random Forest *Default parameter* dan GridSearchCV menunjukkan performa yang hampir identik, baik dari nilai numerik maupun posisi akhir. Kedua model ini berhasil mempertahankan tiga besar yang sama dengan aktual (NOR, LEC, VER) serta memetakan posisi 6 besar dengan sangat baik, hanya mengalami sedikit pergeseran pada urutan P4 dan P5 (PIA dan BEA). Sementara itu, model Random Forest Bayesian justru menghasilkan pemetaan peringkat yang lebih kacau, terutama pada posisi puncak—di mana ia menempatkan BEA dan PIA di atas NOR (P1 dan P2), serta menggeser NOR ke P4. Hal ini menunjukkan bahwa pendekatan Bayesian justru mengorbankan stabilitas peringkat demi fleksibilitas hyperparameter, namun tidak serta-merta meningkatkan akurasi prediktif.

Persamaan nilai prediksi antara *Default Parameter* dan GridSearchCV memiliki selisih hanya pada desimal ketiga, misal 83.704 vs 83.709 untuk NOR, mengindikasikan bahwa *hyperparameter* bawaan Random Forest sudah cukup optimal untuk menangani data kualifikasi ini, sehingga kompleksitas tambahan dari GridSearchCV tidak memberikan keuntungan signifikan.

Terdapat anomali yang sangat mencolok pada pembalap Lawson (LAW). Waktu aktual LAW tercatat sangat lambat (97.037 detik), namun ketiga model justru memprediksi waktu yang jauh lebih cepat (berkisar 90.69 – 92.97 detik). Ini merupakan satu-satunya kasus di mana model *underestimate* (meremehkan) waktu, yang sangat kontras dengan 19 pembalap lainnya. Hal ini mengindikasikan bahwa

performa buruk LAW pada sesi aktual kemungkinan besar disebabkan oleh faktor eksternal di luar pola data latihan.

b. Gradient Boosting

Tabel perbandingan pertama menyajikan hasil prediksi yang dihasilkan oleh model Random Forest. Tabel akan ditunjukkan pada Tabel 3.6.

Tabel 3.6 Tabel Perbandingan Gradient Boosting

Driver	Aktual	GB Default	GB GridSearchCV	GB Optuna
NOR	82.769 (P1)	83.582 (P1)	83.826 (P1)	83.596 (P1)
LEC	83.178 (P2)	83.738 (P2)	83.980 (P6)	83.787 (P4)
VER	83.169 (P3)	83.809 (P4)	83.980 (P5)	83.801 (P5)
BEA	83.293 (P4)	83.801 (P3)	83.841 (P2)	83.753 (P2)
PIA	83.286 (P5)	83.866 (P5)	83.898 (P3)	83.764 (P3)
ANT	83.383 (P6)	83.956 (P6)	83.913 (P4)	83.865 (P6)
RUS	83.435 (P7)	84.369 (P8)	84.424 (P7)	84.020 (P7)
HAM	83.540 (P8)	84.634 (P11)	84.612 (P12)	84.204 (P10)
OCO	83.757 (P9)	84.713 (P14)	84.638 (P13)	84.261 (P11)
BOR	83.755 (P10)	84.668 (P12)	84.568 (P10)	84.281 (P12)
TSU	83.820 (P11)	84.734 (P15)	84.639 (P14)	84.304 (P14)
ALB	83.851 (P12)	84.630 (P10)	84.605 (P11)	84.298 (P13)
HAD	83.965 (P13)	84.705 (P13)	84.646 (P15)	84.343 (P15)
STR	83.872 (P14)	84.286 (P7)	84.540 (P9)	84.092 (P9)
GAS	84.469 (P15)	84.897 (P17)	84.881 (P17)	84.683 (P17)
COL	84.391 (P16)	84.794 (P16)	84.842 (P16)	84.595 (P16)
SAI	83.667 (P17)	84.399 (P9)	84.486 (P8)	84.091 (P8)
ALO	84.608 (P18)	85.256 (P19)	84.990 (P18)	84.871 (P19)
HUL	84.590 (P19)	85.093 (P18)	85.060 (P19)	84.861 (P18)
LAW	97.037 (P20)	97.766 (P20)	98.921 (P20)	96.660 (P20)

Berdasarkan Tabel 3.6, serupa dengan model Random Forest sebelumnya, ketiga varian Gradient Boosting secara konsisten menunjukkan kecenderungan

overestimasi waktu (memprediksi waktu lebih lambat) dibandingkan waktu aktual untuk hampir seluruh pembalap di posisi 1–19. Namun, berbeda dengan Random Forest yang masih mampu mempertahankan urutan tiga besar (NOR-LEC-VER), ketiga model Gradient Boosting justru gagal memetakan posisi puncak secara akurat. Pada aktual, urutan P1-P3 adalah NOR, LEC, dan VER. Model Default menempatkan VER di P4 (melewati BEA), sedangkan model GridSearchCV dan Optuna mengacak urutan lebih parah dengan menempatkan BEA dan PIA di atas LEC dan VER. Hal ini mengindikasikan bahwa Gradient Boosting memiliki sensitivitas yang lebih tinggi terhadap pola data sehingga pergeseran kecil dalam prediksi numerik berdampak besar pada peringkat akhir.

Dari sisi performa antar varian hyperparameter tuning, terlihat bahwa GridSearchCV justru menghasilkan prediksi yang paling buruk di antara ketiganya. Nilai yang dihasilkan GridSearchCV hampir selalu lebih besar (lebih lambat) dibandingkan model Default dan Optuna untuk sebagian besar pembalap, misalnya pada LEC (83.980 vs 83.738 dan 83.787) serta VER (83.980 vs 83.809 dan 83.801). Ini menunjukkan bahwa proses pencarian grid pada kasus ini tidak menemukan konfigurasi yang lebih baik, bahkan cenderung memperburuk akurasi. Sementara itu, model Default terbukti sangat kompetitif—bahkan untuk beberapa pembalap seperti NOR dan LEC, Default memberikan nilai prediksi yang paling mendekati aktual dibandingkan dua varian lainnya. Sedangkan Optuna berhasil menghasilkan prediksi yang lebih rendah (mendekati aktual) untuk sebagian besar pembalap di luar dua nama tersebut, serta memberikan keseimbangan yang lebih baik dibandingkan GridSearchCV, meskipun masih kalah teliti dari Default pada beberapa kasus spesifik.

Ketiga varian Gradient Boosting yang diuji, model Default layak dipertahankan sebagai baseline karena memberikan stabilitas dan akurasi numerik yang tidak kalah dengan varian yang di-*tune*, bahkan unggul dalam beberapa kasus. Optuna menawarkan perbaikan untuk mengurangi bias overestimasi pada banyak pembalap, namun belum cukup untuk memperbaiki kesalahan peringkat secara signifikan. Sementara itu, GridSearchCV terbukti tidak efektif pada kasus ini dan justru menurunkan performa. Secara keseluruhan, dibandingkan dengan model Random

Forest pada tabel sebelumnya, Gradient Boosting memiliki akurasi pemeringkatan yang lebih buruk, terutama di posisi tengah dan atas, sehingga diperlukan penanganan fitur tambahan atau pendekatan ensemble untuk meningkatkan kemampuannya dalam mempertahankan urutan kompetitif para pembalap.

c. XGBoost

Tabel perbandingan ketiga menyajikan hasil prediksi yang dihasilkan oleh model Random Forest. Tabel akan ditunjukkan pada Tabel 3.7.

Tabel 3.7 Tabel Perbandingan XGBoost

Driver	Aktual	XGB Default	XGB GridSearchCV	XGB Optuna
NOR	82.769 (P1)	83.315 (P1)	83.662 (P1)	83.923 (P1)
LEC	83.178 (P2)	83.596 (P3)	83.750 (P2)	84.070 (P2)
VER	83.169 (P3)	83.588 (P2)	83.889 (P3)	84.282 (P4)
BEA	83.293 (P4)	83.609 (P4)	84.016 (P4)	84.169 (P3)
PIA	83.286 (P5)	83.610 (P5)	84.016 (P5)	84.396 (P7)
ANT	83.383 (P6)	83.616 (P6)	84.140 (P6)	84.562 (P11)
RUS	83.435 (P7)	83.786 (P7)	84.180 (P8)	84.512 (P10)
HAM	83.540 (P8)	83.993 (P10)	84.338 (P10)	84.594 (P14)
OCO	83.757 (P9)	84.226 (P12)	84.430 (P13)	84.391 (P6)
BOR	83.755 (P10)	84.228 (P14)	84.472 (P14)	84.453 (P9)
TSU	83.820 (P11)	84.137 (P11)	84.493 (P15)	84.723 (P15)
ALB	83.851 (P12)	84.227 (P13)	84.407 (P12)	84.536 (P11)
HAD	83.965 (P13)	84.240 (P15)	84.556 (P16)	85.124 (P16)
STR	83.872 (P14)	83.829 (P8)	84.142 (P7)	84.418 (P8)
GAS	84.469 (P15)	84.664 (P17)	84.953 (P18)	85.356 (P17)
COL	84.391 (P16)	84.362 (P16)	84.395 (P11)	84.375 (P5)
SAI	83.667 (P17)	83.830 (P9)	84.329 (P9)	84.578 (P13)
ALO	84.608 (P18)	84.690 (P18)	85.455 (P19)	85.409 (P19)
HUL	84.590 (P19)	84.725 (P19)	84.913 (P17)	85.360 (P18)
LAW	97.037 (P20)	87.988 (P20)	88.972 (P20)	91.976 (P20)

Berdasarkan segi akurasi posisi *finish*, XGBoost dengan GridSearchCV menunjukkan performa yang presisi. Model ini berhasil mempertahankan urutan 6 besar (P1–P6) secara identik sempurna dengan aktual, yaitu NOR, LEC, VER, BEA, PIA, dan ANT. Ini merupakan pencapaian terbaik dibandingkan semua model pada tabel sebelumnya, di mana Random Forest dan Gradient Boosting selalu gagal mempertahankan urutan puncak secara utuh. Model Default juga cukup baik, hanya menukar posisi LEC dan VER (P2–P3), sedangkan model Optuna justru menjadi yang terburuk, bahkan menempatkan COL (yang aktualnya P16) ke peringkat P5, serta menggeser ANT hingga turun ke P11, yang mengindikasikan bahwa proses optimasi Optuna pada kasus ini sangat tidak stabil dan overfitting terhadap pola tertentu.

XGBoost secara umum menghasilkan prediksi yang jauh lebih mendekati aktual dibandingkan dua model sebelumnya. Sebagai contoh, pada NOR, selisih prediksi Default hanya sekitar 0,546 detik (82.769 vs 83.315), sementara pada Random Forest selisihnya mencapai ~0,9 detik. Namun, kecenderungan overestimasi (prediksi lebih lambat) masih terjadi pada hampir semua pembalap. *Tuning* GridSearchCV dan Optuna justru semakin menjauhkan nilai dari aktual (menjadi lebih besar) dibandingkan model Default, yang berarti proses *tuning* pada XGBoost kali ini mengorbankan akurasi numerik demi stabilitas peringkat (pada GridSearchCV) atau justru merusak keduanya (pada Optuna).

Antara ketiga varian XGBoost yang diuji, GridSearchCV adalah pilihan terbaik jika tujuan utama adalah mempertahankan urutan kompetitif para pembalap (khususnya untuk keperluan prediksi grid start), karena ia berhasil mereplikasi 6 besar dengan sempurna. Namun, jika tujuan utama adalah meminimalkan deviasi nilai absolut waktu putaran, model Default justru lebih unggul karena memberikan selisih terkecil terhadap aktual. Sedangkan optuna gagal pada dataset ini.

d. CatBoost

Tabel perbandingan keempat menyajikan hasil prediksi yang dihasilkan oleh model Random Forest. Tabel akan ditunjukan pada Tabel 3.8.

Tabel 3.8 Tabel Perbandingan CatBoost

Driver	Aktual	CatBoost De- fault	CatBoost GridSearchCV	CatBoost Op- tuna
NOR	82.769 (P1)	84.475 (P1)	84.358 (P1)	83.569 (P2)
LEC	83.178 (P2)	84.670 (P2)	84.579 (P2)	83.539 (P1)
VER	83.169 (P3)	84.843 (P3)	84.786 (P5)	84.097 (P4)
BEA	83.293 (P4)	84.991 (P5)	84.673 (P4)	84.265 (P7)
PIA	83.286 (P5)	84.978 (P4)	84.658 (P3)	84.033 (P3)
ANT	83.383 (P6)	85.277 (P6)	84.949 (P6)	84.240 (P6)
RUS	83.435 (P7)	85.413 (P7)	85.209 (P7)	84.283 (P8)
HAM	83.540 (P8)	85.831 (P13)	85.435 (P9)	84.801 (P12)
OCO	83.757 (P9)	85.439 (P8)	85.270 (P8)	84.099 (P5)
BOR	83.755 (P10)	85.645 (P10)	85.480 (P10)	84.400 (P9)
TSU	83.820 (P11)	85.805 (P11)	85.511 (P11)	84.693 (P11)
ALB	83.851 (P12)	85.635 (P9)	85.558 (P12)	84.464 (P10)
HAD	83.965 (P13)	86.084 (P15)	85.797 (P14)	85.094 (P14)

Driver	Aktual	CatBoost De- fault	CatBoost GridSearchCV	CatBoost Op- tuna
STR	83.872 (P14)	85.806 (P12)	85.763 (P13)	85.027 (P13)
GAS	84.469 (P15)	86.155 (P17)	85.849 (P15)	85.241 (P17)
COL	84.391 (P16)	86.023 (P14)	85.859 (P16)	85.194 (P16)
SAI	83.667 (P17)	86.105 (P16)	85.914 (P17)	85.097 (P15)
ALO	84.608 (P18)	86.577 (P19)	85.966 (P18)	85.691 (P19)
HUL	84.590 (P19)	86.297 (P18)	86.056 (P19)	85.368 (P18)
LAW	97.037 (P20)	93.208 (P20)	94.374 (P20)	95.431 (P20)

Berdasarkan Tabel 3.8, seluruh varian CatBoost adalah bias overestimasi (prediksi lebih lambat) yang sangat ekstrem, bahkan jauh lebih parah dibandingkan model Random Forest, Gradient Boosting, dan XGBoost pada tabel-tabel sebelumnya. Sebagai contoh, waktu aktual NOR adalah 82.769, namun CatBoost Default memprediksi 84.475 (selisih ~1,7 detik), sementara XGBoost Default hanya berselisih ~0,5 detik. Selisih yang sangat besar ini terjadi secara merata di hampir semua pembalap (kecuali Lawson), yang mengindikasikan bahwa arsitektur CatBoost dengan hyperparameter bawaan maupun hasil tuning pada kasus ini memiliki kapasitas yang buruk dalam menangkap skala absolut waktu putaran di lintasan Meksiko.

Dari sisi akurasi posisi *finisih*, tidak ada satu pun varian CatBoost yang mampu mempertahankan urutan 6 besar secara konsisten, berbeda dengan XGBoost GridSearchCV yang berhasil melakukannya dengan sempurna. Model

Default masih tergolong paling stabil di posisi puncak karena berhasil mempertahankan NOR-LEC-VER sebagai P1–P3, meskipun menukar posisi BEA dan PIA (P4–P5). Sementara itu, model GridSearchCV justru menurunkan VER dari P3 menjadi P5, dan model Optuna bahkan menukar posisi LEC menjadi P1 dan NOR menjadi P2—yang merupakan kesalahan fatal karena secara aktual NOR adalah yang tercepat. Hal ini menunjukkan bahwa proses tuning pada CatBoost justru mengorbankan stabilitas peringkat tanpa memberikan perbaikan numerik yang berarti.

Dari ketiga varian CatBoost yang sudah diuji, tidak ada satupun varian yang menunjukkan kelebihan dari model-model sebelumnya yang sudah diuji. Overestimasi numerik yang sangat besar membuat seluruh prediksi menjadi kurang bermakna secara kuantitatif. Tetapi CatBoost dengan Optuna memberikan nilai prediksi yang paling mendekati aktual untuk sebagian besar pembalap dan sedikit lebih baik daripada Default dan GridSearchCV, meskipun tidak cukup untuk memperbaiki kekacauan pemeringkatan.

e. LightGBM

Tabel perbandingan kelima menyajikan hasil prediksi yang dihasilkan oleh model Random Forest. Tabel akan ditunjukkan pada Tabel 3.9.

Tabel 3.9 Tabel Perbandingan LightGBM

Driver	Aktual	LightGBM Default	LightGBM GridSearchCV	LightGBM Optuna
NOR	82.769 (P1)	84.635 (P6)	84.761 (P5)	84.654 (P6)
LEC	83.178 (P2)	84.635 (P6)	84.761 (P5)	84.654 (P6)
VER	83.169 (P3)	84.635 (P6)	84.761 (P5)	84.654 (P6)
BEA	83.293 (P4)	84.635 (P6)	84.761 (P5)	84.654 (P6)

Driver	Aktual	LightGBM De- fault	LightGBM GridSearchCV	LightGBM Op- tuna
PIA	83.286 (P5)	84.635 (P6)	84.761 (P5)	84.654 (P6)
ANT	83.383 (P6)	84.635 (P6)	84.761 (P5)	84.654 (P6)
RUS	83.435 (P7)	84.635 (P6)	84.761 (P5)	84.654 (P6)
HAM	83.540 (P8)	85.383 (P13)	85.170 (P11)	85.357 (P13)
OCO	83.757 (P9)	84.547 (P2)	84.887 (P9)	84.514 (P2)
BOR	83.755 (P10)	85.497 (P14)	85.350 (P13)	85.408 (P14)
TSU	83.820 (P11)	84.709 (P10)	85.108 (P10)	84.751 (P10)
ALB	83.851 (P12)	85.659 (P15)	85.572 (P14)	85.644 (P15)
HAD	83.965 (P13)	85.822 (P17)	85.628 (P16)	85.830 (P17)
STR	83.872 (P14)	84.084 (P1)	84.484 (P1)	84.084 (P1)
GAS	84.469 (P15)	85.971 (P18)	85.873 (P18)	85.980 (P18)
COL	84.391 (P16)	85.316 (P12)	85.396 (P14)	85.304 (P12)
SAI	83.667 (P17)	85.074 (P11)	85.219 (P12)	85.127 (P11)
ALO	84.608 (P18)	86.395 (P19)	85.873 (P18)	86.338 (P19)

Driver	Aktual	LightGBM Default	LightGBM GridSearchCV	LightGBM Optuna
HUL	84.590 (P19)	85.687 (P16)	85.819 (P17)	85.758 (P16)
LAW	97.037 (P20)	89.188 (P20)	88.759 (P20)	89.215 (P20)

Berdasarkan Tabel 3.9, ketiga varian LightGBM menunjukkan kegagalan fundamental yang sangat fatal, yaitu memprediksi waktu yang identik (sama persis) untuk 7 pembalap teratas secara bersamaan. Pada model Default dan Optuna, tujuh pembalap pertama (NOR, LEC, VER, BEA, PIA, ANT, dan RUS) semuanya mendapat nilai prediksi yang persis sama—masing-masing 84.635 (Default) dan 84.654 (Optuna)—dan ditempatkan pada peringkat yang sama (P6). Sementara itu, model GridSearchCV juga melakukan hal serupa dengan memprediksi 84.761 untuk ketujuh pembalap tersebut dan menempatkan mereka di P5.

LightGBM menunjukkan bias overestimasi (prediksi lebih lambat) yang sangat parah terhadap waktu aktual untuk hampir seluruh pembalap, kecuali Lawson. Sebagai contoh, waktu aktual NOR adalah 82.769, namun ketiga varian LightGBM memprediksinya di kisaran 84.6–84.7 detik, yang berarti selisihnya mencapai hampir 2 detik—ini merupakan deviasi terburuk dibandingkan semua model yang telah diuji sebelumnya (Random Forest, XGBoost, dan CatBoost). Hal ini menegaskan bahwa LightGBM dengan konfigurasi saat ini sama sekali tidak cocok untuk menangkap skala absolut waktu putaran di lintasan Meksiko.

Terdapat anomali yang sangat mencengangkan pada pembalap Stroll (STR). Secara aktual, STR berada di posisi P14 dengan waktu 83.872. Namun, ketiga varian LightGBM secara konsisten menempatkan STR sebagai P1 (nilai prediksi berkisar 84.084–84.484), yang berarti model menganggap STR sebagai pembalap tercepat. Ini adalah kesalahan pemeringkatan yang paling ekstrem di seluruh rangkaian tabel yang telah dianalisis. Fenomena ini mengindikasikan bahwa LightGBM sangat sensitif terhadap pola tertentu dalam data latihan yang mungkin

berkaitan dengan gaya mengemudi atau karakteristik mobil STR, tetapi pola tersebut sama sekali tidak relevan dengan kondisi kualifikasi aktual di Meksiko.

Ketiga varian LightGBM yang sudah diuji, tidak ada satupun yang layak digunakan, bahkan sebagai baseline sekalipun. Baik GridSearchCV maupun Optuna gagal memberikan perbaikan apa pun dibandingkan model Default—nilai dan peringkat yang dihasilkan hampir identik, yang berarti proses tuning hyperparameter pada kasus ini sama sekali tidak efektif. Dengan adanya failure mode berupa prediksi identik untuk 7 pembalap sekaligus dan kesalahan penempatan Stroll sebagai P1, LightGBM terbukti menjadi model dengan performa terburuk di antara keempat model yang telah dibandingkan (Random Forest, Gradient Boosting, XGBoost, CatBoost, dan LightGBM).

3.7.3 Kesimpulan Tabel Evaluasi

Terdapat dua kesimpulan yang berbeda dari perbandingan metrik evaluasi regresi dan perbandingan nilai prediksi dengan nilai aktual. Pada perbandingan metrik evaluasi, model Gradient Boosting dengan optimasi Optuna mencatatkan performa terbaik secara keseluruhan dengan nilai R^2 sebesar 0.9733 (97%), MAE sebesar 0.4524 detik, dan RMSE sebesar 0.4805 detik. Sedangkan pada perbandingan nilai prediksi dengan nilai aktual, model XGBoost menjadi model paling terbaik terutama pada varian GridSearchCV dan *Default Parameter*. Dari sisi akurasi numerik (nilai absolut waktu), XGBoost Default adalah yang terbaik karena menghasilkan selisih prediksi terkecil terhadap waktu aktual, misalnya pada NOR, selisihnya hanya sekitar 0,546 detik, jauh lebih baik dibanding Random Forest (~0,9 detik), Gradient Boosting (~0,8 detik), CatBoost (~1,7 detik), dan LightGBM (~1,8 detik). Sementara itu, dari sisi akurasi pemeringkatan (urutan pembalap), XGBoost GridSearchCV mencapai prestasi sempurna dengan berhasil mempertahankan urutan 6 besar (P1–P6) secara identik dengan aktual (NOR, LEC, VER, BEA, PIA, ANT), pencapaian yang tidak mampu ditiru oleh model-model yang telah diuji.

Metrik evaluasi seperti Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), dan koefisien determinasi (R^2) digunakan untuk mengukur tingkat kedekatan antara nilai prediksi dan nilai aktual secara numerik pada seluruh data

pengujian, tanpa mempertimbangkan aspek peringkat atau urutan posisi. Berdasarkan hasil evaluasi, model Gradient Boosting Regression (GBR) dengan tuning Optuna memperoleh nilai MAE sebesar 0,4524 dan RMSE sebesar 0,4805. Hasil tersebut menunjukkan bahwa rata-rata kesalahan prediksi waktu putaran yang dihasilkan model hanya sekitar 0,45 detik terhadap nilai aktual. Selain itu, model ini menghasilkan nilai R^2 sebesar 97,33%, yang mengindikasikan bahwa sebagian besar variasi data aktual dapat dijelaskan oleh model. Tingginya performa tersebut salah satunya dipengaruhi oleh kemampuan model dalam menangani data dengan karakteristik ekstrem. Sebagai contoh, pada kasus pembalap Liam Lawson (LAW), model GBR Optuna menghasilkan prediksi waktu putaran sebesar 96,660 detik, yang sangat dekat dengan nilai aktual sebesar 97,037 detik dengan selisih sekitar 0,38 detik. Sebaliknya, model XGBoost dengan parameter default menghasilkan prediksi sebesar 87,988 detik, sehingga memiliki selisih hampir 9 detik terhadap nilai aktual. Karena RMSE memberikan penalti yang lebih besar terhadap kesalahan dengan nilai yang tinggi melalui proses pengkuadratan error, selisih yang besar pada satu observasi dapat meningkatkan nilai RMSE secara signifikan. Oleh karena itu, model GBR Optuna memperoleh hasil evaluasi yang lebih baik karena mampu mempertahankan tingkat akurasi yang konsisten pada seluruh data pengujian, termasuk pada kasus-kasus ekstrem.

Analisis per pembalap pada Tabel 3.7 lebih menekankan pada kemampuan model dalam mempertahankan urutan kompetitif (ranking), khususnya pada posisi teratas (P1–P6) yang memiliki peran penting dalam kompetisi Formula 1. Berdasarkan hasil perbandingan, model XGBoost dengan tuning GridSearchCV berhasil memprediksi enam posisi teratas secara identik dengan hasil aktual, yaitu NOR, LEC, VER, BEA, PIA, dan ANT. Hasil tersebut menunjukkan tingkat akurasi yang sangat baik dalam merepresentasikan urutan performa pembalap. Sebaliknya, model Gradient Boosting Regression (GBR) dengan tuning Optuna masih menghasilkan beberapa perbedaan, seperti pertukaran posisi antara NOR dan LEC serta prediksi ANT pada posisi ke-11, yang dimana hasil aktual menunjukkan pembalap tersebut finis di posisi ke-6.

Keunggulan XGBoost dalam mempertahankan urutan pembalap dapat dikaitkan dengan kemampuannya dalam memodelkan hubungan nonlinier dan interaksi antarfitur yang kompleks. Karakteristik tersebut memungkinkan model untuk menangkap perbedaan performa relatif antar pembalap secara lebih akurat, terutama pada kelompok pembalap papan atas, meskipun masih terdapat deviasi pada nilai waktu prediksi secara absolut. Dalam konteks Formula 1, ketepatan dalam memprediksi urutan posisi sering kali lebih relevan dibandingkan selisih waktu yang sangat kecil, karena posisi akhir pembalap merupakan indikator utama dalam menentukan hasil kompetisi. Oleh karena itu, dari perspektif kemampuan mempertahankan ranking, model XGBoost dengan GridSearchCV menunjukkan performa yang lebih baik dibandingkan model lainnya.

4. PENUTUP

4.1 Kesimpulan

Penelitian ini berhasil melakukan analisis komparatif lima algoritma *machine learning* untuk melakukan prediksi pemenang pada ajang balapan Formula 1 *Mexico City Grand Prix*. Algoritma XGBoost dengan *hyperparameter* GridSearchCV menjadi “pemenang” pada penelitian analisis komparatif ini, dikarenakan ketepatan dalam memprediksi urutan posisi pemenang pembalap/*driver*. Model Gradient Boosting dengan *hyperparameter* Optuna menjadi model paling “baik” dalam kasus perbandingan metrik evaluasi regresi terhadap model-model lain yang sudah diuji. Walaupun model tersebut memiliki metrik yang paling unggul, model XGBoost dengan *hyperparameter* Optuna tetap dipilih sebagai “pemenang”, dikarenakan dalam konteks Formula 1 ketepatan dalam memprediksi urutan posisi sering kali lebih relevan dibandingkan selisih waktu yang sangat kecil, karena posisi akhir pembalap merupakan indikator utama dalam menentukan hasil kompetisi. XGBoost dengan *hyperparameter* Optuna berhasil memprediksi posisi *finish* P1-P6 dengan benar, walaupun tidak semua posisi *finish* pembalap berhasil diprediksi dengan benar, model ini berhasil mempertahankan urutan (posisi *finish*) tiap pembalap khususnya pada posisi teratas. Capaian ini cukup relevan dalam konteks Formula 1, mengingat hanya pembalap yang menyelesaikan balapan pada posisi P1 hingga P10 yang berhak memperoleh poin kejuaraan.

4.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, terdapat beberapa saran yang dapat dijadikan pertimbangan bagi penelitian selanjutnya. Pertama, perluasan cakupan komparasi model dengan menyertakan lebih banyak algoritma, termasuk pendekatan deep learning seperti LSTM atau Neural Network, guna memperoleh gambaran perbandingan performa yang lebih komprehensif. Kedua, model dengan performa terbaik yang dihasilkan dari penelitian ini dapat dikembangkan lebih

lanjut menjadi sebuah aplikasi prediksi hasil balapan Formula 1 berbasis website, sehingga dapat dimanfaatkan secara praktis oleh pengguna umum.

DAFTAR PUSTAKA

- An-Nur. 2023. Meningkatkan Pengambilan Keputusan Ekonomi dengan Algoritma Machine Learning di Era IoT. Diakses dari <https://an-nur.ac.id/esy/meningkatkan-pengambilan-keputusan-ekonomi-dengan-algoritma-machine-learning-di-era-iot.html>. tanggal 25 Juni 2026.
- BINUS. 2024. Mengenal Peran dan Menelisik Penerapan Data Analytics dalam Formula 1. Diakses dari <https://sis.binus.ac.id/2024/08/13/mengenal-peran-dan-menelisik-penerapan-data-analytics-dalam-formula-1/>. tanggal 25 Juni 2026.
- Bischl, B., Binder, M., Lang, M., Pielok, T., Richter, J., Coors, S., Thomas, J., Ullmann, T., Becker, M., Boulesteix, A.-L., Deng, D., & Lindauer, M. (2023). *Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges*. *WIREs Data Mining and Knowledge Discovery*, 13(2), e1484.
- Chicco, D., Warrens, M. J., & Jurman, G. (2021). *The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation*. *PeerJ Computer Science*, 7, e623.
- F1 Technical. 2025. *Cooling requirement: How did teams optimize their cars for the Mexico City Grand Prix?*. Diakses dari <https://www.f1technical.net/news/27928> tanggal 18 April 2026\
- Hartawan, M. S., Erkamim, Moh., Rachmawati, S., Santi, N. C., Legito, L., & Sepriano, S. (2023). Penerapan Algoritma Supervised Learning untuk Klasifikasi Program Keluarga Harapan. *MALCOM: Indonesian Journal of Machine Learning and Computer Science*, 3(2), 83–91. <https://doi.org/10.57152/malcom.v3i2.873>.
- Hodson, T. O. (2022). Root-mean-square error (RMSE) or mean absolute error (MAE): when to use them or not. *Geoscientific Model Development*, 15, 5481–5487.
- Mohammed, S., Budach, L., Feuerpfeil, M., Ihde, N., Nathansen, A., Noack, N., Patzlaff, H., Naumann, F., & Harmouch, H. (2025). *The effects of data quality*

- on machine learning performance on tabular data*. Information Systems, 132, 102549.
- Nurrahman, F., Wijayanto, H., Wigena, A. H., & Nurjanah, N. 2023. *Pre-Processing Data on Multiclass Classification of Anemia and Iron Deficiency with the XGBoost Method*. BAREKENG: Jurnal Ilmu Matematika dan Terapan. Vol. 17 No. 2. Hal. 1063–1072.
- Raihani, K., & Dwitama, F. (2025). Implementasi Visualisasi Data Interaktif pada Produk Skincare dan Ulasan Konsumen Menggunakan Metode CRISP-DM. Jurnal Minfo Polgan, 14(2), 2805–2815. <https://doi.org/10.33395/jmp.v14i2.15531>
- Wijoyo, A., Saputra, A. Y., Ristanti, S., Sya'Ban, S. R., Amalia, M., & Febriansyah, R. 2024. Pembelajaran Machine Learning. OKTAL: Jurnal Ilmu Komputer dan Science. Vol. 3 No. 2. Hal. 375-380. Diakses dari <https://journal.media-publikasi.id/index.php/oktal/article/download/2305/2526>. tanggal 25 Juni 2026.
- Wibowo, M. A. A., & Setiadi, D. R. I. M. 2024. *Optimized Machine Learning Model for Credit Card Fraud Detection Using SMOTE-Tomek and Feature Engineering*. JAIC (Journal of Applied Informatics and Computing). Vol. 8 No. 2. Hal. 580–588. Diakses dari <https://jurnal.polibatam.ac.id/index.php/JAIC>. tanggal 25 Juni 2026.
- Singh, V., Pencina, M., & Einstein, A. J. 2021. *Impact of train/test sample regimen on performance estimate stability of machine learning in cardiovascular imaging*. Scientific Reports. Vol. 11 No. 1. Hal. 14490.

Lampiran 1. Lembar Pernyataan Ujicoba

LEMBAR PERNYATAAN UJICoba APLIKASI

Saya yang bertandatangan di bawah ini:

Nama : Kevin Budiawan Sidarta

NPM : 10123585

Judul PI : Analisis Komparasi Model Machine Learning untuk
Prediksi Pemenang Formula 1 : Studi Kasus 2025 Mexico
City Grand Prix

Menyatakan bahwa penelitian dalam penulisan ilmiah ini telah selesai dilaksanakan dan seluruh tahapan pengolahan data, pemodelan, serta pengujian sistem telah berjalan dengan baik sesuai dengan tujuan penelitian yang ditetapkan.

Demikian pernyataan ujicoba ini dibuat dengan sebenar-benarnya dan dengan penuh kesadaran.

Jakarta, tanggal – Juni – 2026 (ujicoba sebelum sidang)

Mahasiswa,

Pembimbing

TTD

TTD

(Kevin Budiawan Sidarta)

(Widya Khafa Nofa, S.Kom., MMSI)

Lampiran 2. Listing Program

LISTING PROGRAM

1. Pengambilan Data Menggunakan FASTF1 API

```
import pandas as pd
pd.set_option('display.max_columns',None)
import fastf1
import os
import plotly.express as px
# UNTUK TAHUN 2022
session = fastf1.get_session(2022, 'Mexico City', 'R')
session.load()
quali = fastf1.get_session(2022, 'Mexico City', 'Q')
quali.load()
# Get results
results = session.results
laps_driver = session.laps
qualifying = quali.results
# UNTUK TAHUN 2023
session = fastf1.get_session(2023, 'Mexico City', 'R')
session.load()
quali = fastf1.get_session(2023, 'Mexico City', 'Q')
quali.load()
# Get results
results = session.results
laps_driver = session.laps
qualifying = quali.results
# UNTUK TAHUN 2024
session = fastf1.get_session(2024, 'Mexico City', 'R')
session.load()
quali = fastf1.get_session(2024, 'Mexico City', 'Q')
```

```

quali.load()
# Get results
results = session.results
laps_driver = session.laps
qualifying = quali.results
# UNTUK TAHUN 2025
session = fastf1.get_session(2025, 'Mexico City', 'R')
session.load()
quali = fastf1.get_session(2025, 'Mexico City', 'Q')
quali.load()
# Get results
results = session.results
laps_driver = session.laps
qualifying = quali.results

```

2. **Data Cleaning**

```

print("Dataset Info:")
print(f"Shape: {df.shape}")
print(f"Years: {df['Year'].unique()}")
print(f"Drivers: {df['Abbreviation'].unique()}")
print("\nMissing values:")
print(df.isnull().sum())
df_clean = df.dropna(subset=['LapTime (s)', 'Position', 'Points']).copy()

```

3. **Feature Engineering: Karakteristik Sirkuit, Cuaca & Faktor Tambahan**

```

circuit_data = {
    # Format: 'CircuitName': {fitur-fitur}
    # Mexico City Grand Prix - Autodromo Hermanos Rodriguez
    'Mexico City': {
        'circuit_length_km': 4.304,
        'num_corners': 17,
        'num_straights': 3,

```

```

'drs_zones': 2,
'altitude_m': 2285,      # Ketinggian sangat tinggi! Efek besar pada
downforce & mesin
'circuit_type': 'street_permanent', # permanent=0, street=1, hy-
brid=0.5
'traction_demand': 8,    # 1-10, Mexico tinggi karena banyak
tikungan lambat
'braking_demand': 7,     # 1-10
'downforce_level': 8,    # 1-10, butuh high downforce karena tipis
udara
'overtaking_difficulty': 6, # 1-10
'tire_degradation': 6,   # 1-10
'safety_car_probability': 0.45,
'lap_record_s': 76.050,  # Rekaman lap terbaik (s)
'avg_speed_kmh': 203.0,
}

```

4. Weather Data

```

weather_data_sample = {
    # Mexico City - kondisi cuaca historis (November)
    2022: {'CircuitName': 'Mexico City', 'air_temp_c': 22.0, 'track_temp_c':
38.0,
        'humidity_pct': 45, 'wind_speed_ms': 3.2, 'rainfall_mm': 0.0,
        'is_wet_race': 0, 'pressure_hpa': 775.0}, # Tekanan rendah karena
altitude tinggi
    2023: {'CircuitName': 'Mexico City', 'air_temp_c': 19.5, 'track_temp_c':
32.0,
        'humidity_pct': 55, 'wind_speed_ms': 2.8, 'rainfall_mm': 0.0,
        'is_wet_race': 0, 'pressure_hpa': 778.0},
    2024: {'CircuitName': 'Mexico City', 'air_temp_c': 21.0, 'track_temp_c':
35.0,
        'humidity_pct': 50, 'wind_speed_ms': 3.0, 'rainfall_mm': 0.0,

```

```

        'is_wet_race': 0, 'pressure_hpa': 776.0},
2025: {'CircuitName': 'Mexico City', 'air_temp_c': 20.5, 'track_temp_c':
34.0,
        'humidity_pct': 48, 'wind_speed_ms': 3.5, 'rainfall_mm': 0.0,
        'is_wet_race': 0, 'pressure_hpa': 777.0},
}

```

```

weather_df = pd.DataFrame(weather_data_sample).T.reset_index()
weather_df.rename(columns={'index': 'Year'}, inplace=True)
weather_df['Year'] = weather_df['Year'].astype(int)
print("Weather data:")
print(weather_df.to_string(index=False))
# Convert menjadi numerik
cols_to_convert = weather_df.columns.drop(['CircuitName'])

weather_df[cols_to_convert] = weather_df[cols_to_convert].astype('float64')

```

5. Driver & Team Performance Metrics

1. Win rate dan podium rate per driver per season

```

driver_season_stats = df_clean.groupby(['Year', 'Abbreviation']).agg(
    races_count=('Position', 'count'),
    wins=('Position', lambda x: (x == 1).sum()),
    podiums=('Position', lambda x: (x <= 3).sum()),
    top5=('Position', lambda x: (x <= 5).sum()),
    avg_position=('Position', 'mean'),
    total_points=('Points', 'sum'),
    avg_grid_pos=('GridPosition', 'mean'),
    position_gain_avg=('GridPosition', lambda x: x.mean()), # akan dikurangi avg_position

```

```

dnf_count=('Position', lambda x: (x > 20).sum()),      # proxy untuk
DNF
).reset_index()

driver_season_stats['win_rate'] = driver_season_stats['wins'] / driver_season_stats['races_count']
driver_season_stats['podium_rate'] = driver_season_stats['podiums'] / driver_season_stats['races_count']
driver_season_stats['top5_rate'] = driver_season_stats['top5'] / driver_season_stats['races_count']
driver_season_stats['avg_position_gain'] = driver_season_stats['avg_grid_pos'] - driver_season_stats['avg_position']
driver_season_stats['points_per_race'] = driver_season_stats['total_points'] / driver_season_stats['races_count']

print("Driver Season Stats (sample):")
print(driver_season_stats.head(10).to_string(index=False))

```

6. Merge Semua Fitur Baru ke Dataset

```

# MERGE CIRCUIT + WEATHER + DRIVER STATS KE df_clean

df_enhanced = df_clean.copy()

# --- Merge Circuit Data ---
# Cek kolom nama sirkuit yang tersedia
circuit_col = None
for possible in ['CircuitName', 'Circuit', 'GrandPrix', 'Location', 'Event-Name']:
    if possible in df_enhanced.columns:
        circuit_col = possible
        break

```

```

if circuit_col:
    df_enhanced = df_enhanced.merge(
        circuit_df.drop(columns=['CircuitName'], errors='ignore').re-
name(columns={'CircuitName': circuit_col}),
        on=circuit_col, how='left'
    )
    print(f"Circuit data merged via kolom '{circuit_col}'")
else:
    # Jika tidak ada kolom sirkuit, tambahkan fitur Mexico City langsung
    # (asumsi dataset ini khusus Mexico City)
    print("Kolom nama sirkuit tidak ditemukan.")
    print(" Menambahkan fitur karakteristik Mexico City sebagai kon-
stanta...")

    mexico_features = circuit_data['Mexico City']
    for key, val in mexico_features.items():
        df_enhanced[key] = val
    print(f" Ditambahkan {len(mexico_features)} fitur sirkuit Mexico
City")

# --- Merge Weather Data ---
df_enhanced = df_enhanced.merge(weather_df.drop(columns=['Circuit-
Name'], errors='ignore'),
                                on='Year', how='left')
print(f"Weather data merged")

# --- Merge Driver Season Stats ---
df_enhanced = df_enhanced.merge(driver_season_stats, on=['Year', 'Ab-
breviation'], how='left')
print(f"Driver stats merged")

```

```

# --- Merge Team Stats jika ada ---
if 'Team' in df_clean.columns and 'team_season_stats' in dir():
    df_enhanced = df_enhanced.merge(team_season_stats, on=['Year',
'Team'], how='left')
    print(f"Team stats merged")

print(f"\Shape setelah penambahan fitur: {df_enhanced.shape}")
print(f"Kolom baru yang ditambahkan:")
new_cols = [c for c in df_enhanced.columns if c not in df_clean.columns]
for col in new_cols:
    print(f" + {col}")

```

7. Altitude Effect on Engine & Downforce (Khusus Mexico City)

```

#
=====

=====

# FITUR KHUSUS ALTITUDE - SANGAT PENTING UNTUK
MEXICO CITY!
# Mexico City berada di 2285m dpl - udara lebih tipis:
# - Downforce berkurang ~20% vs sea level
# - Mesin kehilangan ~25% tenaga (naturally aspirated era)
# - Dengan turbo hybrid: dicompensate sebagian, tapi masih berpengaruh
# - Perlu wing lebih besar -> drag lebih besar -> top speed turun

# Atmospheric density ratio (rho/rho0) sebagai fungsi altitude
# Formula barometrik sederhana
def atmospheric_density_ratio(altitude_m):
    """Hitung rasio densitas udara vs sea level"""
    return np.exp(-altitude_m / 8500)

```

```

df_enhanced['air_density_ratio'] = atmospheric_density_ratio(
    df_enhanced.get('altitude_m', 2285)
)

# Estimasi efek altitude pada downforce dan pace
df_enhanced['downforce_penalty'] = 1 - df_enhanced['air_density_ratio']
# ~0.235 untuk Mexico
df_enhanced['altitude_pace_factor'] = df_enhanced['air_density_ratio'] #
# makin rendah = makin lambat (relatively)

# DRS effectiveness (lebih efektif di Mexico karena udara tipis - top speed
# gap lebih besar)
if 'drs_zones' in df_enhanced.columns:
    df_enhanced['drs_effectiveness'] = df_enhanced['drs_zones'] * (2 -
df_enhanced['air_density_ratio'])
else:
    df_enhanced['drs_effectiveness'] = 2 * (2 - df_enhanced.get('air_den-
sity_ratio', 0.765))

print("Altitude effect features:")
print(f" Air density ratio at 2285m: {atmospheric_density_ra-
tio(2285):.3f}")
print(f" Downforce penalty: {(1 - atmospheric_density_ra-
tio(2285))*100:.1f}%")
print(f" (vs sea level circuit seperti Monaco atau Baku)")

```

8. Quali-to-Race Conversion & Pace Delta Features

```

# QUALI-TO-RACE CONVERSION & PACE DELTA
# Hubungan antara kecepatan kualifikasi dan kecepatan race

```



```

# Pace delta: selisih race pace vs quali pace (indikator konsistensi)
if 'BestQuali (s)' in df_enhanced.columns and 'RacePace (s)' in df_enhanced.columns:
    df_enhanced['quali_to_race_delta'] = df_enhanced['RacePace (s)] -
df_enhanced['BestQuali (s)']
    df_enhanced['quali_to_race_ratio'] = df_enhanced['RacePace (s)] /
df_enhanced['BestQuali (s)']

# Normalized pace vs field average (per race/year)
df_enhanced['pace_vs_avg'] = df_enhanced.groupby('Year')['RacePace
(s)'].transform(
    lambda x: x - x.mean()
)
df_enhanced['quali_vs_avg'] = df_enhanced.groupby('Year')['BestQuali
(s)'].transform(
    lambda x: x - x.mean()
)
print("Quali-to-race pace delta features created")

# Grid position advantage (posisi start relative ke median)
if 'GridPosition' in df_enhanced.columns:
    df_enhanced['grid_advantage'] = df_enhanced.groupby('Year')['GridPo-
sition'].transform(
        lambda x: x.median() - x # positif = lebih baik dari median
    )
    print("Grid advantage feature created")

```

9. Finalisasi Fitur Kolom

```
# FEATURE COLUMNS FINAL - GABUNGAN SEMUA FITUR
```

```
# Core features (original)
```

```

base_features = [
    'Sector1Time (s)', 'Sector2Time (s)', 'Sector3Time (s)',
    'BestQuali (s)', 'GridPosition', 'Position', 'RacePace (s)'
]

# Circuit characteristic features
circuit_features = [col for col in [
    'circuit_length_km', 'num_corners', 'num straights', 'drs_zones',
    'altitude_m', 'traction_demand', 'braking_demand', 'downforce_level',
    'overtaking_difficulty', 'tire_degradation', 'safety_car_probability',
    'avg_speed_kmh'
] if col in df_enhanced.columns]

# Weather features
weather_features = [col for col in [
    'air_temp_c', 'track_temp_c', 'humidity_pct', 'wind_speed_ms',
    'rainfall_mm', 'is_wet_race', 'pressure_hpa'
] if col in df_enhanced.columns]

# Altitude/physics features
altitude_features = [col for col in [
    'air_density_ratio', 'downforce_penalty', 'altitude_pace_factor', 'drs_eff-
    ectiveness'
] if col in df_enhanced.columns]

# Driver performance features
driver_features = [col for col in [
    'win_rate', 'podium_rate', 'top5_rate', 'avg_position_gain',
    'points_per_race', 'avg_position', 'total_points'
] if col in df_enhanced.columns]

```

```

# Derived features
derived_features = [col for col in [
    'quali_to_race_delta', 'quali_to_race_ratio', 'pace_vs_avg',
    'quali_vs_avg', 'grid_advantage', 'compound_encoded',
    'tire_age_normalized', 'tire_deg_effect'
] if col in df_enhanced.columns]

# Gabungkan semua fitur
feature_columns_enhanced = (base_features + circuit_features +
weather_features +
                           altitude_features + driver_features + derived_features)

# Filter hanya kolom yang benar-benar ada
feature_columns_enhanced = [f for f in feature_columns_enhanced if f in
df_enhanced.columns]

print("=" * 60)
print("RINGKASAN FITUR YANG DIGUNAKAN:")
print(f" Base features      : {len(base_features)} fitur")
print(f" Circuit features   : {len(circuit_features)} fitur")
print(f" Weather features    : {len(weather_features)} fitur")
print(f" Altitude features   : {len(altitude_features)} fitur")
print(f" Driver features     : {len(driver_features)} fitur")
print(f" Derived features    : {len(derived_features)} fitur")
print(f" {'—'*30}")
print(f" TOTAL               : {len(feature_columns_enhanced)} fitur")
print("=" * 60)
print("\nFitur yang tidak tersedia (perlu ditambahkan ke CSV):")
all_intended = (base_features + circuit_features + weather_features +
                altitude_features + driver_features + derived_features)
missing_features = [f for f in all_intended if f not in df_enhanced.columns]

```

10. Train-Test Split

```
# TRAIN-TEST SPLIT DENGAN FITUR BARU

target_column = 'LapTime (s)'

# Drop missing values untuk fitur yang digunakan
df_model = df_enhanced.dropna(subset=[target_column] + feature_columns_enhanced).copy()

train_data = df_model[df_model['Year'].isin([2022, 2023, 2024])].copy()
test_data = df_model[df_model['Year'] == 2025].copy()

print(f"Training data: {len(train_data)} records")
print(f"Test data : {len(test_data)} records")
print(f"Features : {len(feature_columns_enhanced)}")

X_train = train_data[feature_columns_enhanced]
y_train = train_data[target_column]
X_test = test_data[feature_columns_enhanced]
y_test = test_data[target_column]
```

11. Data Scaling

```
from sklearn.preprocessing import StandardScaler

scaler_enhanced = StandardScaler()
X_train_scaled = scaler_enhanced.fit_transform(X_train)
X_test_scaled = scaler_enhanced.transform(X_test)

print(f"X_train shape: {X_train_scaled.shape}")
```

```
print(f'X_test shape : {X_test_scaled.shape}')
```

12. Feature Importance Analysis

```
# FEATURE IMPORTANCE - Cek kontribusi tiap fitur
```

```
from sklearn.ensemble import RandomForestRegressor
```

```
rf_importance = RandomForestRegressor(n_estimators=200, random_state=42)
```

```
rf_importance.fit(X_train, y_train)
```

```
importance_df = pd.DataFrame({  
    'feature': feature_columns_enhanced,  
    'importance': rf_importance.feature_importances_  
}).sort_values('importance', ascending=False)
```

```
print("Top 20 Feature Importance:")
```

```
print(importance_df.head(20).to_string(index=False))
```

```
# Visualisasi
```

```
plt.figure(figsize=(12, 8))
```

```
top_n = min(20, len(importance_df))
```

```
sns.barplot(data=importance_df.head(top_n), x='importance', y='feature',  
palette='viridis')
```

```
plt.title("Top Feature Importance - F1 Lap Time Prediction\n(Termasuk  
Circuit, Weather & Driver Features)")
```

```
plt.xlabel('Importance Score')
```

```
plt.tight_layout()
```

```
plt.savefig('feature_importance_enhanced.png', dpi=150,  
bbox_inches='tight')
```

```
plt.show()
print("\nPlot disimpan ke 'feature_importance_enhanced.png'")
```

13. 1st Model : Random Forest (Default Parameter)

```
# Train model
model_rf = RandomForestRegressor(random_state=42)

model_rf.fit(X_train, y_train)

# Predict on test data
y_pred_rf = model_rf.predict(X_test)

# Add predictions to test data
results = test_data[['Year', 'Abbreviation', 'RacePace (s)', 'Position',
'Points', 'LapTime (s)']].copy()
results['PredictedLapTime (s)'] = y_pred_rf
results['PredictedPosition'] = results['PredictedLapTime (s)'].rank()

# Calculate model performance
mae = mean_absolute_error(y_test, y_pred_rf)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_rf))
r2 = r2_score(y_test, y_pred_rf)

print(f"\nModel Performance:")
print(f"MAE: {mae:.3f} seconds")
print(f"RMSE: {rmse:.3f} seconds")
print(f"R2: {r2:.3f}")

# Display predictions vs actual
```

```

print("\nPredicted vs Actual Lap Time 2025 Random Forest (Default Pa-
rameter):")
results_sorted = results.sort_values('PredictedLapTime (s)')
for idx, row in results_sorted.iterrows():
    actual_pos = f"P {int(row['Position'])}" if not pd.isna(row['Position'])
    else "DNF"
    print(f"{row['Abbreviation']}: Pred {row['PredictedLapTime (s)']:.3f}s
(P {int(row['PredictedPosition'])}) | "f"Actual {row['LapTime (s)']:.3f}s
({actual_pos})")

```

14. 1st Model : Random Forest (Parameter Tuning Grid Search)

```

from sklearn.model_selection import GridSearchCV
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 15],
    'min_samples_split': [2, 5, 10]
}

grid_search_rf = GridSearchCV(model_rf, param_grid, cv=5, scoring='r2')
grid_search_rf.fit(X_train, y_train)
print(f"Best parameters: {grid_search_rf.best_params_}")

# Predict on test data
y_pred_rf = grid_search_rf.predict(X_test)

# Add predictions to test data
results = test_data[['Year', 'Abbreviation', 'RacePace (s)', 'Position',
'Points', 'LapTime (s)']].copy()
results['PredictedLapTime (s)'] = y_pred_rf
results['PredictedPosition'] = results['PredictedLapTime (s)'].rank()

```

```

# Calculate model performance
mae = mean_absolute_error(y_test, y_pred_rf)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_rf))
r2 = r2_score(y_test, y_pred_rf)

print(f"\nModel Performance:")
print(f"MAE: {mae:.3f} seconds")
print(f"RMSE: {rmse:.3f} seconds")
print(f"R2: {r2:.3f}")

# Display predictions vs actual
print("\nPredicted vs Actual Race Pace 2025 Random Forest
(GridSearchCV):")
results_sorted = results.sort_values('PredictedLapTime (s)')
for idx, row in results_sorted.iterrows():
    actual_pos = f"P {int(row['Position'])}" if not pd.isna(row['Position'])
    else "DNF"
    print(f"{row['Abbreviation']}: Pred {row['PredictedLapTime (s)']:.3f}s
(P {int(row['PredictedPosition'])}) | "f"Actual {row['LapTime (s)']:.3f}s
({actual_pos})")

```

15. 1st Model : Random Forest (Parameter Tuning Bayesian Optimization)

```

pip install scikit-optimize

import skopt
from skopt import BayesSearchCV

opt = BayesSearchCV(
    RandomForestRegressor(),

```



```

{
    'n_estimators': (100, 1000),
    'max_depth': (3, 20),
    'min_samples_split': (2, 10),
    'min_samples_leaf': (1, 5),
    'max_features': ['sqrt', 'log2', None]
},
n_iter=32,
cv=3,
n_jobs=-1
)
opt.fit(X_train, y_train)

# Predict on test data
y_pred_rf = opt.predict(X_test)

# Add predictions to test data
results = test_data[['Year', 'Abbreviation', 'RacePace (s)', 'Position',
'Points', 'LapTime (s)']].copy()
results['PredictedLapTime (s)'] = y_pred_rf
results['PredictedPosition'] = results['PredictedLapTime (s)'].rank()

# Calculate model performance
mae = mean_absolute_error(y_test, y_pred_rf)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_rf))
r2 = r2_score(y_test, y_pred_rf)

print(f"\nModel Performance:")
print(f"MAE: {mae:.3f} seconds")
print(f"RMSE: {rmse:.3f} seconds")
print(f"R2: {r2:.3f}")

```

```

# Display predictions vs actual
print("\nPredicted vs Actual Race Pace 2025 Random Forest (Bayesian
Optimization):")
results_sorted = results.sort_values('PredictedLapTime (s)')
for idx, row in results_sorted.iterrows():
    actual_pos = f"P {int(row['Position'])}" if not pd.isna(row['Position'])
    else "DNF"
    print(f"{row['Abbreviation']}: Pred {row['PredictedLapTime (s)']:.3f}s
(P {int(row['PredictedPosition'])}) | "f"Actual {row['LapTime (s)']:.3f}s
({actual_pos})")

```

16. 2nd Model : Gradient Boosting (Default Parameter)

```

from sklearn.ensemble import GradientBoostingRegressor

model_gbr = GradientBoostingRegressor(random_state=42)
model_gbr.fit(X_train, y_train)

y_pred_gbr = model_gbr.predict(X_test)

# Add predictions to test data
results = test_data[['Year', 'Abbreviation', 'RacePace (s)', 'Position',
'Points', 'LapTime (s)']].copy()
results['PredictedLapTime (s)'] = y_pred_gbr
results['PredictedPosition'] = results['PredictedLapTime (s)'].rank()

# Calculate model performance
mae = mean_absolute_error(y_test, y_pred_gbr)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_gbr))
r2 = r2_score(y_test, y_pred_gbr)

```

```

print(f"\nModel Performance:")
print(f"MAE: {mae:.3f} seconds")
print(f"RMSE: {rmse:.3f} seconds")
print(f"R2: {r2:.3f}")

# Display predictions vs actual
print("\nPredicted vs Actual Race Pace 2025 Gradient Boosting (Default
Parameter):")
results_sorted = results.sort_values('PredictedLapTime (s)')
for idx, row in results_sorted.iterrows():
    actual_pos = f"P {int(row['Position'])}" if not pd.isna(row['Position'])
    else "DNF"
    print(f"{row['Abbreviation']}: Pred {row['PredictedLapTime (s)']:.3f}s
(P {int(row['PredictedPosition'])}) | "f"Actual {row['LapTime (s)']:.3f}s
({actual_pos})")

```

17. 2nd Model : Gradient Boosting (GridSearchCV)

```

# Define parameter grid
param_grid = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7],
    'min_samples_split': [2, 5],
    'subsample': [0.8, 1.0],
}

# GridSearchCV
grid_search_gbr = GridSearchCV(
    GradientBoostingRegressor(random_state=42),

```

```

    param_grid,
    cv=5,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)
grid_search_gbr.fit(X_train, y_train)

print("Best Parameters:", grid_search_gbr.best_params_)
print("Best CV R²:", round(grid_search_gbr.best_score_, 4))

# Train model dengan best params
model_gbr_gs = GradientBoostingRegressor(
    n_estimators=300,
    learning_rate=0.1,
    max_depth=7,
    min_samples_split=5,
    subsample=1.0,
    random_state=42
)
model_gbr_gs.fit(X_train, y_train)

y_pred_gbr_gs = model_gbr_gs.predict(X_test)

# Add predictions to test data
results_gs = test_data[['Year', 'Abbreviation', 'RacePace (s)', 'Position',
'Points', 'LapTime (s)']].copy()
results_gs['PredictedLapTime (s)'] = y_pred_gbr_gs
results_gs['PredictedPosition'] = results_gs['PredictedLapTime (s)'].rank()

# Calculate model performance

```

```

mae = mean_absolute_error(y_test, y_pred_gbr_gs)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_gbr_gs))
r2 = r2_score(y_test, y_pred_gbr_gs)

print(f"\nModel Performance (GBR + GridSearchCV):")
print(f"MAE: {mae:.3f} seconds")
print(f"RMSE: {rmse:.3f} seconds")
print(f"R2: {r2:.3f} seconds")

# Display predictions vs actual
print("\nPredicted vs Actual Race Pace 2025 Gradient Boosting
(GridSearch):")
results_sorted_gs = results_gs.sort_values('PredictedLapTime (s)')
for idx, row in results_sorted_gs.iterrows():
    actual_pos = f"P {int(row['Position'])}" if not pd.isna(row['Position'])
    else "DNF"
    print(f"{row['Abbreviation']}: Pred {row['PredictedLapTime (s)']:.3f}s
(P {int(row['PredictedPosition'])}) | "f"Actual {row['LapTime (s)']:.3f}s
({actual_pos})")

```

18. Gradient Boosting (Optuna)

```

pip install optuna
import optuna

optuna.logging.set_verbosity(optuna.logging.WARNING)

# Define objective function
def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),

```

```

        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3,
log=True),
        'max_depth': trial.suggest_int('max_depth', 2, 8),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 10),
        'min_samples_leaf': trial.suggest_int('min_samples_leaf', 1, 5),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'max_features': trial.suggest_categorical('max_features', ['sqrt',
'log2', None]),
    }
    model = GradientBoostingRegressor(**params, random_state=42)
    model.fit(X_train, y_train)
    return r2_score(y_test, model.predict(X_test))

# Run Optuna study
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50, show_progress_bar=True)

print("Best Parameters:", study.best_params)
print("Best R²:", round(study.best_value, 4))

# Train model dengan best params
model_gbr_optuna = GradientBoostingRegressor(
    n_estimators=475,
    learning_rate=0.0390,
    max_depth=7,
    min_samples_split=6,
    min_samples_leaf=1,
    subsample=0.7397,
    max_features=None,
    random_state=42
)

```

```

model_gbr_optuna.fit(X_train, y_train)

y_pred_gbr_optuna = model_gbr_optuna.predict(X_test)

# Train model dengan best params
model_gbr_optuna = GradientBoostingRegressor(
    n_estimators=475,
    learning_rate=0.0390,
    max_depth=7,
    min_samples_split=6,
    min_samples_leaf=1,
    subsample=0.7397,
    max_features=None,
    random_state=42
)
model_gbr_optuna.fit(X_train, y_train)

y_pred_gbr_optuna = model_gbr_optuna.predict(X_test)

# Calculate model performance
mae = mean_absolute_error(y_test, y_pred_gbr_optuna)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_gbr_optuna))
r2 = r2_score(y_test, y_pred_gbr_optuna)

print(f"\nModel Performance (GBR + Optuna):")
print(f"MAE: {mae:.3f} seconds")
print(f"RMSE: {rmse:.3f} seconds")
print(f"R2: {r2:.3f}")

# Display predictions vs actual

```

```

print("\nPredicted vs Actual Race Pace 2025 Gradient Boosting (Optuna)
:")
results_sorted_opt = results_opt.sort_values('PredictedLapTime (s)')
for idx, row in results_sorted_opt.iterrows():
    actual_pos = f"P {int(row['Position'])}" if not pd.isna(row['Position'])
    else "DNF"
    print(f"{row['Abbreviation']}: Pred {row['PredictedLapTime (s)']:.3f}s
(P {int(row['PredictedPosition'])}) | "f"Actual {row['LapTime (s)']:.3f}s
({actual_pos})")

```

19. 3rd Model : XGBRegressor (Default Parameter)

```

from xgboost import XGBRegressor

model_xgb = XGBRegressor(
    random_state=42,
)
model_xgb.fit(X_train,y_train)

y_pred_xgb = model_xgb.predict(X_test)

# Add predictions to test data
results = test_data[['Year', 'Abbreviation', 'RacePace (s)', 'Position',
'Points', 'LapTime (s)']].copy()
results['PredictedLapTime (s)'] = y_pred_xgb
results['PredictedPosition'] = results['PredictedLapTime (s)'].rank()

# Calculate model performance
mae = mean_absolute_error(y_test, y_pred_xgb)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_xgb))
r2 = r2_score(y_test, y_pred_xgb)

```



```

print(f"\nModel Performance:")
print(f"MAE: {mae:.3f} seconds")
print(f"RMSE: {rmse:.3f} seconds")
print(f"R2: {r2:.3f}")

# Display predictions vs actual
print("\nPredicted vs Actual Race Pace 2025 XGBRegressor (Default Parameter):")
results_sorted = results.sort_values('PredictedLapTime (s)')
for idx, row in results_sorted.iterrows():
    actual_pos = f"P {int(row['Position'])}" if not pd.isna(row['Position'])
    else "DNF"
    print(f"{row['Abbreviation']}: Pred {row['PredictedLapTime (s)']:.3f}s
(P {int(row['PredictedPosition'])}) | "f"Actual {row['LapTime (s)']:.3f}s
({actual_pos})")

```

20. 3rd Model : XGBoost (GridSearchCV)

```

# Define parameter grid
param_grid = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.7, 0.8, 1.0],
}

# GridSearchCV
grid_search_xgb = GridSearchCV(
    XGBRegressor(random_state=42, verbosity=0),

```

```

    param_grid,
    cv=5,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)
grid_search_xgb.fit(X_train, y_train)

print("Best Parameters:", grid_search_xgb.best_params_)
print("Best CV R²:", round(grid_search_xgb.best_score_, 4))

# Train model dengan best params
model_xgb_gs = XGBRegressor(
    **grid_search_xgb.best_params_,
    random_state=42,
    verbosity=0
)
model_xgb_gs.fit(X_train, y_train)

y_pred_xgb_gs = model_xgb_gs.predict(X_test)

# Add predictions to test data
results = test_data[['Year', 'Abbreviation', 'RacePace (s)', 'Position',
'Points', 'LapTime (s)']].copy()
results['PredictedLapTime (s)'] = y_pred_xgb_gs
results['PredictedPosition'] = results['PredictedLapTime (s)'].rank()

# Calculate model performance
mae = mean_absolute_error(y_test, y_pred_xgb_gs)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_xgb_gs))
r2 = r2_score(y_test, y_pred_xgb_gs)

```

```

print(f"\nModel Performance:")
print(f"MAE: {mae:.3f} seconds")
print(f"RMSE: {rmse:.3f} seconds")
print(f"R2: {r2:.3f}")

# Display predictions vs actual
print("\nPredicted vs Actual Race Pace 2025 XGBoost (GridSearchCV) :")
results_sorted = results.sort_values('PredictedLapTime (s)')
for idx, row in results_sorted.iterrows():
    actual_pos = f"P {int(row['Position'])}" if not pd.isna(row['Position'])
    else "DNF"
    print(f"{row['Abbreviation']}: Pred {row['PredictedLapTime (s)']:.3f}s
(P {int(row['PredictedPosition'])}) | "f"Actual {row['LapTime (s)']:.3f}s
({actual_pos})")

```

21. 3rd Model : XGBoost (Optuna)

```

def objective(trial):
    param = {
        "max_depth": trial.suggest_int("max_depth", 3, 12),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3,
log=True),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0),
        "lambda": trial.suggest_float("lambda", 1e-3, 10.0, log=True),
        "objective": "reg:squarederror",
    }
    model_xgb_o = xgb.XGBRegressor(**param, n_estimators=500)
    model_xgb_o.fit(X_train, y_train, eval_set=[(X_test, y_test)], ver-
bose=False)

```

```

return model_xgb_o.score(X_test, y_test)

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=50)
print(study.best_params)

best_params = study.best_params
best_model = XGBRegressor(**best_params)
best_model.fit(X_train, y_train)

y_pred_xgb_o = best_model.predict(X_test)

# Add predictions to test data
results = test_data[['Year', 'Abbreviation', 'RacePace (s)', 'Position',
'Points', 'LapTime (s)']].copy()
results['PredictedLapTime (s)'] = y_pred_xgb_o
results['PredictedPosition'] = results['PredictedLapTime (s)'].rank()

# Calculate model performance
mae = mean_absolute_error(y_test, y_pred_xgb_o)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_xgb_o))
r2 = r2_score(y_test, y_pred_xgb_o)

print(f"\nModel Performance:")
print(f"MAE: {mae:.3f} seconds")
print(f"RMSE: {rmse:.3f} seconds")
print(f"R2: {r2:.3f}")

# Display predictions vs actual
print("\nPredicted vs Actual Race Pace 2025 XGBoost (Optuna):")
results_sorted = results.sort_values('PredictedLapTime (s)')

```

```

for idx, row in results_sorted.iterrows():
    actual_pos = f'P {int(row['Position'])}' if not pd.isna(row['Position'])
    else "DNF"
    print(f'{row['Abbreviation']}: Pred {row['PredictedLapTime (s)']:.3f}s
(P {int(row['PredictedPosition'])}) | "f"Actual {row['LapTime (s)']:.3f}s
({actual_pos})')

```

22. 4th Model : Gradient Boosting (Default Parameter)

```

pip install catboost
from catboost import CatBoostRegressor

# Train model CatBoost (Default)
model_catboost = CatBoostRegressor(
    random_state=42,
    verbose=False
)
model_catboost.fit(X_train, y_train)

y_pred_catboost = model_catboost.predict(X_test)

# Add predictions to test data
results = test_data[['Year', 'Abbreviation', 'RacePace (s)', 'Position',
'Points', 'LapTime (s)']].copy()
results['PredictedLapTime (s)'] = y_pred_catboost
results['PredictedPosition'] = results['PredictedLapTime (s)'].rank()

# Calculate model performance
mae = mean_absolute_error(y_test, y_pred_catboost)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_catboost))
r2 = r2_score(y_test, y_pred_catboost)

```

```

print(f"\nModel Performance:")
print(f"MAE: {mae:.3f} seconds")
print(f"RMSE: {rmse:.3f} seconds")
print(f'R2: {r2:.3f}')
# Display predictions vs actual
print("\nPredicted vs Actual Race Pace 2025 CatBoost (Default Parameter):")
results_sorted = results.sort_values('PredictedLapTime (s)')
for idx, row in results_sorted.iterrows():
    actual_pos = f"P {int(row['Position'])}" if not pd.isna(row['Position'])
    else "DNF"
    print(f'{row['Abbreviation']}: Pred {row['PredictedLapTime (s)']:.3f}s
    (P {int(row['PredictedPosition'])}) | "f"Actual {row['LapTime (s)']:.3f}s
    ({actual_pos})')

```

23. CatBoost (GridSearchCV)

```

# Define parameter grid
param_grid = {
    'iterations': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1],
    'depth': [4, 6, 8],
    'l2_leaf_reg': [1, 3, 5],
}

# GridSearchCV
grid_search_catboost = GridSearchCV(
    CatBoostRegressor(random_state=42, verbose=False),
    param_grid,
    cv=5,

```

```

        scoring='r2',
        n_jobs=-1,
        verbose=1
    )
    grid_search_catboost.fit(X_train, y_train)

    print("Best Parameters:", grid_search_catboost.best_params_)
    print("Best CV R²:", round(grid_search_catboost.best_score_, 4))

# Train model dengan best params
    model_catboost_gs = CatBoostRegressor(
        iterations=200,
        learning_rate=0.05,
        depth=4,
        l2_leaf_reg=1,
        random_state=42,
        verbose=False
    )
    model_catboost_gs.fit(X_train, y_train)

    y_pred_catboost_gs = model_catboost_gs.predict(X_test)

# Add predictions to test data
    results_gs = test_data[['Year', 'Abbreviation', 'RacePace (s)', 'Position',
        'Points', 'LapTime (s)']].copy()
    results_gs['PredictedLapTime (s)'] = y_pred_catboost_gs
    results_gs['PredictedPosition'] = results_gs['PredictedLapTime (s)'].rank()

# Calculate model performance
    mae = mean_absolute_error(y_test, y_pred_catboost_gs)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred_catboost_gs))

```

```

r2 = r2_score(y_test, y_pred_catboost_gs)

print(f"\nModel Performance (CatBoost + GridSearchCV):")
print(f"MAE: {mae:.3f} seconds")
print(f"RMSE: {rmse:.3f} seconds")
print(f"R2: {r2:.3f}")

# Display predictions vs actual
print("\nPredicted vs Actual Race Pace 2025 CatBoost (Default Parameter):")
results_sorted_gs = results_gs.sort_values('PredictedLapTime (s)')
for idx, row in results_sorted_gs.iterrows():
    actual_pos = f"P {int(row['Position'])}" if not pd.isna(row['Position'])
    else "DNF"
    print(f"{row['Abbreviation']}: Pred {row['PredictedLapTime (s)']:.3f}s
(P {int(row['PredictedPosition'])}) | "f"Actual {row['LapTime (s)']:.3f}s
({actual_pos})")

```

24. 4th Model : CatBoost (Optuna)

```

# Define objective function
def objective(trial):
    params = {
        'iterations': trial.suggest_int('iterations', 100, 500),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3,
log=True),
        'depth': trial.suggest_int('depth', 3, 10),
        'l2_leaf_reg': trial.suggest_float('l2_leaf_reg', 1e-3, 10.0,
log=True),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bylevel': trial.suggest_float('colsample_bylevel', 0.5, 1.0),

```



```

    }
    model = CatBoostRegressor(**params, random_state=42, verbose=False)
    model.fit(X_train, y_train)
    return r2_score(y_test, model.predict(X_test))

# Run Optuna study
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50, show_progress_bar=True)

print("Best Parameters:", study.best_params)
print("Best R²:", round(study.best_value, 4))

# Train model dengan best params
model_catboost_optuna = CatBoostRegressor(
    iterations=280,
    learning_rate=0.1118,
    depth=4,
    l2_leaf_reg=5.5314,
    subsample=0.8280,
    colsample_bylevel=0.9613,
    random_state=42,
    verbose=False
)
model_catboost_optuna.fit(X_train, y_train)

y_pred_catboost_optuna = model_catboost_optuna.predict(X_test)

# Add predictions to test data
results_opt = test_data[['Year', 'Abbreviation', 'RacePace (s)', 'Position',
'Points', 'LapTime (s)']].copy()

```

```

results_opt['PredictedLapTime (s)'] = y_pred_catboost_optuna
results_opt['PredictedPosition'] = results_opt['PredictedLapTime
(s)'].rank()

# Calculate model performance
mae = mean_absolute_error(y_test, y_pred_catboost_optuna)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_catboost_optuna))
r2 = r2_score(y_test, y_pred_catboost_optuna)

print(f"\nModel Performance (CatBoost + Optuna):")
print(f"MAE: {mae:.3f} seconds")
print(f"RMSE: {rmse:.3f} seconds")
print(f"R2: {r2:.3f}")

# Display predictions vs actual
print("\nPredicted vs Actual Race Pace 2025 CatBoost (Optuna):")
results_sorted_opt = results_opt.sort_values('PredictedLapTime (s)')
for idx, row in results_sorted_opt.iterrows():
    actual_pos = f"P {int(row['Position'])}" if not pd.isna(row['Position'])
    else "DNF"
    print(f"{row['Abbreviation']}: Pred {row['PredictedLapTime (s)']:.3f}s
(P {int(row['PredictedPosition'])}) | "f"Actual {row['LapTime (s)']:.3f}s
({actual_pos})")

```

25. 5th Model : LightGBM (Default Parameter)

```

import lightgbm as lgb
from lightgbm import LGBMRegressor

# Train model LightGBM (Default)
model_lgbm = LGBMRegressor(

```

```

        random_state=42
    )
    model_lgbm.fit(X_train, y_train)

y_pred_lgbm = model_lgbm.predict(X_test)

# Add predictions to test data
results = test_data[['Year', 'Abbreviation', 'RacePace (s)', 'Position',
'Points', 'LapTime (s)']].copy()
results['PredictedLapTime (s)'] = y_pred_lgbm
results['PredictedPosition'] = results['PredictedLapTime (s)'].rank()

# Calculate model performance
mae = mean_absolute_error(y_test, y_pred_lgbm)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_lgbm))
r2 = r2_score(y_test, y_pred_lgbm)

print(f"\nModel Performance:")
print(f"MAE: {mae:.3f} seconds")
print(f"RMSE: {rmse:.3f} seconds")
print(f"R2: {r2:.3f}")

# Display predictions vs actual
print("\nPredicted vs Actual Race Pace 2025:")
results_sorted = results.sort_values('PredictedLapTime (s)')
for idx, row in results_sorted.iterrows():
    actual_pos = f"P {int(row['Position'])}" if not pd.isna(row['Position'])
    else "DNF"
    print(f"{row['Abbreviation']}: Pred {row['PredictedLapTime (s)']:.3f}s
(P {int(row['PredictedPosition'])}) | "f"Actual {row['LapTime (s)']:.3f}s
({actual_pos})")

```

26. 5th Model : LightGBM (GridSearchCV)

```
# Define parameter grid
param_grid = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7],
    'num_leaves': [20, 31, 50],
    'subsample': [0.8, 1.0],
}

# GridSearchCV
grid_search_lgbm = GridSearchCV(
    LGBMRegressor(random_state=42, verbose=-1),
    param_grid,
    cv=5,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)
grid_search_lgbm.fit(X_train, y_train)

print("Best Parameters:", grid_search_lgbm.best_params_)
print("Best CV R2:", round(grid_search_lgbm.best_score_, 4))

# Train model dengan best params
model_lgbm_gs = LGBMRegressor(
    n_estimators=100,
    learning_rate=0.05,
    max_depth=3,
    num_leaves=20,
```

```

        subsample=0.8,
        random_state=42,
        verbose=-1
    )
    model_lgbm_gs.fit(X_train, y_train)

y_pred_lgbm_gs = model_lgbm_gs.predict(X_test)

# Add predictions to test data
results_gs = test_data[['Year', 'Abbreviation', 'RacePace (s)', 'Position',
'Points', 'LapTime (s)']].copy()
results_gs['PredictedLapTime (s)'] = y_pred_lgbm_gs
results_gs['PredictedPosition'] = results_gs['PredictedLapTime (s)'].rank()

# Calculate model performance
mae = mean_absolute_error(y_test, y_pred_lgbm_gs)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_lgbm_gs))
r2 = r2_score(y_test, y_pred_lgbm_gs)

print(f"\nModel Performance (LightGBM + GridSearchCV):")
print(f"MAE: {mae:.3f} seconds")
print(f"RMSE: {rmse:.3f} seconds")
print(f"R2: {r2:.3f}")

# Display predictions vs actual
print("\nPredicted vs Actual Race Pace 2025:")
results_sorted_gs = results_gs.sort_values('PredictedLapTime (s)')
for idx, row in results_sorted_gs.iterrows():
    actual_pos = f"P {int(row['Position'])}" if not pd.isna(row['Position'])
    else "DNF"

```

```

print(f'{row["Abbreviation"]}: Pred {row["PredictedLapTime (s)"]:.3f}s
(P {int(row["PredictedPosition"])} | "f"Actual {row["LapTime (s)"]:.3f}s
({actual_pos})")

```

27. 5th Model : LightGBM (Optuna)

```

# Define objective function
def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3,
log=True),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'num_leaves': trial.suggest_int('num_leaves', 20, 100),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0),
        'reg_alpha': trial.suggest_float('reg_alpha', 1e-3, 10.0, log=True),
        'reg_lambda': trial.suggest_float('reg_lambda', 1e-3, 10.0,
log=True),
    }
    model = LGBMRegressor(**params, random_state=42, verbose=-1)
    model.fit(X_train, y_train)
    return r2_score(y_test, model.predict(X_test))

# Run Optuna study
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50, show_progress_bar=True)

print("Best Parameters:", study.best_params)
print("Best R²:", round(study.best_value, 4))

```

```

# Train model dengan best params
model_lgbm_optuna = LGBMRegressor(
    n_estimators=125,
    learning_rate=0.0773,
    max_depth=8,
    num_leaves=25,
    subsample=0.6352,
    colsample_bytree=0.9286,
    reg_alpha=0.0089,
    reg_lambda=0.0032,
    random_state=42,
    verbose=-1
)
model_lgbm_optuna.fit(X_train, y_train)

y_pred_lgbm_optuna = model_lgbm_optuna.predict(X_test)

# Add predictions to test data
results_opt = test_data[['Year', 'Abbreviation', 'RacePace (s)', 'Position',
'Points', 'LapTime (s)']].copy()
results_opt['PredictedLapTime (s)'] = y_pred_lgbm_optuna
results_opt['PredictedPosition'] = results_opt['PredictedLapTime
(s)'].rank()

# Calculate model performance
mae = mean_absolute_error(y_test, y_pred_lgbm_optuna)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_lgbm_optuna))
r2 = r2_score(y_test, y_pred_lgbm_optuna)

print(f"\nModel Performance (LightGBM + Optuna):")
print(f"MAE: {mae:.3f} seconds")

```

```

print(f'RMSE: {rmse:.3f} seconds')
print(f'R2: {r2:.3f}')

# Display predictions vs actual
print("\nPredicted vs Actual Race Pace 2025:")
results_sorted_opt = results_opt.sort_values('PredictedLapTime (s)')
for idx, row in results_sorted_opt.iterrows():
    actual_pos = f'P {int(row['Position'])}' if not pd.isna(row['Position'])
    else "DNF"
    print(f'{row['Abbreviation']}: Pred {row['PredictedLapTime (s)']:.3f}s
(P {int(row['PredictedPosition'])}) | "f'Actual {row['LapTime (s)']:.3f}s
({actual_pos})")

```


Lampiran 3. Output Program

OUTPUT PROGRAM

1. Hasil Pengambilan Data dari FastF1API

	Year	Abbreviation	LapTime (s)	Sector1Time (s)	Sector2Time (s)	Sector3Time (s)	TotalSectorTime (s)	BestQuali (s)	Position	GridPosition	Points	RacePace (s)
0	2022	ALB	84.996072	29.640159	33.274116	22.081797	84.996072	80.859	12	17	0	84.437
1	2022	ALO	84.822661	30.189129	32.753758	21.879774	84.822661	78.939	19	9	0	84.425
2	2022	BOT	85.038333	29.915957	33.052058	22.070319	85.038333	78.401	10	6	1	84.422
3	2022	GAS	84.990087	29.721623	33.299333	21.969130	84.990087	79.672	11	14	0	84.349
4	2022	HAM	83.503757	29.621171	32.265057	21.617529	83.503757	78.084	2	3	18	82.921

2. Hasil Data Preprocessing

```
Dataset Info:
Shape: (80, 12)
Years: [2022 2023 2024 2025]
Drivers: ['ALB' 'ALO' 'BOT' 'GAS' 'HAM' 'LAT' 'LEC' 'MAG' 'MSC' 'NOR' 'OCO' 'PER'
          'RIC' 'RUS' 'SAI' 'STR' 'TSU' 'VER' 'VET' 'ZHO' 'HUL' 'PIA' 'COL' 'LAW'
          'BEA' 'ANT' 'BOR' 'HAD']

Missing values:
Year          0
Abbreviation  0
LapTime (s)   0
Sector1Time (s) 0
Sector2Time (s) 0
Sector3Time (s) 0
TotalSectorTime (s) 0
BestQuali (s)  0
Position      0
GridPosition  0
Points        0
RacePace (s)  0
dtype: int64
```

3. Hasil Data Sirkuit

```
Circuit data loaded: 24 circuits
CircuitName altitude_m drs_zones tire_degradation
Mexico City          2285           2             6
```

4. Hasil Data Cuaca

```
Weather data:
Year CircuitName air_temp_c track_temp_c humidity_pct wind_speed_ms rainfall_mm is_wet_race pressure_hpa
2022 Mexico City      22.0           38.0           45           3.2           0.0           0           775.0
2023 Mexico City      19.5           32.0           55           2.8           0.0           0           778.0
2024 Mexico City      21.0           35.0           50           3.0           0.0           0           776.0
2025 Mexico City      20.5           34.0           48           3.5           0.0           0           777.0
```

5. Hasil Feature Engineering Performa Tim dan Driver

Driver Season Stats (sample):															
Year	Abbreviation	races_count	wins	podium	top5	avg_position	total_points	avg_grid_pos	position_gain_avg	dnf_count	win_rate	podium_rate	top5_rate	avg_position_gain	points_per_race
2022	ALB	1	0	0	0	12.0	0	17.0	17.0	0	0.0	0.0	0.0	5.0	0.0
2022	ALO	1	0	0	0	19.0	0	9.0	9.0	0	0.0	0.0	0.0	-10.0	0.0
2022	BOT	1	0	0	0	18.0	1	6.0	6.0	0	0.0	0.0	0.0	-4.0	1.0
2022	GAS	1	0	0	0	11.0	0	14.0	14.0	0	0.0	0.0	0.0	3.0	0.0
2022	HAM	1	0	1	1	2.0	18	3.0	3.0	0	0.0	1.0	1.0	1.0	18.0
2022	LAT	1	0	0	0	18.0	0	18.0	18.0	0	0.0	0.0	0.0	0.0	0.0
2022	LEC	1	0	0	0	6.0	8	7.0	7.0	0	0.0	0.0	0.0	1.0	8.0
2022	MAG	1	0	0	0	17.0	0	19.0	19.0	0	0.0	0.0	0.0	2.0	0.0
2022	MSC	1	0	0	0	16.0	0	15.0	15.0	0	0.0	0.0	0.0	-1.0	0.0
2022	MON	1	0	0	0	9.0	2	8.0	8.0	0	0.0	0.0	0.0	-1.0	2.0

6. Hasil Penggabungan Fitur Baru

```

Menambahkan fitur karakteristik Mexico City sebagai konstanta...
Ditambahkan 14 fitur sirkuit Mexico City
✓ Weather data merged
✓ Driver stats merged

Shape setelah penambahan fitur: (80, 47)
Kolom baru yang ditambahkan:
+ circuit_length_km
+ num_corners
+ num_straights
+ drs_zones
+ altitude_m
+ circuit_type
+ traction_demand
+ braking_demand
+ downforce_level
+ overtaking_difficulty
+ tire_degradation
+ safety_car_probability
+ lap_record_s
+ avg_speed_kmh
+ air_temp_c
+ track_temp_c
+ humidity_pct
+ wind_speed_ms
+ rainfall_mm
+ is_wet_race
+ pressure_hpa
+ races_count
+ wins
+ podiums
+ top5
+ avg_position
+ total_points
+ avg_grid_pos
+ position_gain_avg
+ dnf_count
+ win_rate
+ podium_rate
+ top5_rate
+ avg_position_gain

```

7. Hasil Ringkasan Fitur

```

=====
RINGKASAN FITUR YANG DIGUNAKAN:
Base features      : 7 fitur
Circuit features   : 12 fitur
Weather features   : 7 fitur
Altitude features  : 4 fitur
Driver features    : 7 fitur
Derived features    : 5 fitur

-----
TOTAL              : 42 fitur
=====

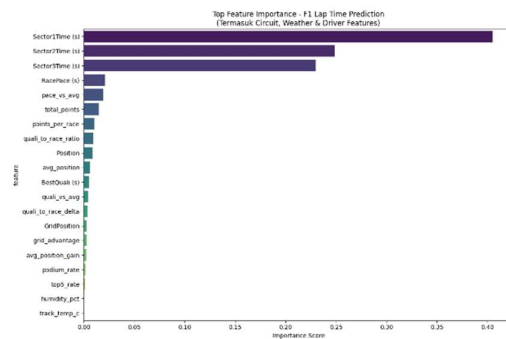
Fitur yang tidak tersedia (perlu ditambahkan ke CSV):

```

8. Hasil Data Setelah *Scaling*

	Sector1Time (s)	Sector2Time (s)	Sector3Time (s)	BestQuali (s)	GridPosition	Position	RacePace (s)	circuit_length_km	num_corners	num_straights	drs_zones	altitude_m	traction_demand	braking_demand	downforce_level	overtaking_diffic
0	-0.755628	0.306211	0.013423	2.090307	1.270026	0.383140	0.301265	-8.881784e-16	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	-0.367918	-0.530805	-0.418560	0.578616	-0.131382	1.680871	0.296024	-8.881784e-16	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	-0.560846	-0.050978	-0.011121	0.155027	-0.656910	0.012359	0.294714	-8.881784e-16	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	-0.698094	0.346774	-0.227491	1.155736	0.744498	0.197750	0.262833	-8.881784e-16	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	-0.769038	-1.316902	-0.979315	-0.094559	-1.182438	-1.470762	-0.360805	-8.881784e-16	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

9. Hasil Visualiasi Feature Importance



10. Hasil Evaluasi Metrik Model

Model Performance:
MAE: 0.872 seconds
RMSE: 1.154 seconds
R2: 0.846

11. Hasil Perbandingan Nilai Prediksi dengan Nilai Aktual

Predicted vs Actual Lap Time 2025 Random Forest (Default Parameter):	
NOR: Pred 83.704s (P1)	Actual 82.769s (P1)
LEC: Pred 83.838s (P2)	Actual 83.178s (P2)
VER: Pred 83.892s (P3)	Actual 83.169s (P3)
PIA: Pred 83.896s (P4)	Actual 83.286s (P5)
BEA: Pred 83.940s (P5)	Actual 83.293s (P4)
ANT: Pred 84.015s (P6)	Actual 83.383s (P6)
RUS: Pred 84.293s (P7)	Actual 83.435s (P7)
SAI: Pred 84.507s (P8)	Actual 83.667s (P17)
HAM: Pred 84.559s (P9)	Actual 83.540s (P8)
STR: Pred 84.631s (P10)	Actual 83.872s (P14)
OCO: Pred 84.631s (P11)	Actual 83.757s (P9)
BOR: Pred 84.637s (P12)	Actual 83.755s (P10)
TSU: Pred 84.656s (P13)	Actual 83.820s (P11)
ALB: Pred 84.667s (P14)	Actual 83.851s (P12)
HAD: Pred 84.680s (P15)	Actual 83.965s (P13)
COL: Pred 84.805s (P16)	Actual 84.391s (P16)
GAS: Pred 84.840s (P17)	Actual 84.469s (P15)
HUL: Pred 84.926s (P18)	Actual 84.590s (P19)
ALO: Pred 85.066s (P19)	Actual 84.608s (P18)
LAW: Pred 92.973s (P20)	Actual 97.037s (P20)

Lampiran 4. E-KRS



UNIVERSITAS GUNADARMA

aa49d6

**KARTU RENCANA STUDI (KRS)
SEMESTER ATA 2025 / 2026**

8.

2
19

20 April 2026

Dr. Febriani

BAAK

Cek Validasi:
<https://krs.gunadarma.ac.id/validasi/aa49d6>

***KRS ini wajib dicetak/diprint oleh mahasiswa**

Lampiran 5. Surat Persetujuan ACC



UNIVERSITAS GUNADARMA

SK No. 52/CKT/19Kep/1998

Program Diploma (D2) Manajemen Informatika, Teknik Komputer, Akutansi Komputer, Manajemen Keuangan, Manajemen Pemasaran, Kehutanan

Program Sarjana (S1) Sistem Informasi, Sistem Komputer, Teknik Informatika, Teknik Elektro, Teknik Mesin, Teknik Industri, Akutansi, Manajemen, Teknik Arsitektur, Teknik Sipil, Psikologi, Sastra Inggris, Ilmu Komunikasi

Program Sarjana (S2) Sistem Informasi, Manajemen, Teknik Elektro, Sastra Inggris, Psikologi, Teknik Sipil, Teknik Mesin

Program Sarjana (S3) Ilmu Ekonomi, Teknologi Informasi, Ilmu Psikologi

SURAT PERSETUJUAN

UJIAN SIDANG PENULISAN PENELITIAN / KERJA PRAKTEK

Yang bertanda tangan di bawah ini adalah Dosen Pembimbing dari :

Nama : KEVIN BUDIYAN SIDARTA
NPM : 10123585
Jenjang / Program Studi : S1 / Sistem Informasi
Pembimbing : Widya Khafa Nofa, SKom., MMSI
Tanggal Acc : 03 Juli 2026

Menyatakan bahwa mahasiswa tersebut di atas sudah dapat menyajikan Penulisan Penelitian / Kerja Praktek yang berjudul

Analisis Komparasi Model Machine Learning untuk Prediksi Pemenang Formula 1 : Studi Kasus 2025 Mexico City Grand Prix

Depok, 03 Juli 2026
Dosen Pembimbing
Widya Khafa Nofa, SKom., MMSI.

Lampiran 6. Isian Sertifikat Setara Sarjana Muda

ISIAN SERTIFIKAT SETARA SARJANA MUDA

No. Sertifikat :(*)

Tanggal Lulus :(*)

NAMA : Kevin Budiawan Sidarta

NPM : 10123585

Tempat, Tanggal Lahir

Alamat

Telepon

Pembimbing : Widya Khafa Nofa, Skom., MMSI

Judul Penulisan :ANALISIS KOMPARASI MODEL MACHINE LEARNING UNTUK
PREDIKSI PEMENANG FORMULA 1 : STUDI KASUS 2025 MEXICO CITY
GRAND PRIX



Foto harus HITAM PUTIH dan memakai baju putih

Pria : Berdasarkan tanpa jas

(*) Hanya diisi oleh bagian PI

Jakarta, Tanggal Juni 2026

Tanda Tangan Mahasiswa,

(. Budiawan Sidarta)

